signal-processing-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `signal_processing_ai999x.about()` for the same answer.

Every capability signal-processing-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import signal_processing_ai999x

result = signal_processing_ai999x.solve('period_from_frequency', f='50 Hz')
result.show()
# T = 0.02 s, omega = 314.159 rad/s
```

By topic

topic	capabilities
Analog filters	Magnitude of an analog Butterworth low-pass response, Minimum order of a analog Butterworth filter, Minimum order of a analog Chebyshev type I filter
Antennas	Gain of a parabolic dish (aperture) antenna
Communication links	Shannon capacity of an AWGN channel, Minimum SNR for a data rate (Shannon limit)
Convolution and correlation	Linear convolution of two sequences, Circular (cyclic) convolution, Cross-correlation and the lag of its peak
Decibels and levels	Power ratio to decibels, Decibels to power ratio, Amplitude (voltage) ratio to decibels, Decibels to amplitude (voltage) ratio, Power in watts to dBm and dBW, Power level in dBm to watts, Signal-to-noise ratio from signal and noise powers, Total power of several signals given in dBm
DFT and FFT	Frequency resolution of an N-point DFT, Frequency of a DFT bin, One bin of the discrete Fourier transform, DFT spectrum of a sequence (FFT), Inverse discrete Fourier transform
Digital communication	Bit error rate of BPSK (and coherent BFSK) in AWGN, Bit and symbol error rates of QPSK in AWGN, Bit error rates of DPSK and noncoherent BFSK, Symbol error rate of square M-QAM in AWGN, Symbol error rate of M-PSK in AWGN
Digital filters	Moving-average filter output, FIR filter output, IIR filter output (difference equation), Impulse response of a digital filter, Step response of a digital filter, Frequency response of a digital filter at one frequency, Group delay of a digital filter, DC gain of a digital filter, Minimum order of a digital (bilinear) Butterworth filter, Bilinear transform of a first-order low-pass filter
Filter design	Digital Butterworth filter design (bilinear transform), FIR low-pass filter design by the window method, Second-order IIR notch filter design
Fourier series and transforms	Harmonic amplitude of a square, triangle or sawtooth wave, Fourier transform of a rectangular pulse, Fourier transform of a decaying exponential, Total harmonic distortion, Analytic signal, Hilbert transform and envelope

topic	capabilities
Modulation	Power in an AM (DSB full-carrier) signal, AM modulation index from the envelope, FM spectrum: Carson bandwidth and Bessel sideband amplitudes
Noise and link budgets	Noise figure of cascaded stages (Friis formula), Free-space path loss, Received power of a radio link (Friis transmission equation)
Radar	Received echo power (radar range equation), Maximum detection range of a radar, Doppler shift of a radar echo
Sampling and signal basics	Period and angular frequency from frequency, Frequency and angular frequency from period, Nyquist rate, Alias (apparent) frequency after sampling, Mean, RMS, energy and average power of a sampled signal, RMS, peak-to-peak and rectified average of a sine wave, Signal-to-quantization-noise ratio of an ideal ADC
Transmission lines	Reflection coefficient, VSWR and return loss of a load, VSWR and return loss from the reflection coefficient, Reflection coefficient and return loss from the VSWR, Input impedance of a terminated lossless line, Quarter-wave impedance transformer, Characteristic impedance of a coaxial line
Wavelets	Haar discrete wavelet transform, Inverse Haar wavelet transform (one level)
Windows and spectral analysis	Main-lobe width of a spectral window, Coefficients of a window function, Periodogram (power spectral density estimate)
Z-transform and stability	Poles and stability of a second-order all-pole system, Poles, zeros and stability of a digital filter

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on

field	meaning
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
period_from_frequency	Period and angular frequency from frequency	f	T, omega
frequency_from_period	Frequency and angular frequency from period	T	f, omega
nyquist_rate	Nyquist rate	fmax	fs_min, Ts_max
alias_frequency	Alias (apparent) frequency after sampling	f, fs	f_alias
power_ratio_to_db	Power ratio to decibels	ratio	dB
db_to_power_ratio	Decibels to power ratio	dB	ratio
amplitude_ratio_to_db	Amplitude (voltage) ratio to decibels	ratio	dB
db_to_amplitude_ratio	Decibels to amplitude (voltage) ratio	dB	ratio
watts_to_dbm	Power in watts to dBm and dBW	P	dBm, dBW
dbm_to_watts	Power level in dBm to watts	dBm	P, dBW
snr_from_powers	Signal-to-noise ratio from signal and noise powers	Ps, Pn	snr_db, snr
signal_statistics	Mean, RMS, energy and average power of a sampled signal	x	rms, mean, energy, power, peak
sine_rms	RMS, peak-to-peak and rectified average of a sine wave	Vp	Vrms, Vpp, Vavg

id	operation	inputs	outputs
quantization_snr	Signal-to-quantization-noise ratio of an ideal ADC	bits	sqnr_db, dynamic_range_db
dbm_power_sum	Total power of several signals given in dBm	levels	total_dBm, total_W
frequency_resolution	Frequency resolution of an N-point DFT	fs, N	df, duration
bin_frequency	Frequency of a DFT bin	k, fs, N	f_k
dft_bin	One bin of the discrete Fourier transform	x, k	magnitude, phase, re, im
dft_spectrum	DFT spectrum of a sequence (FFT)	x	peak_magnitude, peak_bin, magnitude, phase, real, imag
inverse_dft	Inverse discrete Fourier transform	re_, im	real, imag
linear_convolution	Linear convolution of two sequences	x, h	y, length
circular_convolution	Circular (cyclic) convolution	x, h	y
cross_correlation	Cross-correlation and the lag of its peak	x, y	peak_lag, peak_value, r, lags
moving_average	Moving-average filter output	x, M	y
fir_filter_output	FIR filter output	b, x	y
iir_filter_output	IIR filter output (difference equation)	b, a, x	y
impulse_response	Impulse response of a digital filter	b, a, n	h
step_response	Step response of a digital filter	b, a, n	s
frequency_response	Frequency response of a digital filter at one frequency	b, f, fs, a	magnitude, magnitude_db, phase
group_delay	Group delay of a digital filter	b, f, fs, a	samples, seconds
dc_gain	DC gain of a digital filter	b, a	gain, gain_db
butterworth_magnitude	Magnitude of an analog Butterworth low-pass response	N, fc, f	magnitude, attenuation_db
butterworth_order_analog	Minimum order of a analog Butterworth filter	fp, fstop, Rp, As	order, type
butterworth_order_digital	Minimum order of a digital (bilinear) Butterworth filter	fp, fstop, fs, Rp, As	order, type
chebyshev1_order_analog	Minimum order of a analog Chebyshev type I filter	fp, fstop, Rp, As	order, type
second_order_poles	Poles and stability of a second-order all-pole system	a1, a2	radius, p1, p2, stability
poles_zeros	Poles, zeros and stability of a digital filter	b, a	max_pole_radius, zeros, poles, stability
bilinear_rc_lowpass	Bilinear transform of a first-order low-pass filter	fc, fs	b0, b1, a1

id	operation	inputs	outputs
window_mainlobe_width	Main-lobe width of a spectral window	window, N, fs	width_bins, first_null_bins, width_hz
window_coefficients	Coefficients of a window function	window, N, form	w, coherent_gain
fourier_series_harmonic	Harmonic amplitude of a square, triangle or sawtooth wave	waveform, A, n	amplitude, relative, rms
rect_pulse_spectrum	Fourier transform of a rectangular pulse	A, tau, f	X, magnitude, first_null
exponential_pulse_spectrum	Fourier transform of a decaying exponential	A, tau, f	magnitude, phase, re, im
thd	Total harmonic distortion	V	thd, percent, db
analytic_envelope	Analytic signal, Hilbert transform and envelope	x	envelope, hilbert
periodogram	Periodogram (power spectral density estimate)	x, fs	peak_frequency, freqs, psd
am_power	Power in an AM (DSB full-carrier) signal	Pc, m	Pt, Psb, Pusb, efficiency
am_modulation_index	AM modulation index from the envelope	Vmax, Vmin	m, Ac
fm_spectrum	FM spectrum: Carson bandwidth and Bessel sideband amplitudes	df, fm, n	carson_bandwidth, beta, carrier, Jn, pair_power, power_in_carson
shannon_capacity	Shannon capacity of an AWGN channel	B, snr_db	C, efficiency
shannon_min_snr	Minimum SNR for a data rate (Shannon limit)	C, B	snr_db, snr, ebn0_min_db
bpsk_ber	Bit error rate of BPSK (and coherent BFSK) in AWGN	ebn0_db	ber, ber_bfsk
qpsk_error_rates	Bit and symbol error rates of QPSK in AWGN	ebn0_db	ber, ser
noncoherent_ber	Bit error rates of DPSK and noncoherent BFSK	ebn0_db	ber_dpsk, ber_ncfsk
mqam_symbol_error	Symbol error rate of square M-QAM in AWGN	M, ebn0_db	ser, ber_approx
mpsk_symbol_error	Symbol error rate of M-PSK in AWGN	M, ebn0_db	ser, ber_approx
noise_figure_cascade	Noise figure of cascaded stages (Friis formula)	nf_db, gain_db	nf_db, F, Te, gain_db
free_space_path_loss	Free-space path loss	d, f	loss_db, loss, wavelength
friis_received_power	Received power of a radio link (Friis transmission equation)	Pt, Gt_db, Gr_db, f, d	Pr_dbm, Pr
dish_antenna_gain	Gain of a parabolic dish (aperture) antenna	D, f, eta	G_dbi, G, Ae
radar_received_power	Received echo power (radar range equation)	Pt, G_db, f, sigma, R	Pr, Pr_dbm

id	operation	inputs	outputs
radar_max_range	Maximum detection range of a radar	Pt, G_db, f, sigma, Pmin	Rmax
radar_doppler	Doppler shift of a radar echo	v, f	fd, wavelength
reflection_coefficient	Reflection coefficient, VSWR and return loss of a load	RL, Z0, XL	gamma_mag, gamma_angle_deg, gamma_re, gamma_im, vswr, return_loss_db, mismatch_loss_db
vswr_from_reflection	VSWR and return loss from the reflection coefficient	gamma	vswr, return_loss_db, mismatch_loss_db
reflection_from_vswr	Reflection coefficient and return loss from the VSWR	vswr	gamma, return_loss_db, mismatch_loss_db
input_impedance	Input impedance of a terminated lossless line	RL, Z0, length, XL	Rin, Xin, abs_zin
quarter_wave_transformer	Quarter-wave impedance transformer	RL, Z0, f, vf	Zt, length
coax_impedance	Characteristic impedance of a coaxial line	D, d, er	Z0, vf
butterworth_design	Digital Butterworth filter design (bilinear transform)	N, fc, fs, btype	b, a
fir_window_design	FIR low-pass filter design by the window method	N, fc, fs, window	h
notch_filter_design	Second-order IIR notch filter design	f0, Q, fs	b, a
haar_dwt	Haar discrete wavelet transform	x, levels	approximation, details
haar_idwt	Inverse Haar wavelet transform (one level)	cA, cD	x

Period and angular frequency from frequency — `period_from_frequency`

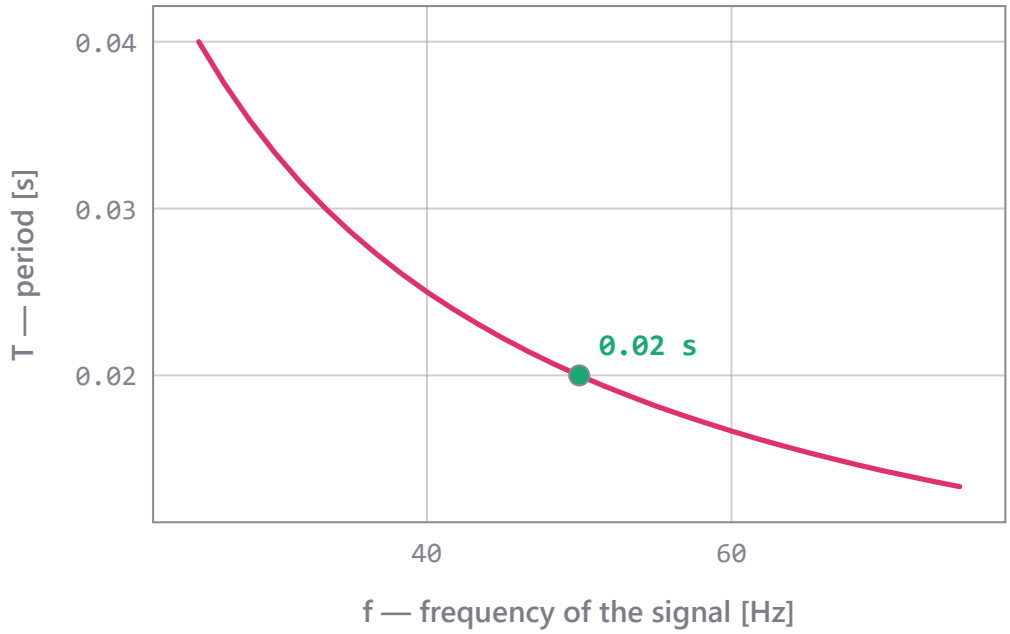
The period $T = 1/f$ and the angular frequency $\omega = 2pif$ of a periodic signal of frequency f.

$$T = \frac{1}{f}, \quad \omega = 2\pi f$$

input	meaning	condition
`f`	frequency of the signal	

output	meaning
`T`	period
`omega`	angular frequency

- **Method:** $T = 1/f$, $\omega = 2$ pi f
- **Assumes:** $f > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



T (period) against f (frequency of the signal) from 25 Hz to 75 Hz. 31 points computed by signal-processing-ai999x; marked: the worked example, f = 50 Hz, T = 0.02 s.

```
signal_processing_ai999x.solve('period_from_frequency', f=50.0)
# -> T = 0.02 s, omega = 314.159 rad/s   (SOLVED)
```

```
signal_processing_ai999x.solve('period_from_frequency', f=2000.0)
# -> T = 0.0005 s, omega = 12566.4 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the period of a signal with a frequency of 50 Hz?	SOLVED	T = 0.02 s, omega = 314.159 rad/s
Find the period and angular frequency of a 2 kHz signal.	SOLVED	T = 0.0005 s, omega = 12566.4 rad/s
What is the period of a signal with a frequency of -5 Hz?	INVALID_INPUT	Period and angular frequency from frequency: frequency must be a positive finite number.
What is the period of the signal?	NEEDS_INPUT	period_from_frequency needs f (frequency of the signal). Example: solve('period_from_frequency', f='50 Hz').

Frequency and angular frequency from period — `frequency_from_period`

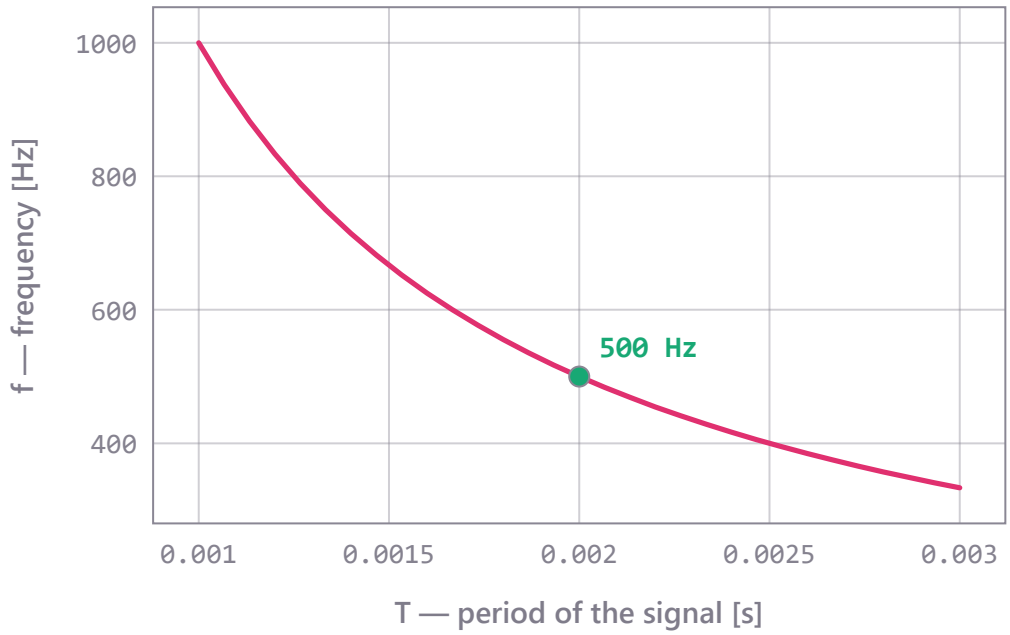
The frequency $f = 1/T$ and the angular frequency $\omega = 2pi/T$ of a periodic signal of period T.

$$f = \frac{1}{T}, \quad \omega = \frac{2\pi}{T}$$

input	meaning	condition
`T`	period of the signal	

output	meaning
`f`	frequency
`omega`	angular frequency

- **Method:** $f = 1/T, \omega = 2$ pi / T
- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f (frequency) against T (period of the signal) from 0.001 s to 0.003 s. 31 points computed by signal-processing-ai999x; marked: the worked example, T = 0.002 s, f = 500 Hz.

```
signal_processing_ai999x.solve('frequency_from_period', T=0.002)
# -> f = 500 Hz, omega = 3141.59 rad/s   (SOLVED)
```

```
signal_processing_ai999x.solve('frequency_from_period', T=2e-05)
# -> f = 50000 Hz, omega = 314159 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the frequency of a signal with a period of 2 ms?	SOLVED	f = 500 Hz, omega = 3141.59 rad/s
Find the frequency of a waveform whose period is 20 us.	SOLVED	f = 50000 Hz, omega = 314159 rad/s
What is the frequency of a signal with a period of 0 s?	INVALID_INPUT	Frequency and angular frequency from period: period must be a positive finite number.

Nyquist rate — `nyquist_rate`

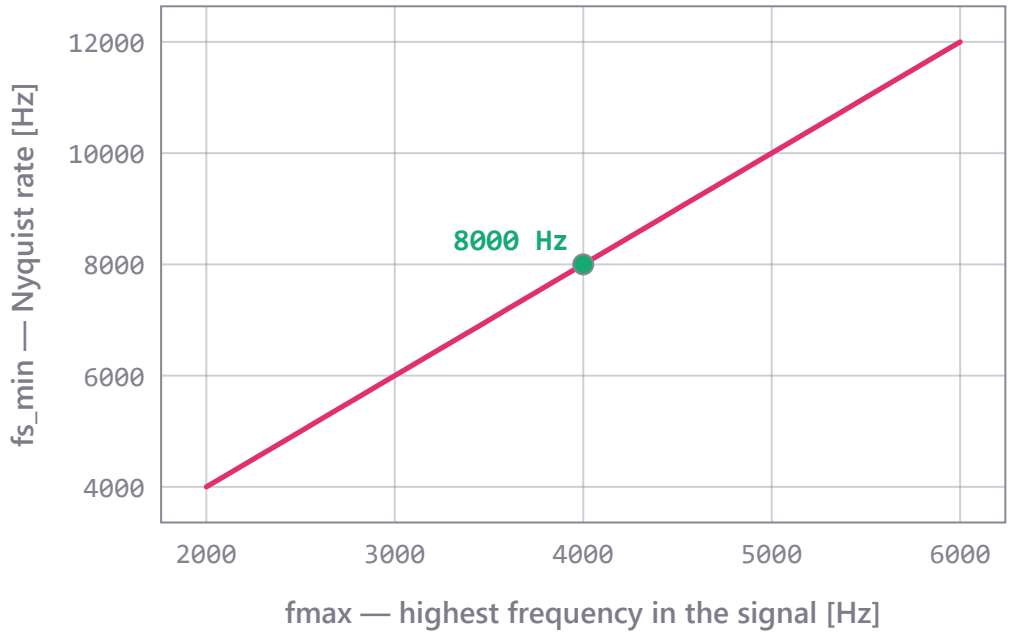
The minimum sampling rate 2*f_max that avoids aliasing of a baseband signal whose highest frequency is f_max.

$$f_{s,\min} = 2 f_{\max}$$

input	meaning	condition
`fmax`	highest frequency in the signal	

output	meaning
`fs_min`	Nyquist rate (minimum sampling rate)
`Ts_max`	longest sampling interval

- **Method:** sampling theorem: $fs \geq 2$ fmax
- **Assumes:** fmax > 0; a baseband (lowpass) signal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fs_min (Nyquist rate (minimum sampling rate)) against fmax (highest frequency in the signal) from 2000 Hz to 6000 Hz. 31 points computed by signal-processing-ai999x; marked: the worked example, fmax = 4000 Hz, fs_min = 8000 Hz.

```
signal_processing_ai999x.solve('nyquist_rate', fmax=4000.0)
# -> fs_min = 8000 Hz, Ts_max = 0.000125 s   (SOLVED)
```

```
signal_processing_ai999x.solve('nyquist_rate', fmax=20000.0)
# -> fs_min = 40000 Hz, Ts_max = 2.5e-05 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the Nyquist rate for a signal with a maximum frequency of 4 kHz?	SOLVED	fs_min = 8000 Hz, Ts_max = 0.000125 s
Find the minimum sampling rate for an audio signal band-limited to 20 kHz.	SOLVED	fs_min = 40000 Hz, Ts_max = 2.5e-05 s

question	outcome	result
What is the Nyquist rate of the signal?	NEEDS_INPUT	nyquist_rate needs fmax (highest frequency in the signal). Example: solve('nyquist_rate', fmax='4000 Hz').

Alias (apparent) frequency after sampling — `alias_frequency`

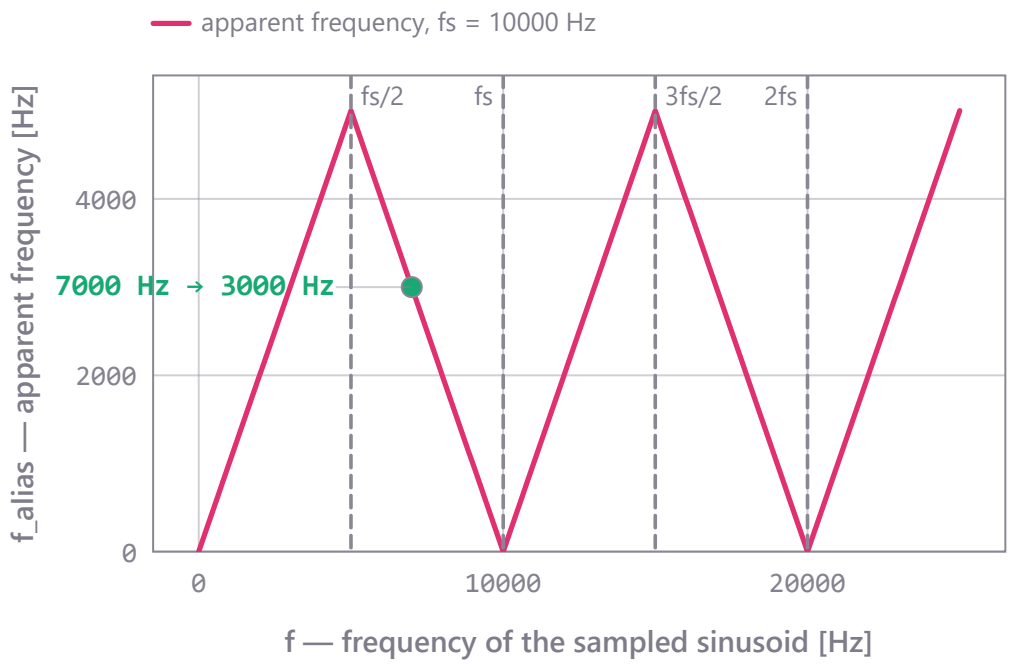
The frequency in [0, $fs/2$] at which a sinusoid of frequency f appears after sampling at rate fs.

$$f_a = |f - f_s \,\mathrm{round}(f/f_s)|$$

input	meaning	condition
`f`	frequency of the sampled sinusoid	
`fs`	sampling rate	

output	meaning
`f_alias`	apparent (principal alias) frequency

- **Method:** fold f about multiples of fs: |f - fs*round(f/fs)|
- **Assumes:** $f \geq 0$; $fs > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The frequency a sinusoid appears at after sampling at fs = 10000 Hz, for input frequencies 0 to 25000 Hz (201 runs of the package): the apparent frequency rises to fs/2 = 5000 Hz, then folds back down, a saw-tooth repeating every fs (dashed lines at multiples of fs/2, stated). Marked: f = 7000 Hz is seen at 3000 Hz.

```
signal_processing_ai999x.solve('alias_frequency', f=7000.0, fs=10000.0)
# -> f_alias = 3000 Hz   (SOLVED)
```

```
signal_processing_ai999x.solve('alias_frequency', f=1200.0, fs=1000.0)
# -> f_alias = 200 Hz   (SOLVED)
```

```
signal_processing_ai999x.solve('alias_frequency', f=300.0, fs=1000.0)
# -> f_alias = 300 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A 7 kHz tone is sampled at 10 kHz. What is the alias frequency?	SOLVED	f_alias = 3000 Hz
What is the apparent frequency of a 1.2 kHz sinusoid sampled at a sampling rate of 1 kHz?	SOLVED	f_alias = 200 Hz
A 7 kHz tone is sampled. What is the alias frequency?	NEEDS_INPUT	alias_frequency needs fs (sampling rate). Example: solve('alias_frequency', f='7000 Hz', fs='10000 Hz').

Power ratio to decibels — `power_ratio_to_db`

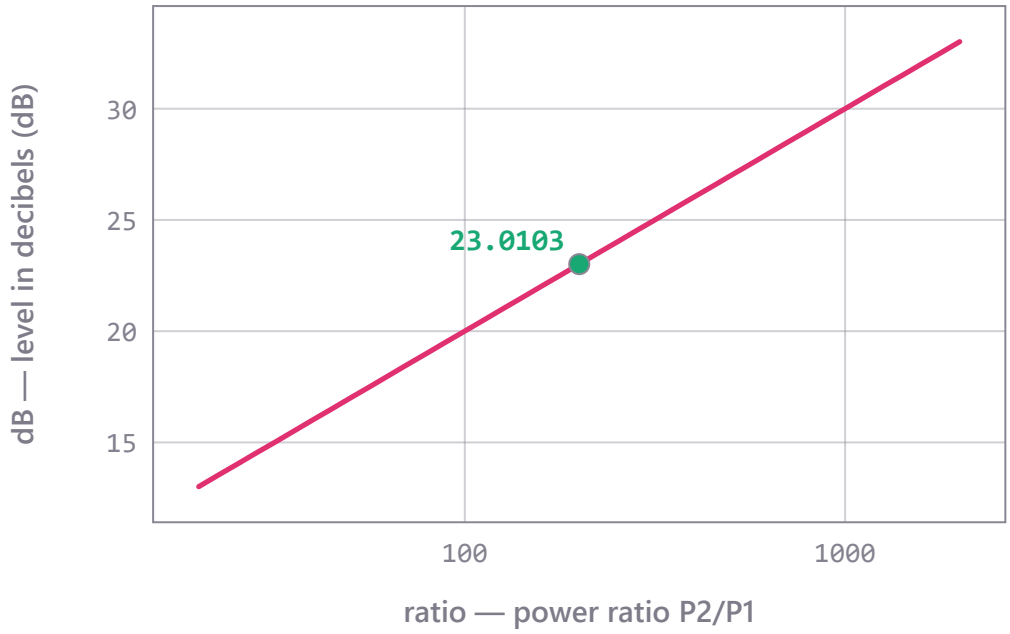
A power ratio expressed in decibels, $10log10$(P2/P1).

$$L = 10 \log_{10} \frac{P_2}{P_1} \text{ dB}$$

input	meaning	condition
`ratio`	power ratio P_2/P_1 (a positive number)	

output	meaning
`dB`	level in decibels (dB)

- **Method:** 10 log10(ratio)
- **Assumes:** ratio > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dB (level in decibels (dB)) against ratio (power ratio P2/P1 (a positive number)) from 20 to 2000 on a logarithmic axis. 41 points computed by signal-processing-ai999x; marked: the worked example, ratio = 200, dB = 23.0103.

```
signal_processing_ai999x.solve('power_ratio_to_db', ratio=200)
# -> dB = 23.0103   (SOLVED)
```

```
signal_processing_ai999x.solve('power_ratio_to_db', ratio=1000)
# -> dB = 30   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Express a power ratio of 200 in decibels.	SOLVED	dB = 23.0103
Convert a power gain of 1000 to dB.	SOLVED	dB = 30
Convert a power gain of -4 to dB.	INVALID_INPUT	Power ratio to decibels: power ratio must be a positive finite number.
Convert a power ratio to dB.	NEEDS_INPUT	power_ratio_to_db needs ratio (power ratio P2/P1 (a positive number)). Example: solve('power_ratio_to_db', ratio=200).

Decibels to power ratio — `db_to_power_ratio`

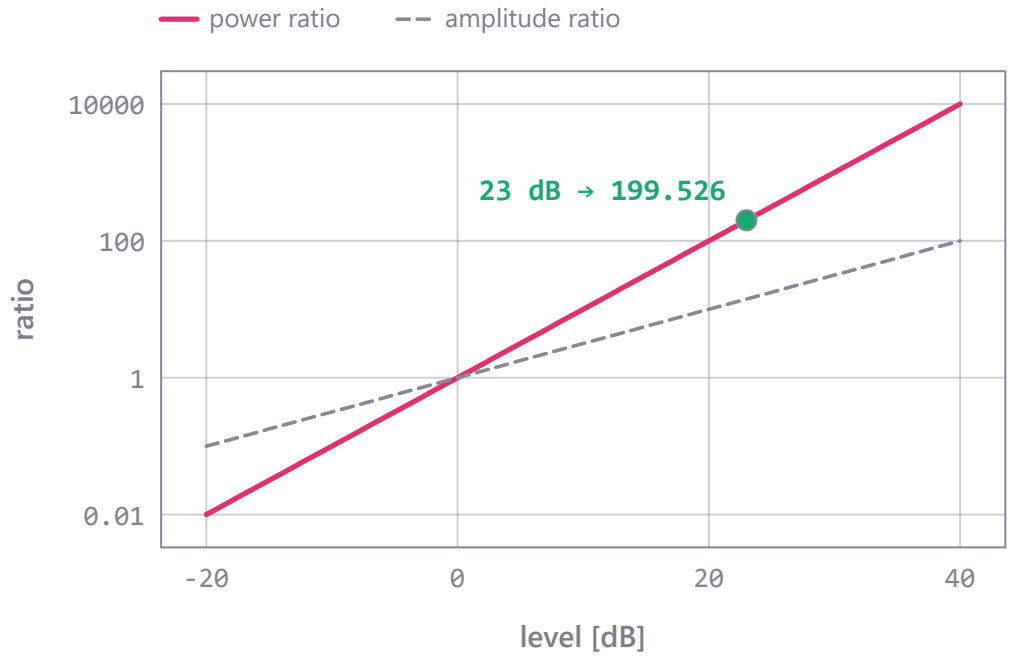
The power ratio $P_2/P_1 = 10^{L/10}$ that a level of L dB represents.

$$\frac{P_2}{P_1} = 10^{L/10}$$

input	meaning	condition
`dB`	level in decibels	

output	meaning
`ratio`	power ratio P_2/P_1

- **Method:** $10^{L/10}$
- **Assumes:** finite level
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The power ratio for levels from −20 to 40 dB (61 runs of the package, log axis), with the amplitude ratio dashed for comparison: 20 dB is a factor 10 in amplitude but 100 in power. Marked: 23 dB, ratio 199.526.

```
signal_processing_ai999x.solve('db_to_power_ratio', dB=23)
# -> ratio = 199.526   (SOLVED)
```

```
signal_processing_ai999x.solve('db_to_power_ratio', dB=30)
# -> ratio = 1000   (SOLVED)
```

```
signal_processing_ai999x.solve('db_to_power_ratio', dB=-3)
# -> ratio = 0.501187   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What power ratio corresponds to 23 dB?	SOLVED	ratio = 199.526
Convert 30 dB to a power ratio.	SOLVED	ratio = 1000
Convert -10 dB to a power ratio.	SOLVED	ratio = 0.1
What power ratio corresponds to the level?	NEEDS_INPUT	db_to_power_ratio needs dB (level in decibels). Example: solve('db_to_power_ratio', dB=23).

Amplitude (voltage) ratio to decibels — `amplitude_ratio_to_db`

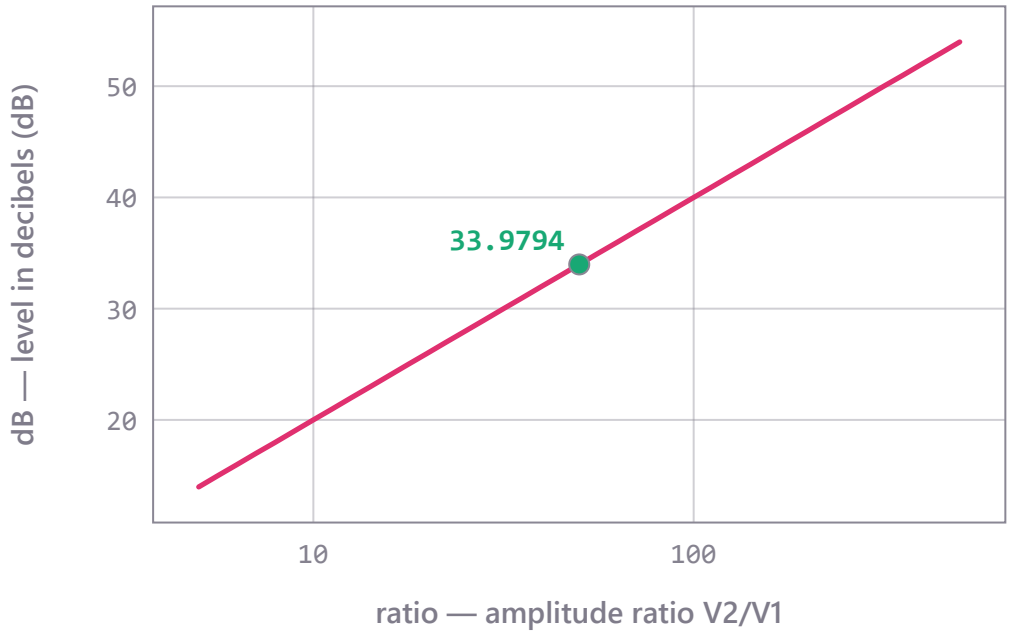
A voltage, current or field amplitude ratio in decibels, $20log10$(V2/V1).

$$L = 20 \log_{10} \frac{V_2}{V_1} \text{ dB}$$

input	meaning	condition
`ratio`	amplitude ratio V_2/V_1 (a positive number)	

output	meaning
`dB`	level in decibels (dB)

- **Method:** 20 log10(ratio)
- **Assumes:** ratio > 0; both amplitudes across equal impedances
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dB (level in decibels (dB)) against ratio (amplitude ratio V2/V1 (a positive number)) from 5 to 500 on a logarithmic axis. 41 points computed by signal-processing-ai999x; marked: the worked example, ratio = 50, dB = 33.9794.

```
signal_processing_ai999x.solve('amplitude_ratio_to_db', ratio=50)
# -> dB = 33.9794   (SOLVED)
```

```
signal_processing_ai999x.solve('amplitude_ratio_to_db', ratio=100)
# -> dB = 40   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Express a voltage ratio of 50 in decibels.	SOLVED	dB = 33.9794

question	outcome	result
Convert a voltage gain of 100 to dB.	SOLVED	dB = 40
Convert a voltage gain of 0 to dB.	INVALID_INPUT	Amplitude (voltage) ratio to decibels: amplitude ratio must be a positive finite number.

Decibels to amplitude (voltage) ratio — `db_to_amplitude_ratio`

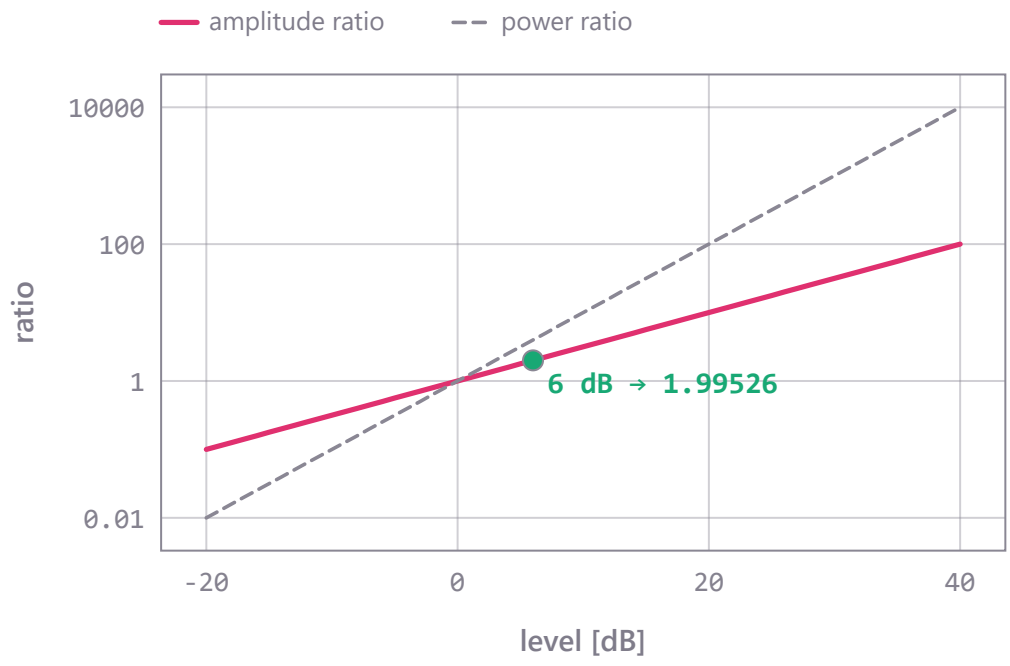
The amplitude ratio $V_2/V_1 = 10^{L/20}$ that a level of L dB represents.

$$\frac{V_2}{V_1} = 10^{L/20}$$

input	meaning	condition
`dB`	level in decibels	

output	meaning
`ratio`	amplitude ratio V_2/V_1

- **Method:** $10^{L/20}$
- **Assumes:** finite level
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The amplitude ratio for levels from −20 to 40 dB (61 runs of the package, log axis), with the power ratio dashed for comparison: 20 dB is a factor 10 in amplitude but 100 in power. Marked: 6 dB, ratio 1.99526.

```
signal_processing_ai999x.solve('db_to_amplitude_ratio', dB=6)
# -> ratio = 1.99526   (SOLVED)
```

```
signal_processing_ai999x.solve('db_to_amplitude_ratio', dB=40)
# -> ratio = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What voltage ratio corresponds to 6 dB?	SOLVED	ratio = 1.99526
Convert 40 dB to an amplitude ratio.	SOLVED	ratio = 100
Convert -20 dB to a voltage ratio.	SOLVED	ratio = 0.1

Power in watts to dBm and dBW — `watts_to_dbm`

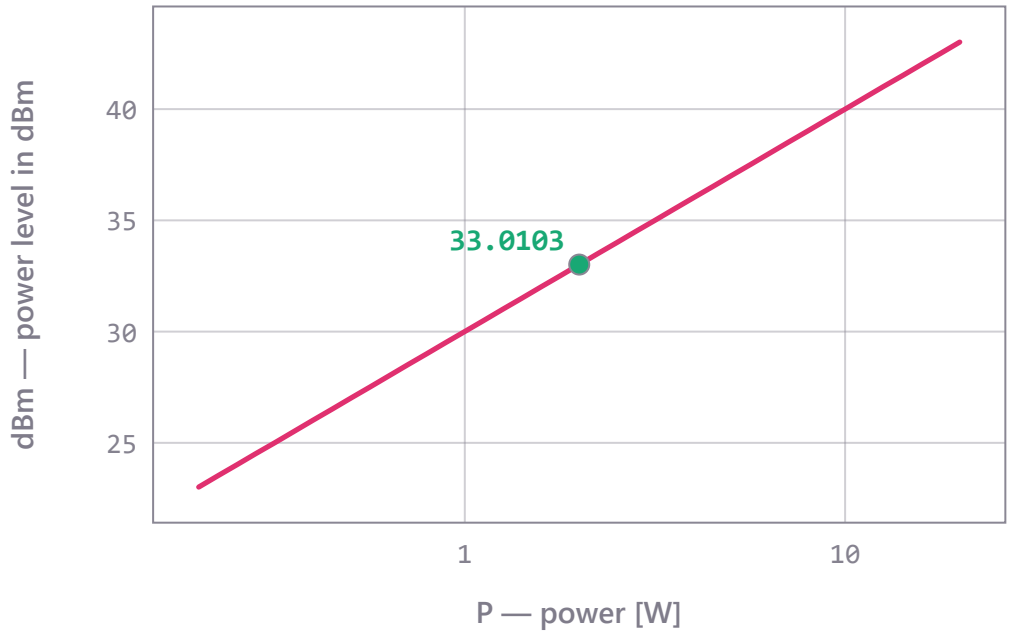
A power expressed as a level relative to 1 mW (dBm) and to 1 W (dBW).

$$P_{\mathrm{dBm}} = 10 \log_{10} \frac{P}{1\,\mathrm{mW}}$$

input	meaning	condition
`P`	power	

output	meaning
`dBm`	power level in dBm
`dBW`	power level in dBW

- **Method:** 10 log10(P/1 mW)
- **Assumes:** $P > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dBm (power level in dBm) against P (power) from 0.2 W to 20 W on a logarithmic axis. 41 points computed by signal-processing-ai999x; marked: the worked example, P = 2 W, dBm = 33.0103.

```
signal_processing_ai999x.solve('watts_to_dbm', P=2.0)
# -> dBm = 33.0103, dBW = 3.0103   (SOLVED)
```

```
signal_processing_ai999x.solve('watts_to_dbm', P=0.05)
# -> dBm = 16.9897, dBW = -13.0103   (SOLVED)
```

```
signal_processing_ai999x.solve('watts_to_dbm', P=0.001)
# -> dBm = 0, dBW = -30   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 2 W to dBm.	SOLVED	dBm = 33.0103, dBW = 3.0103
Express a power of 50 mW in dBm.	SOLVED	dBm = 16.9897, dBW = -13.0103
Convert 0 W to dBm.	INVALID_INPUT	Power in watts to dBm and dBW: power must be a positive finite number.

Power level in dBm to watts — `dbm_to_watts`

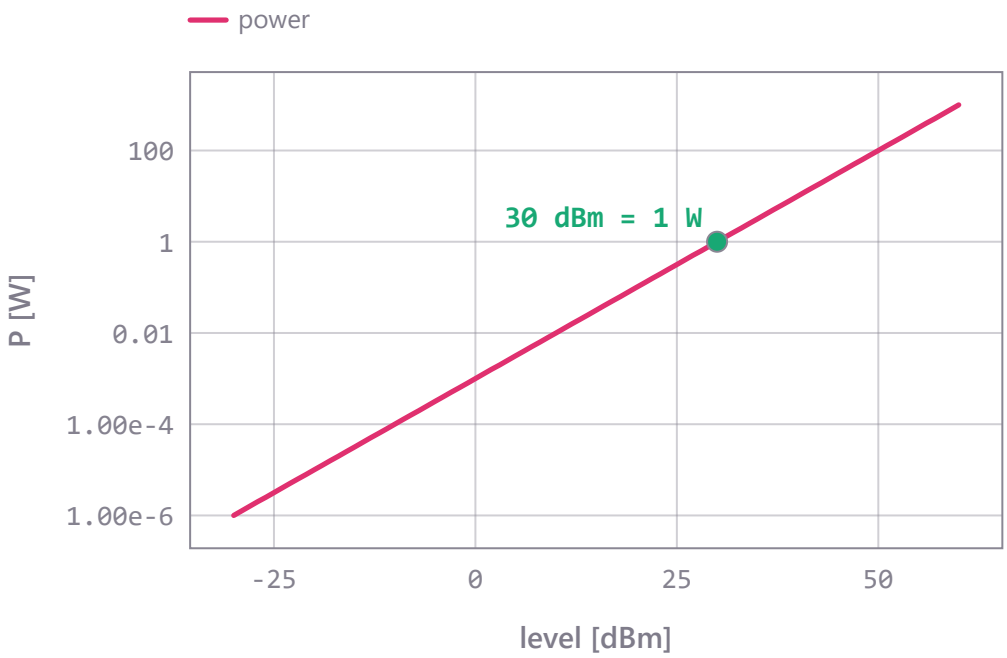
The power in watts of a level given in dBm (and the same level in dBW).

$$P = 1\,\mathrm{mW} \cdot 10^{P_{\mathrm{dBm}}/10}$$

input	meaning	condition
`dBm`	power level in dBm	

output	meaning
`P`	power
`dBW`	power level in dBW

- **Method:** $P = 1\,mW 10^{L/10}$
- **Assumes:** finite level
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Power in watts for levels from −30 to 60 dBm (91 runs of the package, log axis): every 10 dB is a factor 10, 0 dBm is 1 mW. Marked: 30 dBm = 1 W.

```
signal_processing_ai999x.solve('dbm_to_watts', dBm=30)
# -> P = 1 W, dBW = 0   (SOLVED)
```

```
signal_processing_ai999x.solve('dbm_to_watts', dBm=-20)
# -> P = 1e-05 W, dBW = -50   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert 30 dBm to watts.	SOLVED	P = 1 W, dBW = 0
Convert -20 dBm to milliwatts.	SOLVED	P = 1e-05 W, dBW = -50
Convert 43 dBm to W.	SOLVED	P = 19.9526 W, dBW = 13

Signal-to-noise ratio from signal and noise powers — `snr_from_powers`

The signal-to-noise ratio Ps/Pn, as a ratio and in dB.

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \frac{P_s}{P_n}$$

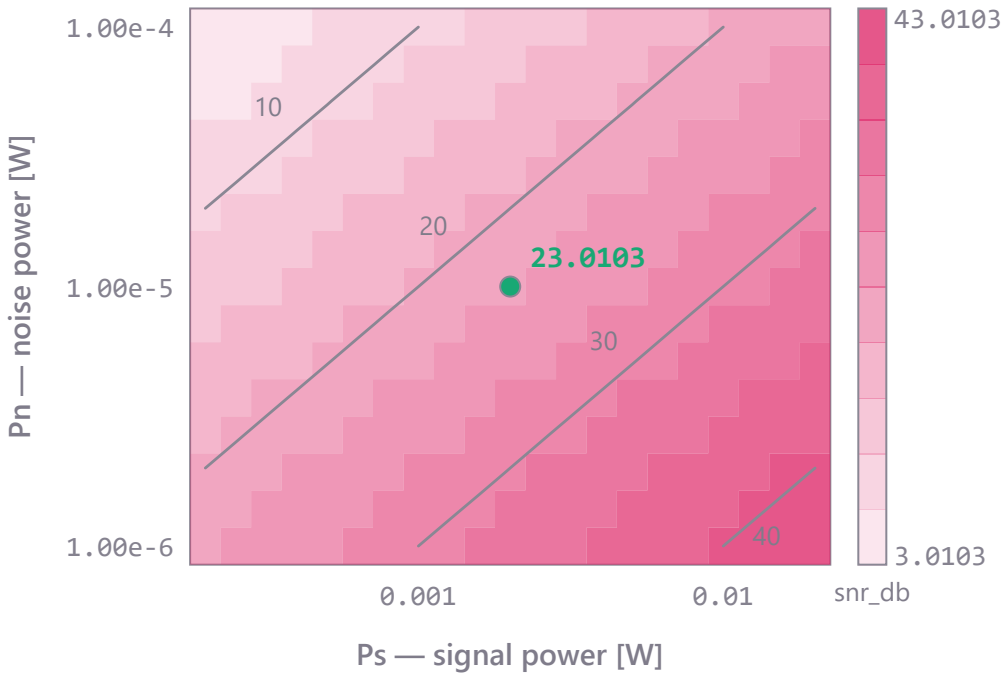
input	meaning	condition
`Ps`	signal power	
`Pn`	noise power	

output	meaning
`snr_db`	signal-to-noise ratio in dB
`snr`	signal-to-noise ratio (linear)

- **Method:** SNR = Ps/Pn; 10 log10(SNR)
- **Assumes:** $Ps > 0$; $Pn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



snr_db (signal-to-noise ratio in dB) against Ps (signal power) from 0.0002 W to 0.02 W on a logarithmic axis. One curve for each value of Pn (noise power): 1.0000e-05 W, 5.0000e-06 W, 2.0000e-05 W. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, Ps = 0.002 W, snr_db = 23.0103.



snr_db (signal-to-noise ratio in dB) over Ps (signal power) from 0.0002 W to 0.02 W and Pn (noise power) from 1.0000e-06 W to 0.0001 W (logarithmic axes): each of the 21 × 15 cells is one run of signal-processing-ai999x, shaded by its value (light 3.0103 to dark 43.0103); iso-lines at snr_db = 10, 20, 30, 40. Marked: the worked example, Ps = 0.002 W, Pn = 1.0000e-05 W, snr_db = 23.0103.

```
signal_processing_ai999x.solve('snr_from_powers', Ps=0.002, Pn=1e-05)
# -> snr = 200, snr_db = 23.0103   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the SNR in dB for a signal power of 2 mW and a noise power of 10 uW?	SOLVED	snr = 200, snr_db = 23.0103
What is the signal-to-noise ratio for a signal power of 1 W and a noise power of 1 mW?	SOLVED	snr = 1000, snr_db = 30
What is the SNR in dB for a signal power of 2 mW?	NEEDS_INPUT	snr_from_powers needs Pn (noise power). Example: solve('snr_from_powers', Ps='0.002 W', Pn='1e-05 W').

Mean, RMS, energy and average power of a sampled signal — `signal_statistics`

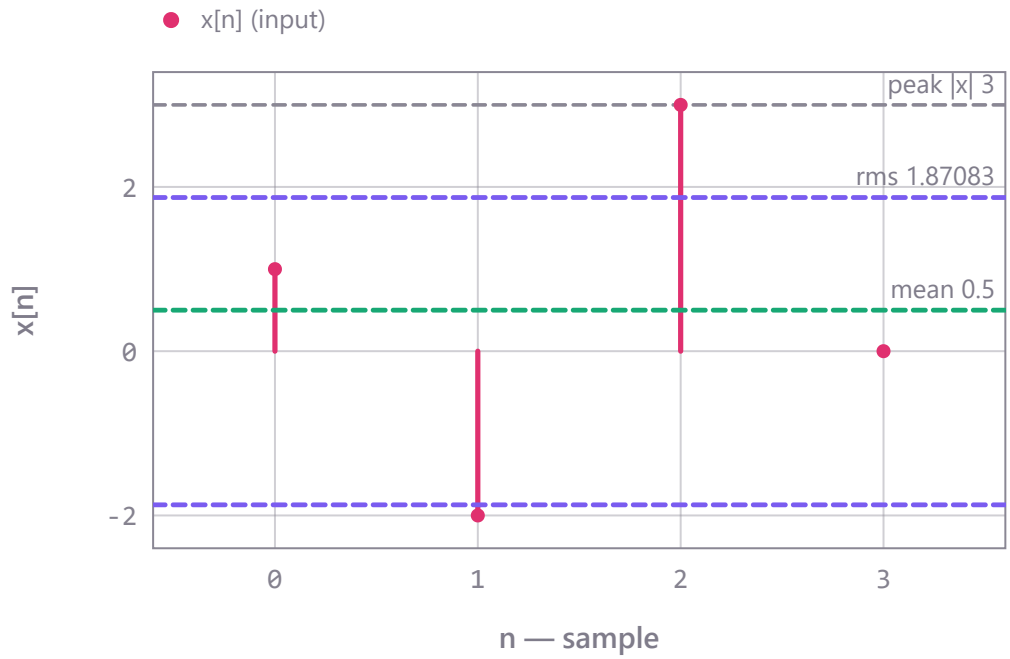
For samples x[$0..N-1$]: the mean, the RMS value, the energy sum |x|^2 and the average power (energy / N).

$$E = \sum_n |x[n]|^2, \quad P = \frac{E}{N}, \quad x_{\mathrm{rms}} = \sqrt{P}$$

input	meaning	condition
`x`	samples of the signal	

output	meaning
`rms`	root-mean-square value
`mean`	mean (DC) value
`energy`	energy: sum of x[n]^2
`power`	average power: energy / N
`peak`	peak absolute value

- **Method:** direct sums (compensated)
- **Assumes:** at least one sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The samples x = [1, -2, 3, 0] with the statistics returned by the package: mean 0.5, rms 1.87083 (drawn at ±rms), peak |x| 3, energy 14, average power 3.5.

```
signal_processing_ai999x.solve('signal_statistics', x='1, -2, 3, 0')
# -> mean = 0.5, rms = 1.87083, energy = 14, power = 3.5, peak = 3   (SOLVED)
```

```
signal_processing_ai999x.solve('signal_statistics', x=[2, -2, 2, -2])
# -> mean = 0, rms = 2, energy = 16, power = 4, peak = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the energy, average power and RMS value of the signal x = [1, -2, 3, 0].	SOLVED	mean = 0.5, rms = 1.87083, energy = 14, power = 3.5, peak = 3
What is the RMS value of the samples [2, -2, 2, -2]?	SOLVED	mean = 0, rms = 2, energy = 16, power = 4, peak = 2
Find the energy of the signal x = [1, a, 3].	INVALID_INPUT	x: 'a' is not a number.

RMS, peak-to-peak and rectified average of a sine wave — `sine_rms`

For a sinusoid of peak amplitude Vp: the RMS value Vp/sqrt(2), the peak-to-peak value $2Vp$ and the average of its absolute value $2Vp/pi$.

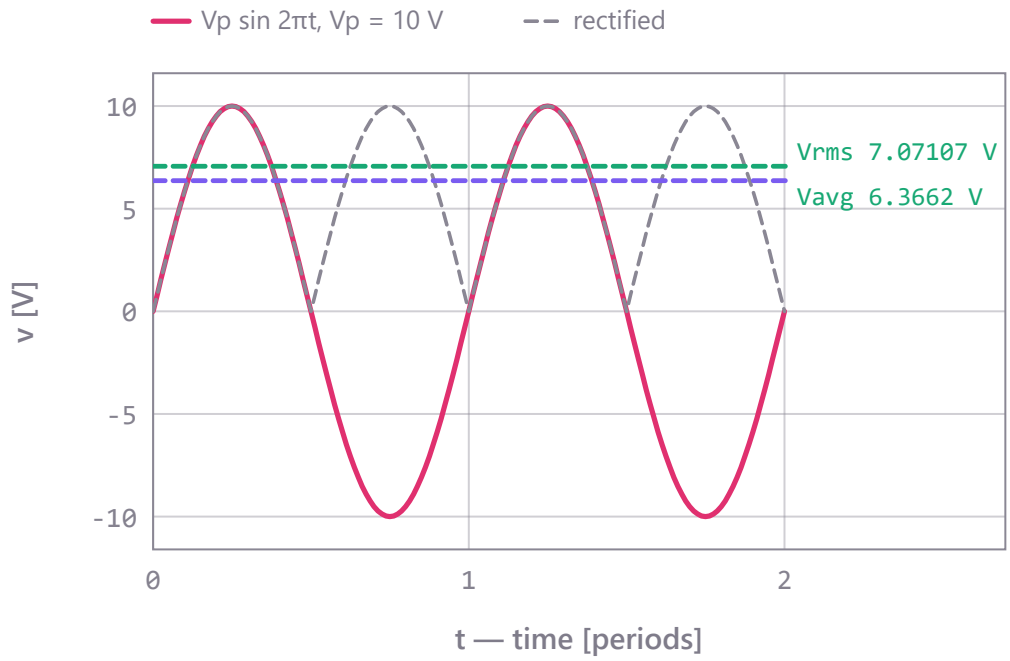
$$V_{\text{rms}} = \frac{V_p}{\sqrt{2}}$$

input	meaning	condition
`Vp`	peak amplitude of the sinusoid	

output	meaning
`Vrms`	RMS value

output	meaning
`Vpp`	peak-to-peak value
`Vavg`	average of the rectified wave

- **Method:** $V_{rms} = \frac{V_p}{\sqrt{2}}$
- **Assumes:** $Vp > 0$; a pure sinusoid with no DC offset
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A sinusoid of peak Vp = 10 V (input) over two periods, with its rectified wave; returned by the package: Vrms = 7.07107 V, average of the rectified wave Vavg = 6.3662 V, peak-to-peak Vpp = 20 V.

```
signal_processing_ai999x.solve('sine_rms', Vp=10.0)
# -> Vrms = 7.07107 V, Vpp = 20 V, Vavg = 6.3662 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the RMS value of a sine wave with a peak voltage of 10 V?	SOLVED	Vrms = 7.07107 V, Vpp = 20 V, Vavg = 6.3662 V
Find the RMS value of a sinusoidal voltage with an amplitude of 170 V.	SOLVED	Vrms = 120.208 V, Vpp = 340 V, Vavg = 108.225 V
Find the RMS value of a sinusoidal voltage.	NEEDS_INPUT	sine_rms needs Vp (peak amplitude of the sinusoid). Example: solve('sine_rms', Vp='10 V').

Signal-to-quantization-noise ratio of an ideal ADC — `quantization_snr`

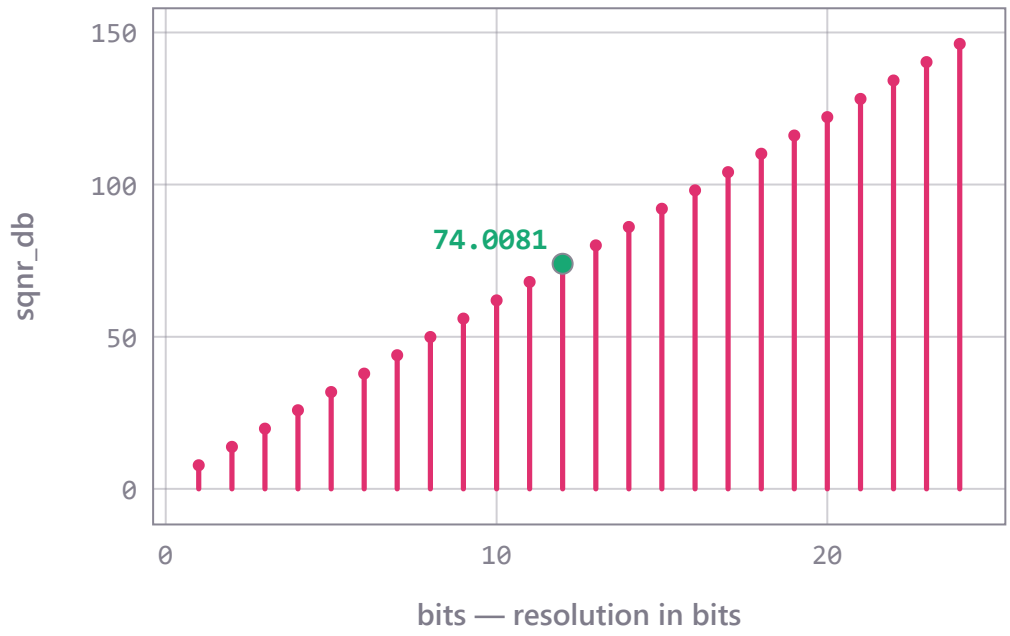
The SQNR of an ideal N-bit uniform quantizer driven by a full-scale sine wave (about 6.02 $N + 1.76$ dB).

$$\mathrm{SQNR} = 10\log_{10}\left(\tfrac{3}{2}\,2^{2N}\right) \approx 6.02N + 1.76\ \mathrm{dB}$$

input	meaning	condition
`bits`	resolution in bits	

output	meaning
`sqnr_db`	signal-to-quantization-noise ratio in dB
`dynamic_range_db`	ideal dynamic range 20 log10(2^N) in dB

- **Method:** ratio of the sine's power $A^2/2$ to the quantization noise power $Delta^2/12$ with $Delta = 2A/2^N$
- **Assumes:** ideal uniform quantizer; full-scale sinusoidal input; noise uniformly distributed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sqnr_db (signal-to-quantization-noise ratio in dB) against bits (resolution in bits) from 1 to 24. 24 points computed by signal-processing-ai999x; marked: the worked example, bits = 12, sqnr_db = 74.0081.

```
signal_processing_ai999x.solve('quantization_snr', bits=12)
# -> sqnr_db = 74.0081, dynamic_range_db = 72.2472   (SOLVED)
```

```
signal_processing_ai999x.solve('quantization_snr', bits=16)
# -> sqnr_db = 98.0905, dynamic_range_db = 96.3296   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the signal-to-quantization-noise ratio of an ideal 12-bit ADC?	SOLVED	sqnr_db = 74.0081, dynamic_range_db = 72.2472
Find the SQNR of a 16-bit quantizer.	SOLVED	sqnr_db = 98.0905, dynamic_range_db = 96.3296
Find the SQNR of a 0-bit quantizer.	INVALID_INPUT	bits: must be at least 1, got 0.

Total power of several signals given in dBm — `dbm_power_sum`

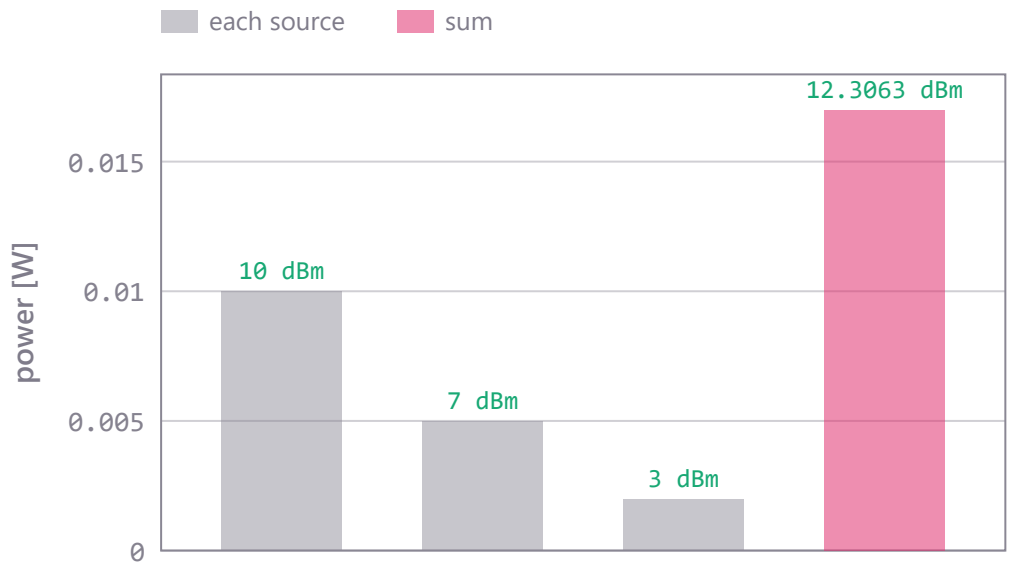
The combined power of uncorrelated signals whose levels are given in dBm (powers add, levels do not).

$$P_{\text{tot}} = 10\log_{10}\sum_i 10^{P_i/10}\ \text{dBm}$$

input	meaning	condition
`levels`	power levels in dBm	

output	meaning
`total_dBm`	total power level in dBm
`total_W`	total power

- **Method:** convert to milliwatts, add, convert back
- **Assumes:** uncorrelated signals (their powers add)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Adding incoherent powers given in dBm: three sources of 10, 7, 3 dBm (chosen input), shown in watts (from the package's dBm-to-watts capability: 0.01, 0.00501187, 0.00199526 W). The package returns their total 0.0170071 W = 12.3063 dBm — the levels add in watts, not in dB.

```
signal_processing_ai999x.solve('dbm_power_sum', levels='10, 10')
# -> total_dBm = 13.0103, total_W = 0.02 W   (SOLVED)
```

```
signal_processing_ai999x.solve('dbm_power_sum', levels=[0, 0, 0, 0])
# -> total_dBm = 6.0206, total_W = 0.004 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the total power of signals at [10, 10] dBm combined?	SOLVED	total_dBm = 13.0103, total_W = 0.02 W
Add the powers [0, 0, 0, 0] dBm.	SOLVED	total_dBm = 6.0206, total_W = 0.004 W

Frequency resolution of an N-point DFT — `frequency_resolution`

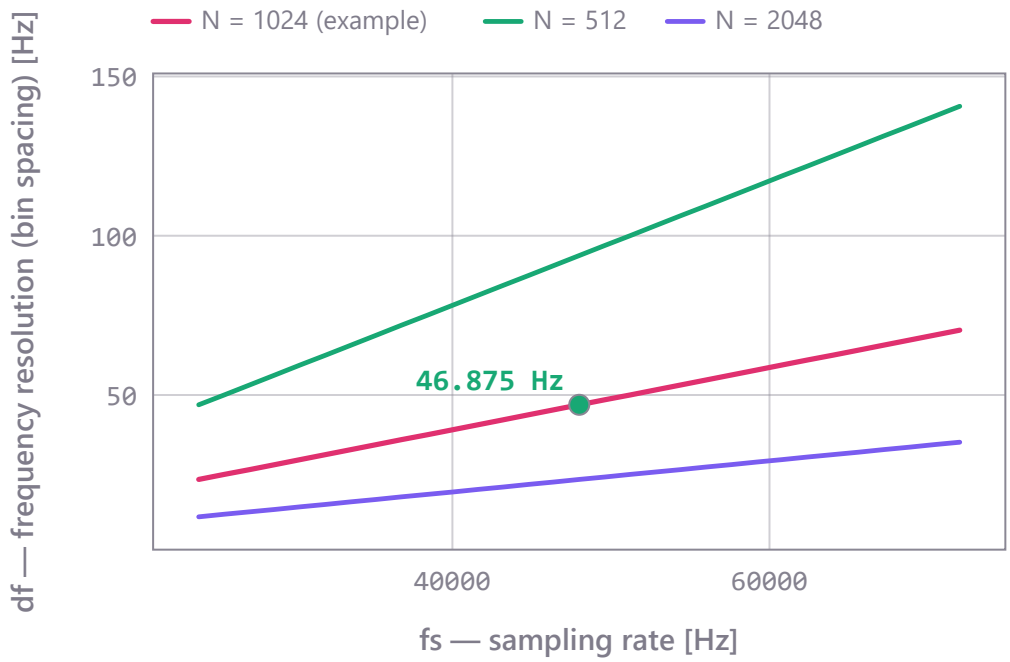
The bin spacing fs/N of an N-point DFT or FFT and the record length N/fs it covers.

$$\Delta f = \frac{f_s}{N}$$

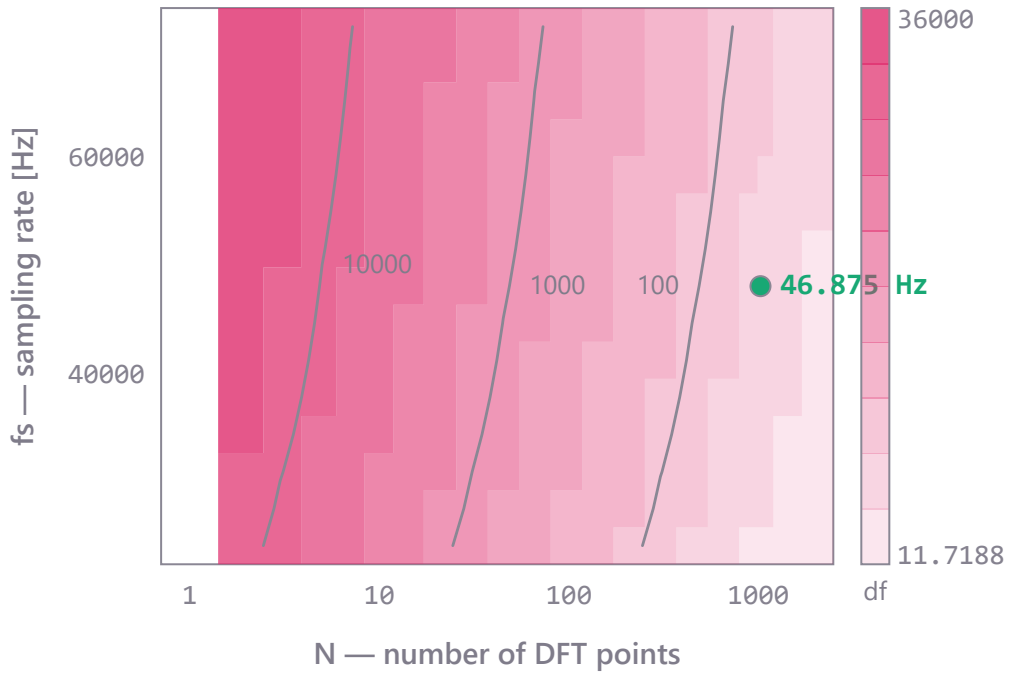
input	meaning	condition
`fs`	sampling rate	
`N`	number of DFT points	

output	meaning
`df`	frequency resolution (bin spacing)
`duration`	record length N/fs

- **Method:** delta $f = fs/N$
- **Assumes:** $fs > 0$; $N \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



df (frequency resolution (bin spacing)) against fs (sampling rate) from 24000 Hz to 72000 Hz. One curve for each value of N (number of DFT points): 1024, 512, 2048. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, fs = 48000 Hz, df = 46.875 Hz.



df (frequency resolution (bin spacing)) over N (number of DFT points) from 1 to 2048 and fs (sampling rate) from 24000 Hz to 72000 Hz (logarithmic horizontal axis): each of the 21 × 15 cells is one run of signal-processing-ai999x, shaded by its value (light 11.7188 Hz to dark 36000 Hz, logarithmic shading); iso-lines at df = 100, 1000, 10000 Hz; blank cells: inputs the capability refuses. Marked: the worked example, N = 1024, fs = 48000 Hz, df = 46.875 Hz.

```
signal_processing_ai999x.solve('frequency_resolution', fs=48000.0, N=1024)
# -> df = 46.875 Hz, duration = 0.0213333 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the frequency resolution of a 1024-point FFT at a sampling rate of 48 kHz?	SOLVED	df = 46.875 Hz, duration = 0.0213333 s
Find the bin spacing of a DFT of 500 samples at a sampling frequency of 1 kHz.	SOLVED	df = 2 Hz, duration = 0.5 s
What is the frequency resolution of a 1024-point FFT?	NEEDS_INPUT	frequency_resolution needs fs (sampling rate). Example: solve('frequency_resolution', fs='48000 Hz', N=1024).

Frequency of a DFT bin — `bin_frequency`

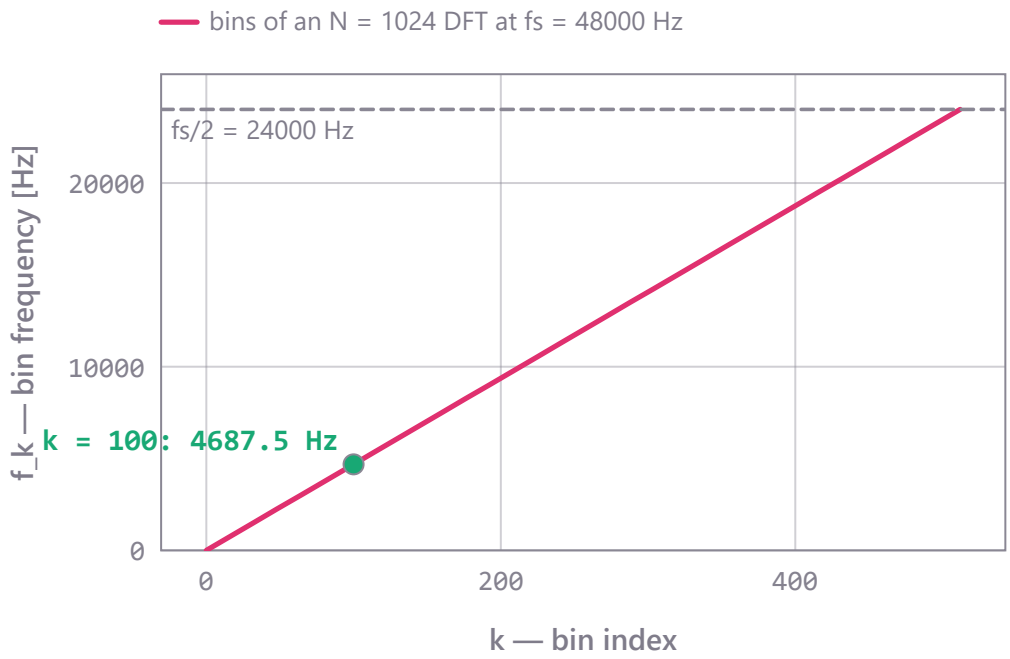
The frequency kfs/N that bin k of an N-point DFT represents $(0 \le k \le N/2)$.

$$f_k = \frac{k\,f_s}{N}$$

input	meaning	condition
`k`	bin index	
`fs`	sampling rate	
`N`	number of DFT points	

output	meaning
`f_k`	frequency of bin k

- **Method:** $f_k = k$ fs / N
- **Assumes:** $0 \le k \le N/2$; $fs > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The centre frequency of DFT bin k for N = 1024 points sampled at fs = 48000 Hz, k = 0 … 512 (one run of the package per plotted bin): a straight line of slope fs/N reaching fs/2 at k = N/2 (dashed, stated). Marked: k = 100, f_k = 4687.5 Hz.

```
signal_processing_ai999x.solve('bin_frequency', k=100, fs=48000.0, N=1024)
# -> f_k = 4687.5 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What frequency does bin 100 of a 1024-point FFT correspond to at a sampling rate of 48 kHz?	SOLVED	f_k = 4687.5 Hz
What frequency does bin 900 of a 1024-point FFT correspond to at a sampling rate of 48 kHz?	INVALID_INPUT	Frequency of a DFT bin: bin index must be at most 512.

One bin of the discrete Fourier transform — `dft_bin`

X[k] = sum_n x[n] exp(-j 2 pi k n / N) for one bin k: real and imaginary parts, magnitude and phase.

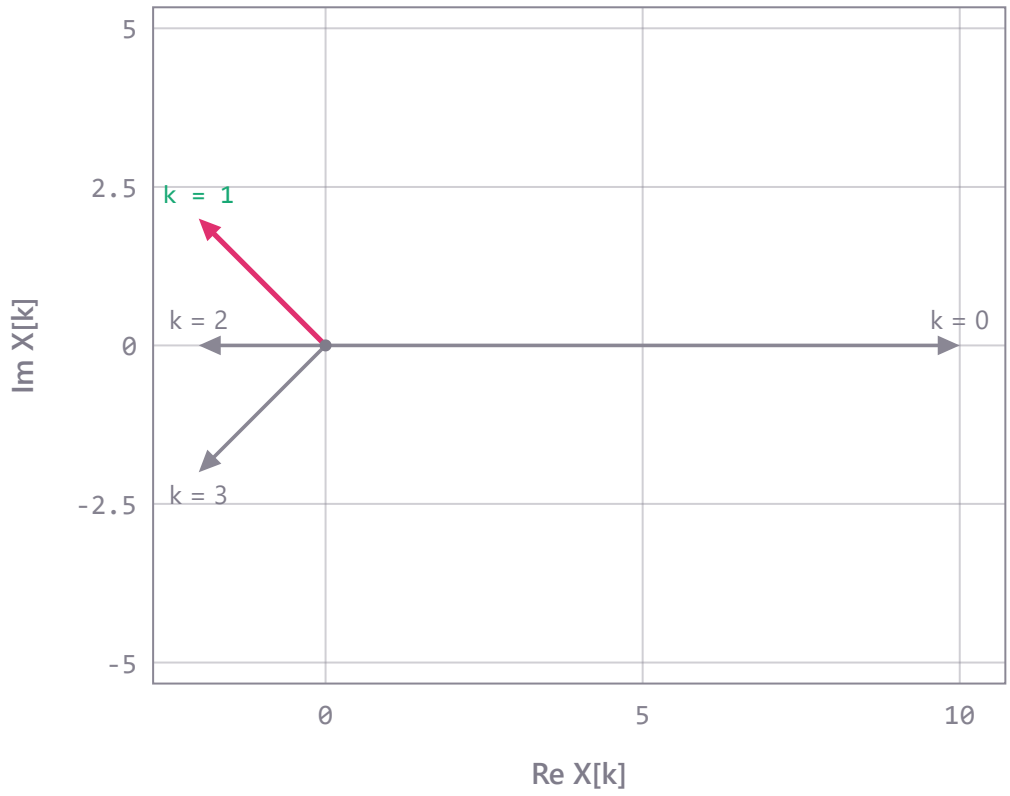
$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}$$

input	meaning	condition
`x`	samples x[$0..N-1$]	

input	meaning	condition
k	bin index $(0 \le k < N)$	

output	meaning
magnitude	|X[k]|
phase	phase of X[k] in radians (-pi, pi]
re	real part of X[k]
im	imaginary part of X[k]

- **Method:** direct evaluation of the DFT sum (compensated sums, exact twiddles at quarter turns)
- **Assumes:** $0 \le k < N$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The DFT bins of x = [1, 2, 3, 4] as phasors in the complex plane, one run of the package per bin: X[0] = 10 + 0j; X[1] = -2 + 2j; X[2] = -2 + 0j; X[3] = -2 − 2j. Highlighted: the worked example k = 1, |X| = 2.82843, phase 2.35619 rad.

```
signal_processing_ai999x.solve('dft_bin', x='1, 2, 3, 4', k=1)
# -> magnitude = 2.82843, phase = 2.35619 rad, re = -2, im = 2   (SOLVED)
```

```
signal_processing_ai999x.solve('dft_bin', x=[1, 2, 3, 4], k=2)
# -> magnitude = 2, phase = 3.14159 rad, re = -2, im = 0   (SOLVED)
```

```
signal_processing_ai999x.solve('dft_bin', x=[1, 0, -1, 0, 1, 0, -1, 0], k=2)
# -> magnitude = 4, phase = 0 rad, re = 4, im = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the DFT of x = [1, 2, 3, 4] at bin k = 1.	SOLVED	magnitude = 2.82843, phase = 2.35619 rad, re = -2, im = 2
Find X[k] for k = 2 of the DFT of x = [1, 0, -1, 0, 1, 0, -1, 0].	SOLVED	magnitude = 4, phase = 0 rad, re = 4, im = 0
Compute the DFT of x = [1, 2, 3, 4] at bin k = 7.	INVALID_INPUT	One bin of the discrete Fourier transform: bin index k must be at most 3.
Compute the DFT of x = [1, 2, 3, 4] at bin k.	NEEDS_INPUT	dft_bin needs k (bin index (0 <= k < N)). Example: solve('dft_bin', x='1, 2, 3, 4', k=1).

DFT spectrum of a sequence (FFT) — `dft_spectrum`

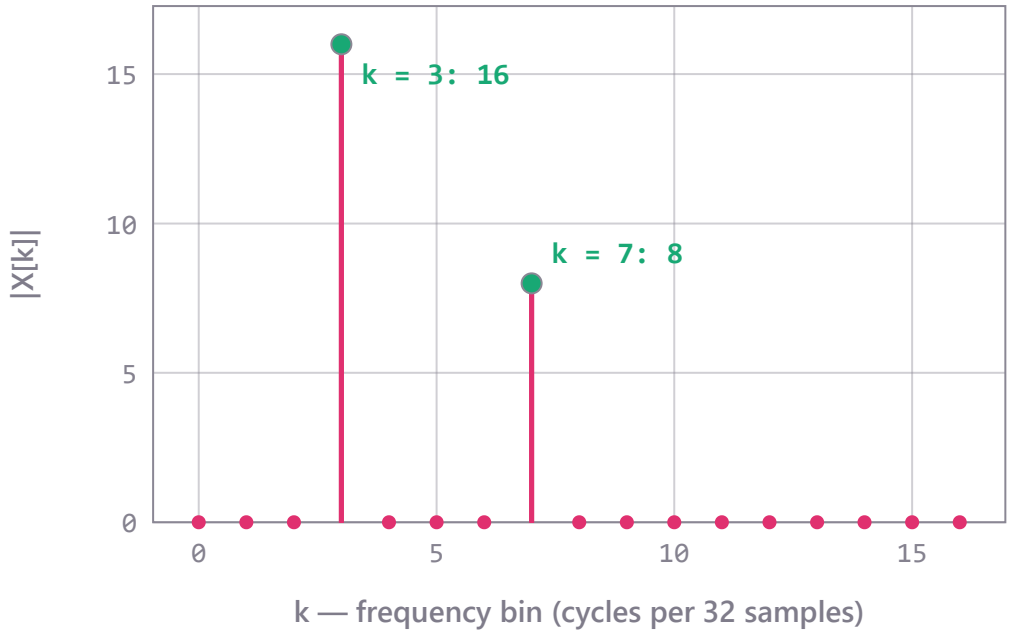
The whole N-point DFT of a real sequence: magnitude, phase, real and imaginary parts of every bin, and the strongest bin up to $N/2$.

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N},\ k = 0, \ldots, N-1$$

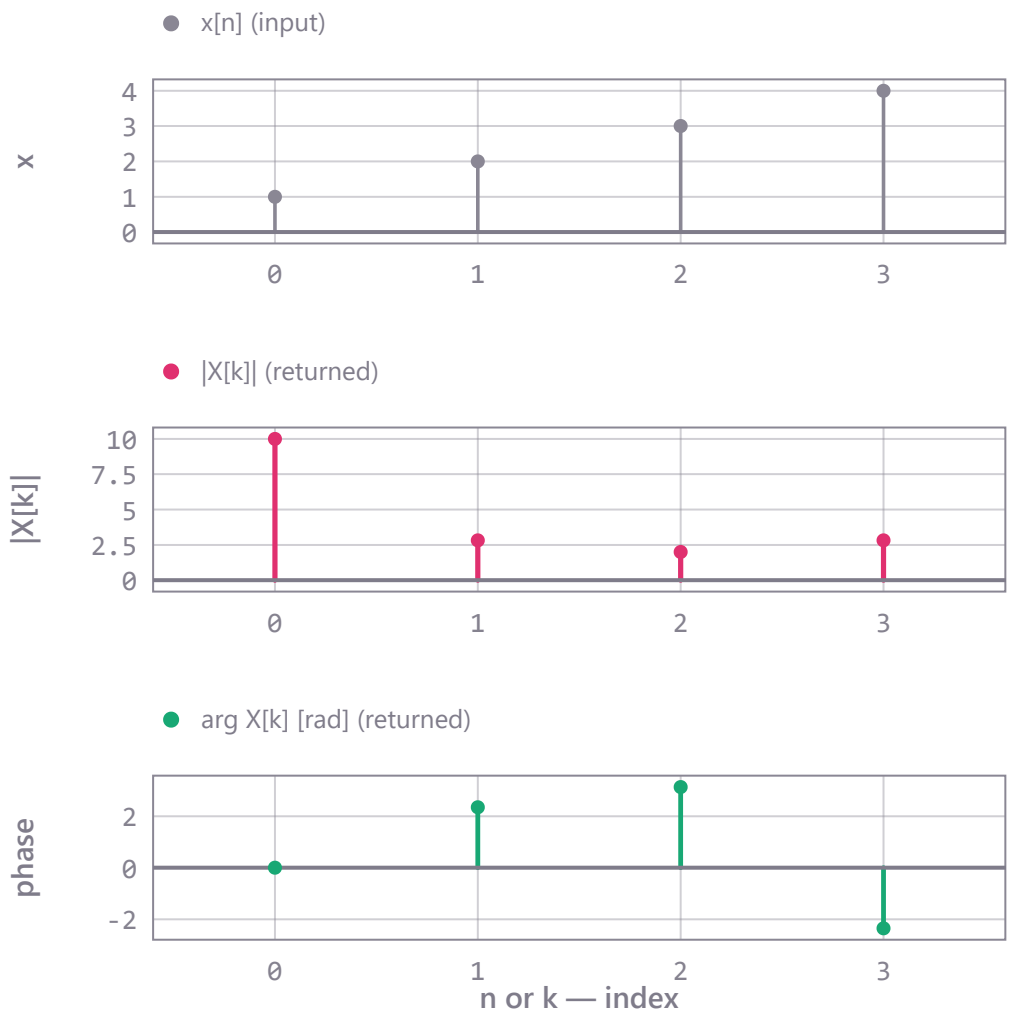
input	meaning	condition
`x`	samples x[$0..N-1$]	

output	meaning
`peak_magnitude`	largest |X[k]| for $0 \le k \le N/2$
`peak_bin`	bin of the largest magnitude
`magnitude`	|X[k]| for every bin
`phase`	phase of X[k] in radians for every bin
`real`	real parts
`imag`	imaginary parts

- **Method:** radix-2 decimation-in-time FFT (N a power of two) or the direct DFT sum
- **Assumes:** real samples
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Magnitude of the 32-point DFT returned by the package for x[n] = sin(2π·3n/32) + 0.5 sin(2π·7n/32), n = 0…31 (bins 0–16 shown; the rest mirror them). The two tones appear at bins 3 and 7; marked: |X[3]| = 16, |X[7]| = 8.



The worked example x = [1, 2, 3, 4] and its DFT returned by the package: magnitudes [10, 2.828, 2, 2.828] and phases [0, 2.356, 3.142, -2.356] rad for bins k = 0 … 3; largest |X[0]| = 10.

```
signal_processing_ai999x.solve('dft_spectrum', x='1, 2, 3, 4')
# -> peak_magnitude = 10, peak_bin = 0, magnitude = [10.0, 2.8284271247461903, 2.0,
2.8284271247461903], phase = [0.0, 2.356194490192345, 3.141592653589793, -2.356194490192345], real
= [10.0, -2.0, -2.0, -2.0], imag = [0.0, 2.0, 0.0, -2.0]   (SOLVED)
```

```
signal_processing_ai999x.solve('dft_spectrum', x=[1, 0, -1, 0, 1, 0, -1, 0])
# -> peak_magnitude = 4, peak_bin = 2, magnitude = [0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 4.0, 0.0], phase
= [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], real = [0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 4.0, 0.0], imag =
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]   (SOLVED)
```

```
signal_processing_ai999x.solve('dft_spectrum', x=[1, 1, 1])
# -> peak_magnitude = 3, peak_bin = 0, magnitude = [3.0, 0.0, 0.0], phase = [0.0, 0.0, 0.0], real =
[3.0, 0.0, 0.0], imag = [0.0, 0.0, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the DFT of x = [1, 2, 3, 4].	SOLVED	peak_magnitude = 10, peak_bin = 0, magnitude = [10.0, 2.8284271247461903, 2.0, 2.8284271247461903], phase = [0.0, 2.356194490192345, 3.141592653589793, -2.356194490192345], real = [10.0, -2.0, -2.0, -2.0], imag = [0.0, 2.0, 0.0, -2.0]
Find the FFT spectrum of the sequence [1, 0, -1, 0, 1, 0, -1, 0].	SOLVED	peak_magnitude = 4, peak_bin = 2, magnitude = [0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 4.0, 0.0], phase = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], real = [0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 4.0, 0.0], imag = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
Compute the DFT of x = [].	INVALID_INPUT	x: needs at least 1 entries, got 0.

Inverse discrete Fourier transform — `inverse_dft`

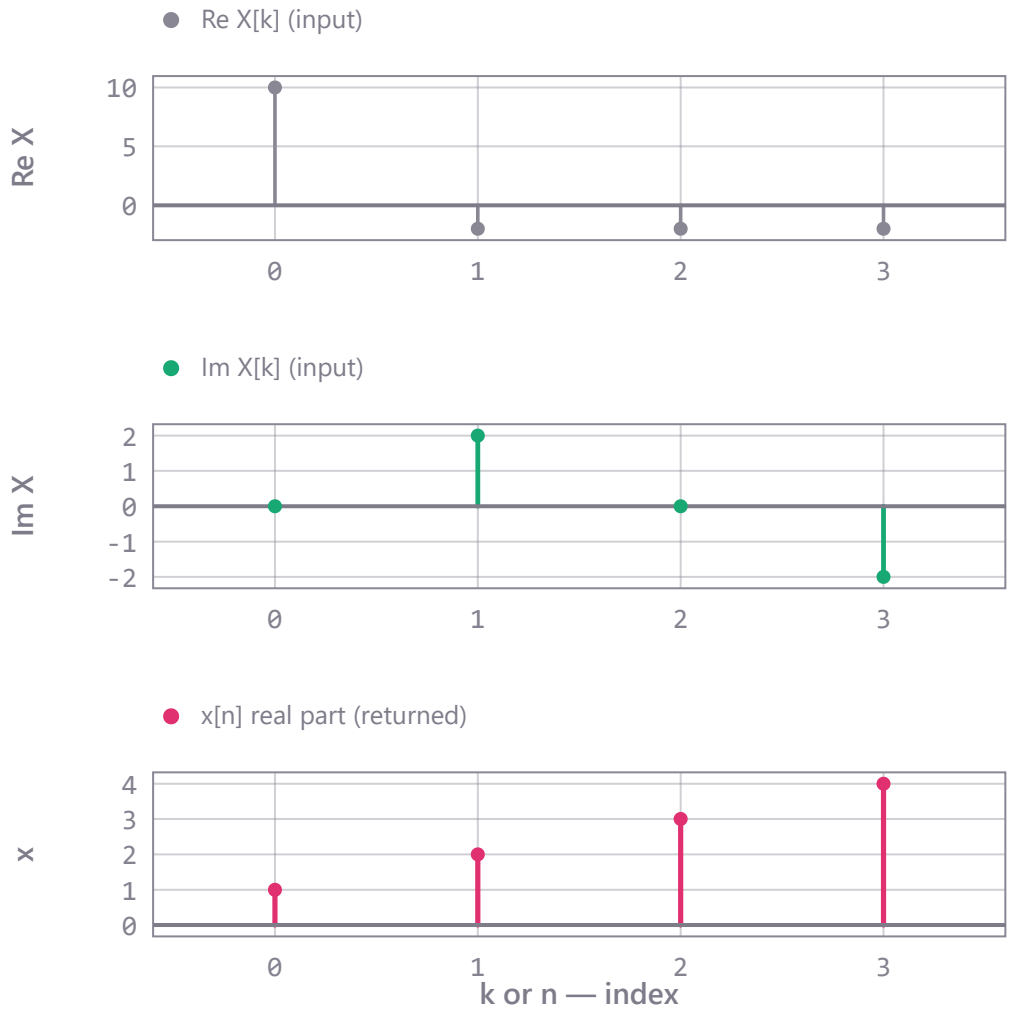
x[n] = $(1/N)$ sum_k X[k] exp(+j 2 pi k n / N), from the real and imaginary parts of X[k].

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N}$$

input	meaning	condition
`re_`	real parts of X[$0..N-1$]	
`im`	imaginary parts of X[$0..N-1$] (zeros if not given)	

output	meaning
`real`	real parts of x[n]
`imag`	imaginary parts of x[n]

- **Method:** inverse FFT (conjugate twiddles) divided by N
- **Assumes:** both lists have the same length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The inverse DFT of the spectrum Re = [10, -2, -2, -2], Im = [0, 2, 0, -2] (inputs): the package returns x = [1, 2, 3, 4] with imaginary parts [0, 0, 0, 0] — a real signal, because the spectrum is conjugate-symmetric.

```
signal_processing_ai999x.solve('inverse_dft', re_='10, -2, -2, -2', im='0, 2, 0, -2')
# -> real = [1.0, 2.0, 3.0, 4.0], imag = [0.0, 0.0, 0.0, 0.0]   (SOLVED)
```

```
signal_processing_ai999x.solve('inverse_dft', re_=[4, 0, 0, 0])
# -> real = [1.0, 1.0, 1.0, 1.0], imag = [0.0, 0.0, 0.0, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the inverse DFT with real parts [10, -2, -2, -2] and imaginary parts [0, 2, 0, -2].	SOLVED	real = [1.0, 2.0, 3.0, 4.0], imag = [0.0, 0.0, 0.0, 0.0]
Compute the inverse DFT with real parts [4, 0, 0, 0].	SOLVED	real = [1.0, 1.0, 1.0, 1.0], imag = [0.0, 0.0, 0.0, 0.0]
Compute the inverse DFT with real parts [1, 2] and imaginary parts [1, 2, 3].	INVALID_INPUT	Inverse discrete Fourier transform: the real and imaginary parts must have the same length.

Linear convolution of two sequences — `linear_convolution`

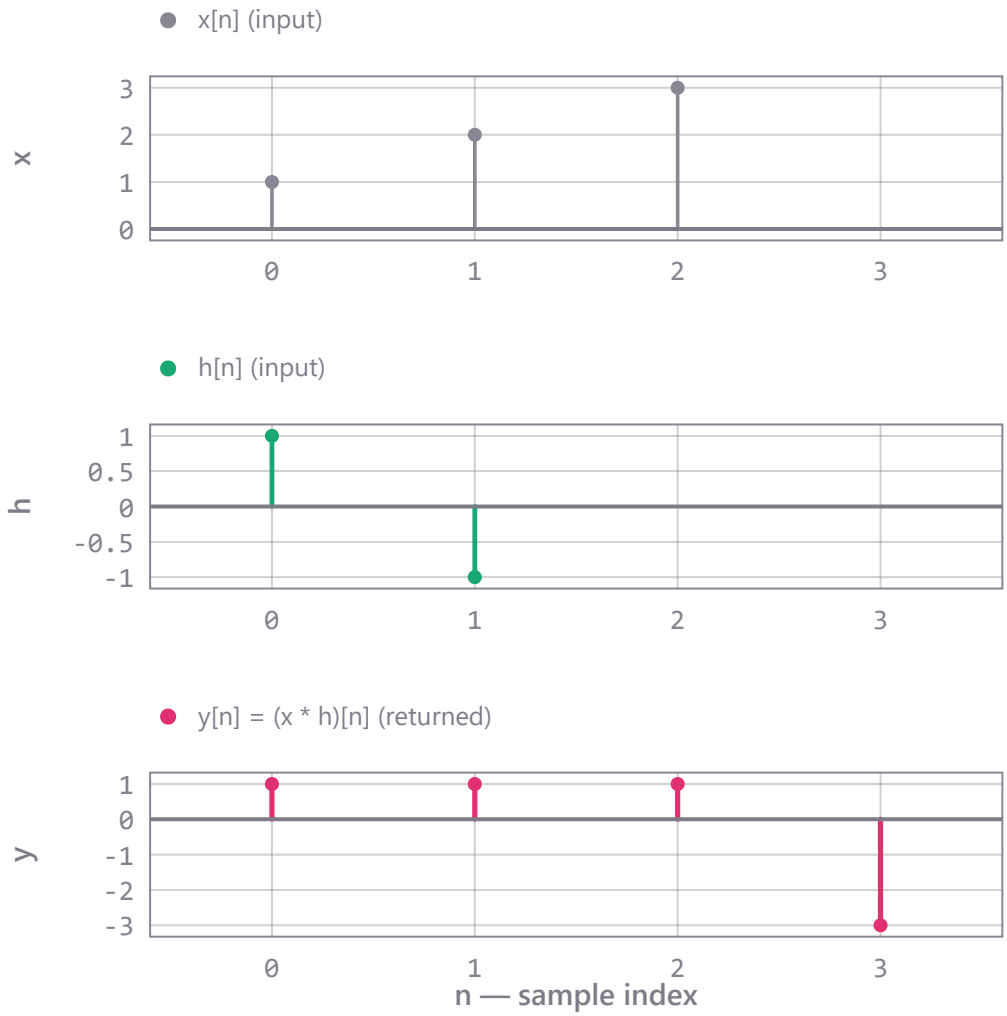
y[n] = sum_k x[k] h[n-k], the full linear convolution (length Nx + Nh - 1).

$$y[n] = \sum_{k} x[k]\, h[n-k]$$

input	meaning	condition
`x`	first sequence x[n]	
`h`	second sequence h[n]	

output	meaning
`y`	convolution y[n], $n = 0$ .. $Nx + Nh - 2$
`length`	number of output samples

- **Method:** direct convolution sum (compensated)
- **Assumes:** finite sequences starting at $n = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Linear convolution of x = [1, 2, 3] with h = [1, -1]: the package returns y = [1, 1, 1, -3], of length 3 + 2 − 1 = 4.

```
signal_processing_ai999x.solve('linear_convolution', x='1, 2, 3', h='1, -1')
# -> y = [1.0, 1.0, 1.0, -3.0], length = 4   (SOLVED)
```

```
signal_processing_ai999x.solve('linear_convolution', x=[1, 1, 1], h=[1, 2])
# -> y = [1.0, 3.0, 3.0, 2.0], length = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the linear convolution of x = [1, 2, 3] and h = [1, -1].	SOLVED	y = [1.0, 1.0, 1.0, -3.0], length = 4
Convolve [1, 1, 1] with [1, 2].	SOLVED	y = [1.0, 3.0, 3.0, 2.0], length = 4
Compute the linear convolution of x = [1, 2, 3].	NEEDS_INPUT	linear_convolution needs h (second sequence h[n]). Example: solve('linear_convolution', x='1, 2, 3', h='1, -1').

Circular (cyclic) convolution — `circular_convolution`

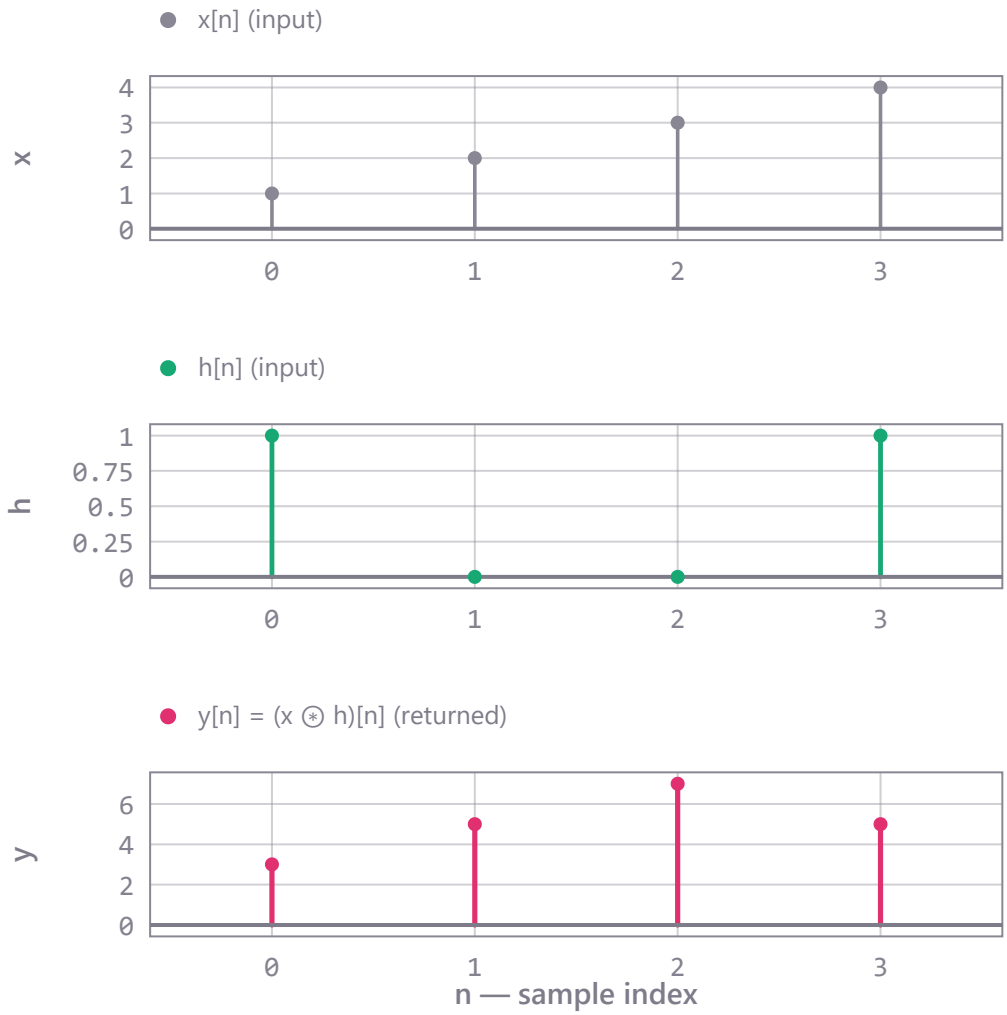
The N-point circular convolution y[n] = sum_k x[k] h[(n-k) mod N] of two sequences of length N.

$$y[n] = \sum_{k=0}^{N-1} x[k]\, h[(n-k) \bmod N]$$

input	meaning	condition
`x`	first sequence (length N)	
`h`	second sequence (length N)	

output	meaning
`y`	circular convolution y[$0..N-1$]

- **Method:** direct cyclic sum (indices modulo N)
- **Assumes:** both sequences have length N
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Circular convolution of x = [1, 2, 3, 4] with h = [1, 0, 0, 1] (length 4, indices wrap around): the package returns y = [3, 5, 7, 5].

```
signal_processing_ai999x.solve('circular_convolution', x='1, 2, 3, 4', h='1, 0, 0, 1')
# -> y = [3.0, 5.0, 7.0, 5.0]   (SOLVED)
```

```
signal_processing_ai999x.solve('circular_convolution', x=[1, 2, 3], h=[0, 1, 0])
# -> y = [3.0, 1.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the circular convolution of x = [1, 2, 3, 4] and h = [1, 0, 0, 1].	SOLVED	y = [3.0, 5.0, 7.0, 5.0]
Compute the circular convolution of x = [1, 2, 3] and h = [1, 2].	INVALID_INPUT	Circular (cyclic) convolution: circular convolution needs two sequences of the same length N.

Cross-correlation and the lag of its peak — `cross_correlation`

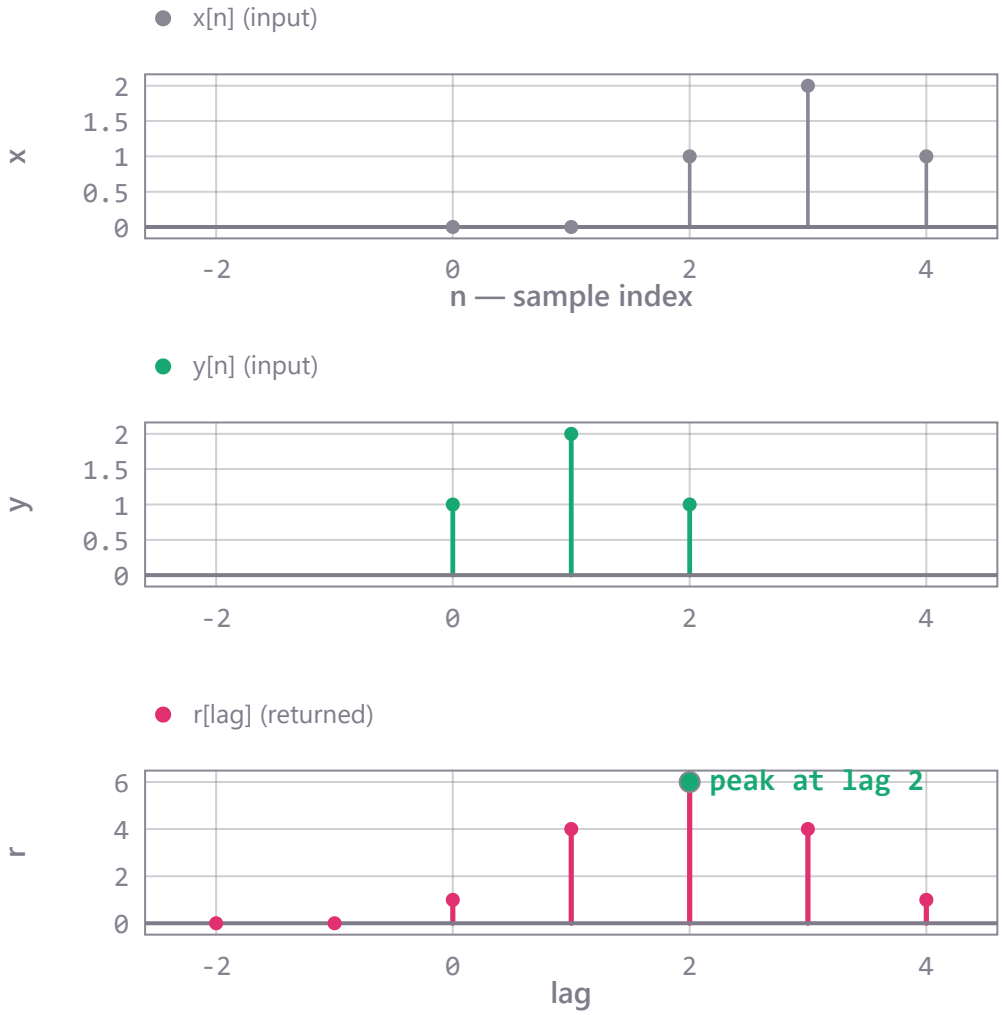
r_xy[l] = sum_n x[n+l] y[n] for every lag l, and the lag at which it is largest (the delay of x relative to y).

$$r_{xy}[\ell] = \sum_n x[n+\ell]\, y[n]$$

input	meaning	condition
`x`	first sequence x[n]	
`y`	second (reference) sequence y[n]	

output	meaning
`peak_lag`	lag of the largest correlation (positive: x lags y)
`peak_value`	largest correlation value
`r`	correlation at every lag
`lags`	the lags, -$(Ny-1)$ .. $Nx-1$

- **Method:** direct correlation sum over every lag (first lag on ties)
- **Assumes:** finite sequences starting at $n = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cross-correlation of x = [0, 0, 1, 2, 1] with y = [1, 2, 1]: the package returns r at lags -2 … 4, peaking at lag 2 with r = 6 — the shift at which y best lines up with x.

```
signal_processing_ai999x.solve('cross_correlation', x='0, 0, 1, 2, 1', y='1, 2, 1')
# -> peak_lag = 2, peak_value = 6, r = [0.0, 0.0, 1.0, 4.0, 6.0, 4.0, 1.0], lags = [-2, -1, 0, 1, 2, 3, 4]   (SOLVED)
```

```
signal_processing_ai999x.solve('cross_correlation', x=[1, 2, 3], y=[1, 1])
# -> peak_lag = 1, peak_value = 5, r = [1.0, 3.0, 5.0, 3.0], lags = [-1, 0, 1, 2]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the cross-correlation of x = [0, 0, 1, 2, 1] and y = [1, 2, 1] and the lag of its peak.	SOLVED	peak_lag = 2, peak_value = 6, r = [0.0, 0.0, 1.0, 4.0, 6.0, 4.0, 1.0], lags = [-2, -1, 0, 1, 2, 3, 4]
Cross-correlate x = [1, 2, 3] with y = [1, 1].	SOLVED	peak_lag = 1, peak_value = 5, r = [1.0, 3.0, 5.0, 3.0], lags = [-1, 0, 1, 2]

Moving-average filter output — `moving_average`

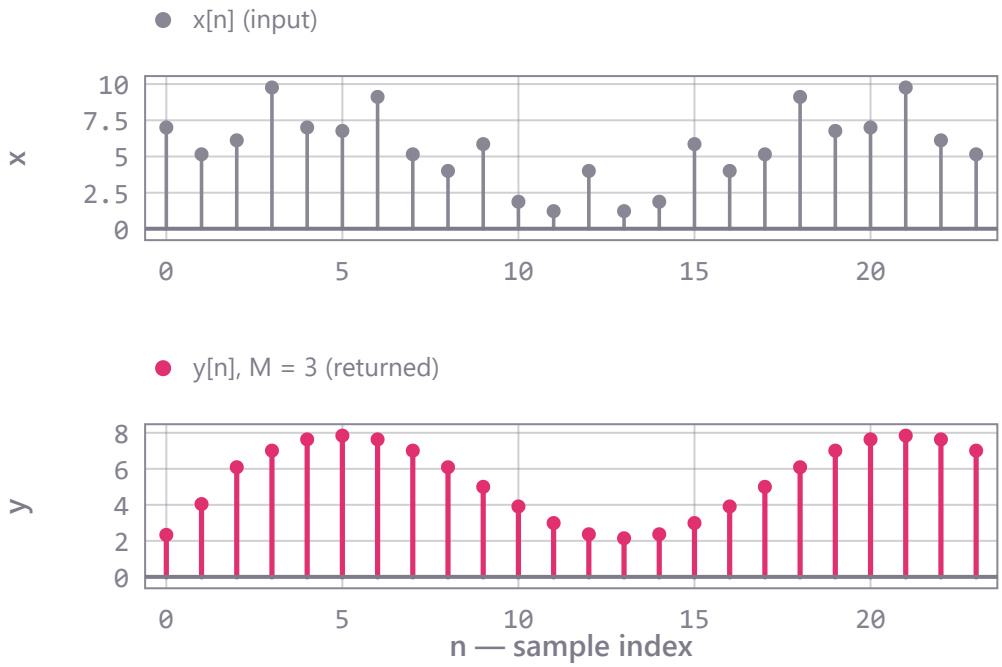
The causal M-point moving average y[n] = $(1/M)$ sum_{$i = 0$}^{$M - 1$} x[n-i], with x[n] = 0 before the first sample.

$$y[n] = \frac{1}{M} \sum_{i=0}^{M-1} x[n-i]$$

input	meaning	condition
`x`	input samples	
`M`	window length (points averaged)	

output	meaning
`y`	output samples (same length as x)

- **Method:** direct sum of the last M samples divided by M
- **Assumes:** $M \geq 1$; zero initial conditions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A 3-point moving average applied to a chosen signal x[n] = 5 + 3 sin(2πn/16) plus a ripple of +2 on every third sample and −1 elsewhere, n = 0 … 23 (input stated); the output y returned by the package keeps the slow wave and smooths the ripple. The worked example is x = [3, 6, 9, 12], M = 3.

```
signal_processing_ai999x.solve('moving_average', x='3, 6, 9, 12', M=3)
# -> y = [1.0, 3.0, 6.0, 9.0]   (SOLVED)
```

```
signal_processing_ai999x.solve('moving_average', x=[2, 4], M=2)
# -> y = [1.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply a 3-point moving average to x = [3, 6, 9, 12].	SOLVED	y = [1.0, 3.0, 6.0, 9.0]
Apply a moving average with a window length of 2 to the signal [2, 4].	SOLVED	y = [1.0, 3.0]
Apply a moving average to x = [3, 6, 9, 12].	NEEDS_INPUT	moving_average needs M (window length (points averaged)). Example: solve('moving_average', x='3, 6, 9, 12', M=3).

FIR filter output — `fir_filter_output`

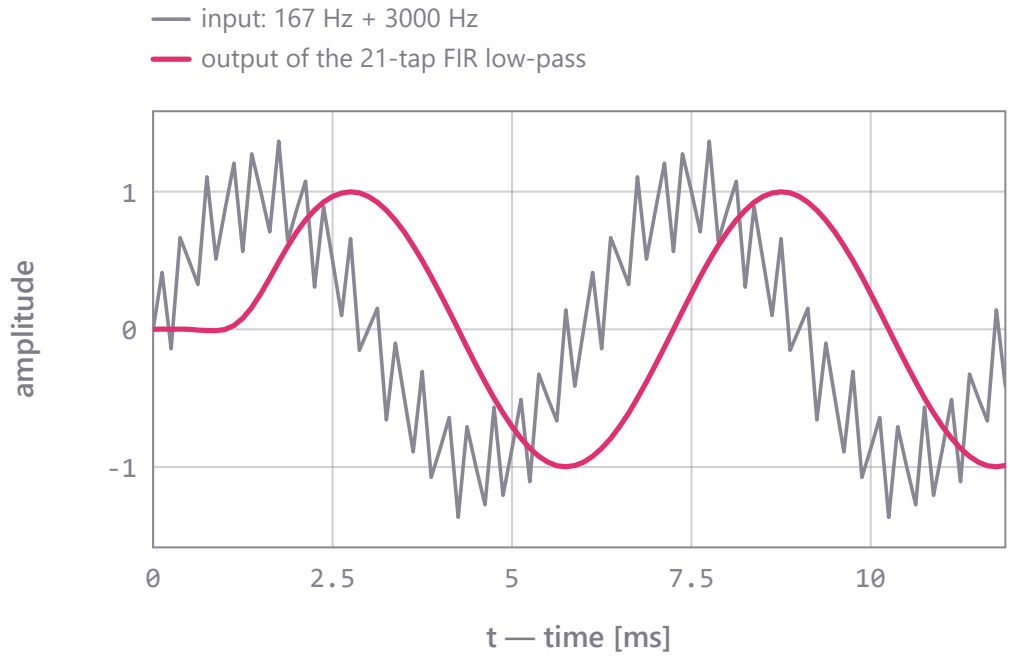
The output y[n] = sum_k b[k] x[n-k] of an FIR filter with coefficients b (zero initial conditions, same length as x).

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k]$$

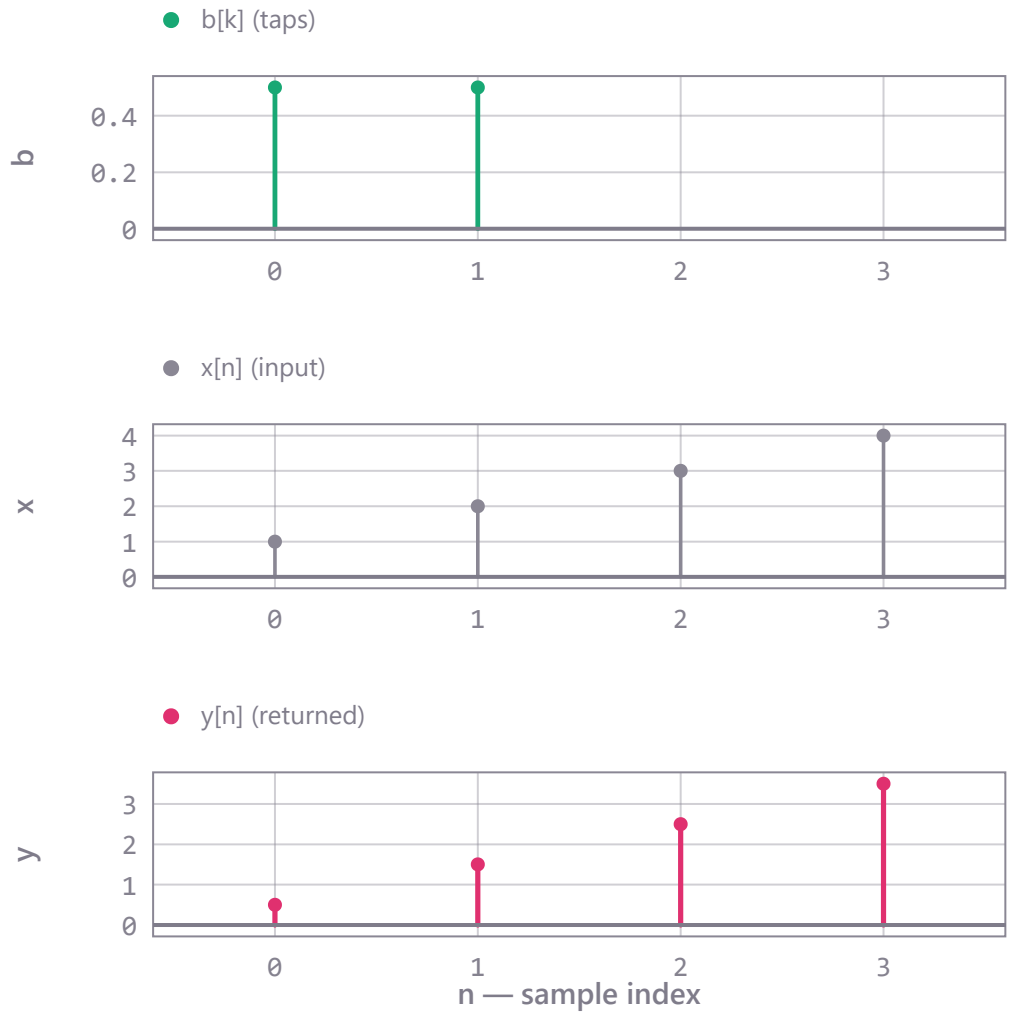
input	meaning	condition
`b`	filter coefficients (taps) b[0..M]	
`x`	input samples	

output	meaning
`y`	output samples

- **Method:** direct-form FIR difference equation
- **Assumes:** zero initial conditions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A sampled signal (fs = 8000 Hz; a 166.7 Hz tone plus a 3000 Hz tone of amplitude 0.4) and the output of the 21-tap Hamming-window FIR low-pass filter (fc = 1000 Hz) designed by the package, the output computed by it sample by sample: the 3000 Hz component is removed and the slow tone passes, delayed by 10 samples (1.25 ms).



The FIR filter b = [0.5, 0.5] applied to x = [1, 2, 3, 4] (the worked example): the package returns y = [0.5, 1.5, 2.5, 3.5], each output the taps-weighted sum of the current and previous inputs.

```
signal_processing_ai999x.solve('fir_filter_output', b='0.5, 0.5', x='1, 2, 3, 4')
# -> y = [0.5, 1.5, 2.5, 3.5]   (SOLVED)
```

```
signal_processing_ai999x.solve('fir_filter_output', b=[1, -1], x=[1, 4, 9, 16])
# -> y = [1.0, 3.0, 5.0, 7.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Filter x = [1, 2, 3, 4] with the FIR filter b = [0.5, 0.5].	SOLVED	y = [0.5, 1.5, 2.5, 3.5]
What is the output of the FIR filter b = [1, -1] for the input x = [1, 4, 9, 16]?	SOLVED	y = [1.0, 3.0, 5.0, 7.0]
Filter x = [1, 2, 3, 4] with an FIR filter.	NEEDS_INPUT	fir_filter_output needs b (filter coefficients (taps) b[0..M]). Example: solve('fir_filter_output', b='0.5, 0.5', x='1,

IIR filter output (difference equation) — `iir_filter_output`

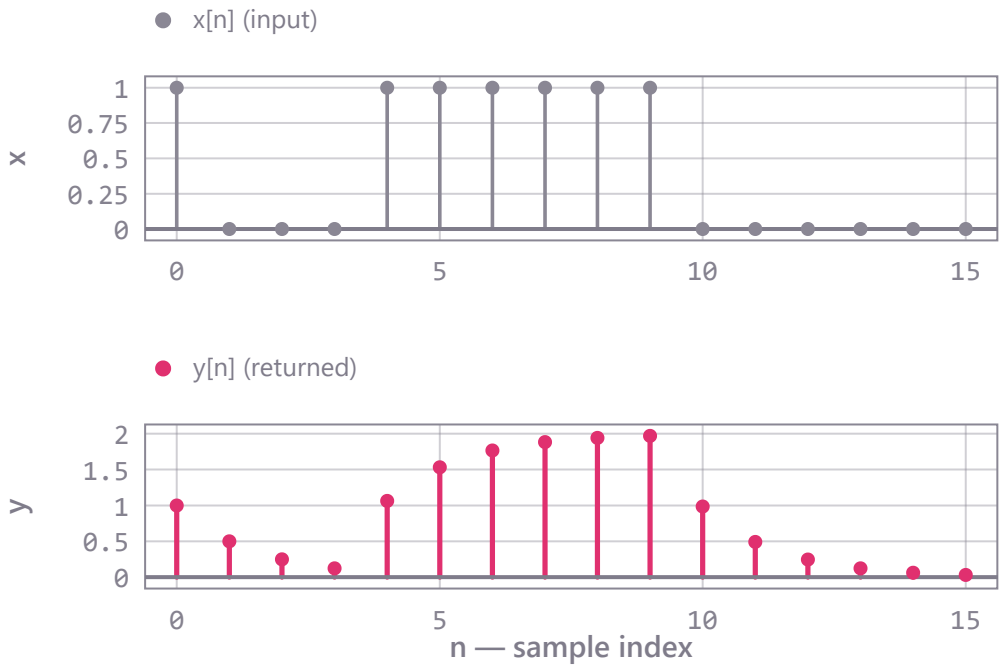
The output of the recursive filter a[0] y[n] = sum_k b[k] x[n-k] - sum_{$k \geq 1$} a[k] y[n-k] (zero initial conditions).

$$a_0 y[n] = \sum_k b_k x[n-k] - \sum_{k \geq 1} a_k y[n-k]$$

input	meaning	condition
b	feed-forward coefficients b	
a	feedback coefficients a (a[0] != 0)	
x	input samples	

output	meaning
y	output samples

- **Method:** direct-form I difference equation
- **Assumes:** a[0] != 0; zero initial conditions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The recursive filter b = [1], a = [1, -0.5] of the worked example driven by an input chosen for the drawing, x = [1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0] (an impulse, then a pulse of six samples): the package's output y decays geometrically after the impulse, rises toward its DC level during the pulse and decays after it; largest output 1.9707.

```
signal_processing_ai999x.solve('iir_filter_output', b='1', a='1, -0.5', x='1, 0, 0, 0')
# -> y = [1.0, 0.5, 0.25, 0.125]   (SOLVED)
```

```
signal_processing_ai999x.solve('iir_filter_output', b=[1, 1], a=[2, -1], x=[2, 2, 2])
# -> y = [1.0, 2.5, 3.25]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the output of the IIR filter with b = [1] and a = [1, -0.5] for the input x = [1, 0, 0, 0].	SOLVED	y = [1.0, 0.5, 0.25, 0.125]

question	outcome	result
Find the output of the IIR filter with b = [1] and a = [0, 1] for the input x = [1, 0].	INVALID_INPUT	IIR filter output (difference equation): a[0] must not be 0.
Find the output of the IIR filter with b = [1] for the input x = [1, 0, 0, 0].	NEEDS_INPUT	iir_filter_output needs a (feedback coefficients a (a[0] != 0)). Example: solve('iir_filter_output', b='1', a='1, -0.5',

Impulse response of a digital filter — `impulse_response`

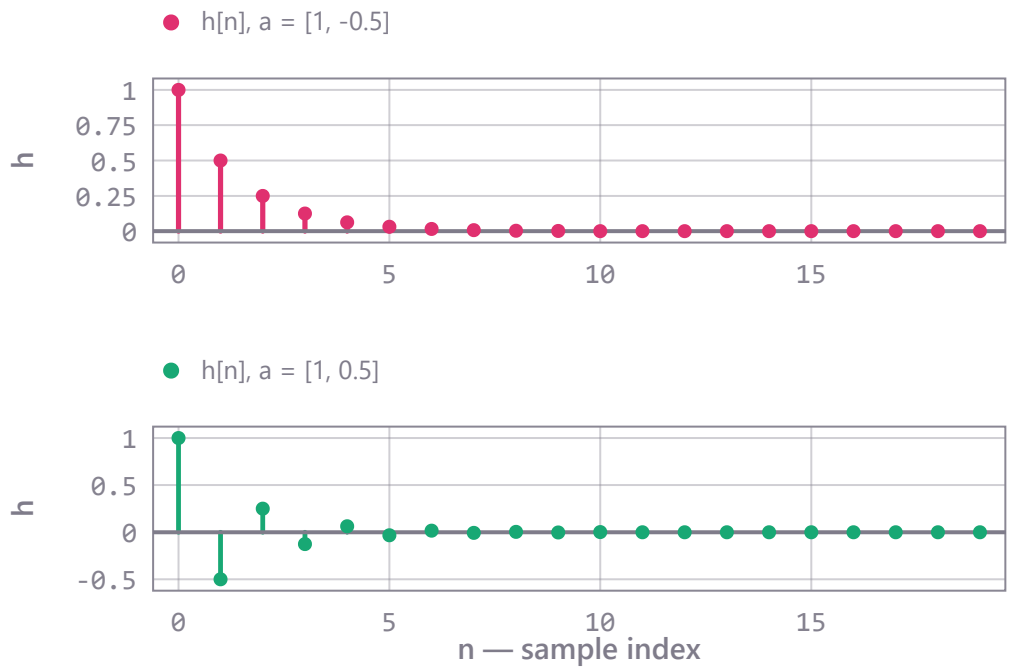
The first n samples of the impulse response h[n] of the filter with coefficients b and a.

$$h[n] = \mathcal{Z}^{-1}\{B(z)/A(z)\}$$

input	meaning	condition
b	numerator coefficients b	
a	denominator coefficients a (a[0] != 0)	
n	number of samples	

output	meaning
h	impulse response h[$0..n-1$]

- **Method:** the difference equation driven by a unit impulse
- **Assumes:** a[0] != 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Impulse responses returned by the package, 20 samples each: the worked example b = [1], a = [1, -0.5] (pole at 0.5: a geometric decay) and the mirrored filter a = [1, 0.5] (pole at -0.5: the same decay with alternating sign).

```
signal_processing_ai999x.solve('impulse_response', b='1', a='1, -0.5', n=5)
# -> h = [1.0, 0.5, 0.25, 0.125, 0.0625]   (SOLVED)
```

```
signal_processing_ai999x.solve('impulse_response', b=[1, 1], a=[1, -1, 0.5], n=4)
# -> h = [1.0, 2.0, 1.5, 0.5]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the first 5 samples of the impulse response of the filter with b = [1] and a = [1, -0.5].	SOLVED	h = [1.0, 0.5, 0.25, 0.125, 0.0625]
Find the impulse response of the filter with b = [1] and a = [1, -0.5].	NEEDS_INPUT	impulse_response needs n (number of samples). Example: solve('impulse_response', b='1', a='1, -0.5', n=5).

Step response of a digital filter — `step_response`

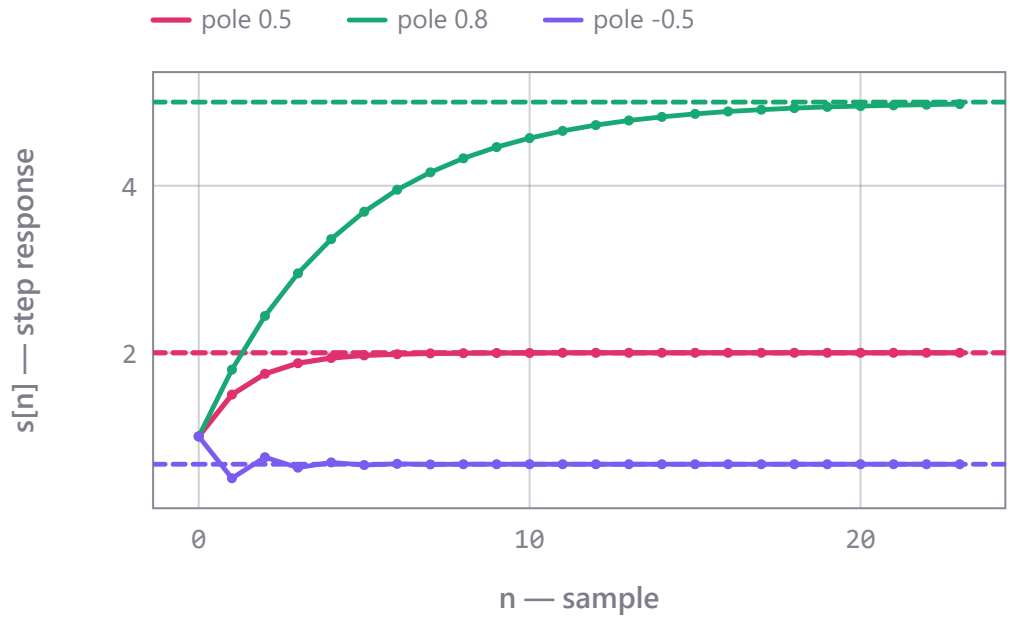
The first n samples of the response s[n] of the filter (b, a) to a unit step.

$$s[n] = \sum_{m=0}^{n} h[m]$$

input	meaning	condition
b	numerator coefficients b	
a	denominator coefficients a (a[0] != 0)	
n	number of samples	

output	meaning
s	step response s[$0..n-1$]

- **Method:** the difference equation driven by a unit step
- **Assumes:** a[0] != 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Step responses returned by the package (24 samples) for b = [1] with a one-pole denominator 1 − p z^{-1}, p = 0.5 (the worked example), 0.8 and −0.5; each settles at its DC gain (dashed, from the package's DC-gain capability): pole 0.5: settles at 2; pole 0.8: settles at 5; pole -0.5: settles at 0.666667. A pole nearer 1 is slower; a negative pole rings.

```
signal_processing_ai999x.solve('step_response', b='1', a='1, -0.5', n=4)
# -> s = [1.0, 1.5, 1.75, 1.875]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the first 4 samples of the step response of the filter with b = [1] and a = [1, -0.5].	SOLVED	s = [1.0, 1.5, 1.75, 1.875]

Frequency response of a digital filter at one frequency — `frequency_response`

H(e^{jw}) = B(e^{jw}) / A(e^{jw}) at $w = 2$ pi f / fs: magnitude, magnitude in dB and phase.

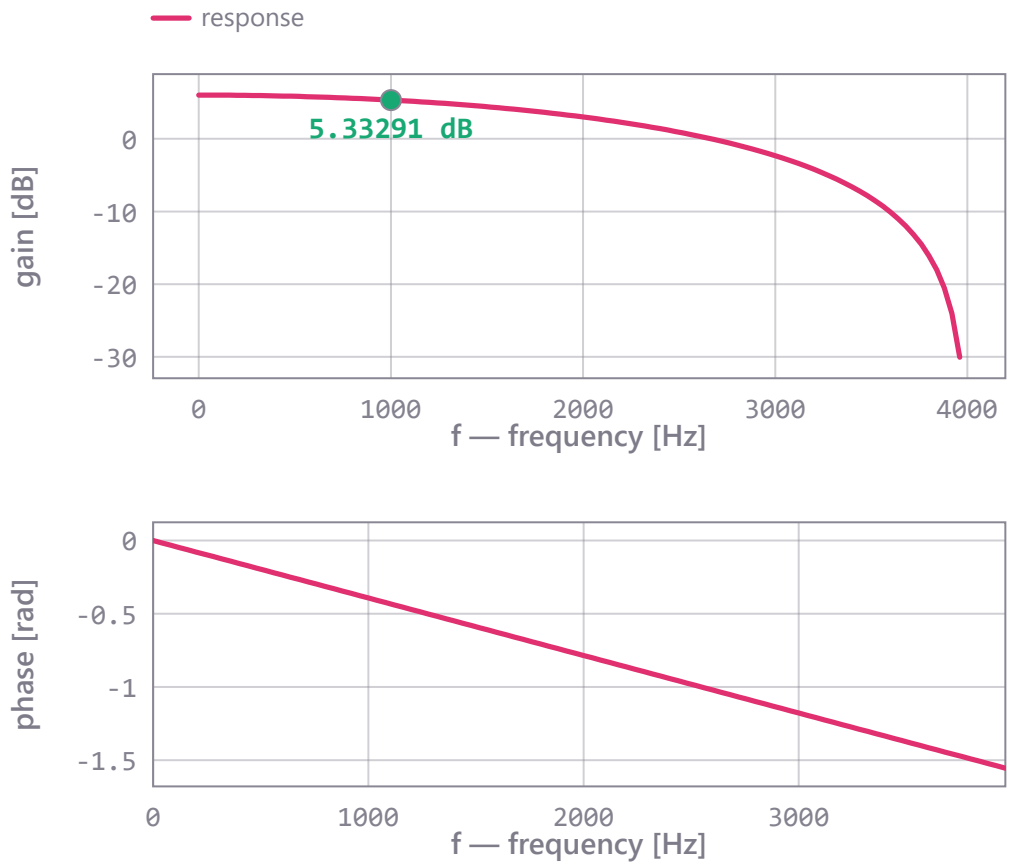
$$H(e^{j\omega}) = \frac{\sum_k b_k e^{-j\omega k}}{\sum_k a_k e^{-j\omega k}},\ \omega = 2\pi f / f_s$$

input	meaning	condition
`b`	numerator coefficients b	
`f`	frequency	
`fs`	sampling rate	
`a`	denominator coefficients a (1 if not given)	

output	meaning
`magnitude`	|H|
`magnitude_db`	20 log10 |H| in dB
`phase`	phase of H in radians

- **Method:** evaluation of the numerator and denominator polynomials in z^{-1} on the unit circle
- **Assumes:** $0 \le f \le fs/2$; no pole on the unit circle at f

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gain and phase of the filter b = [1, 1] at fs = 8000 Hz, from 0 to fs/2 (101 runs of the package). Marked: f = 1000 Hz, |H| = 1.84776 (5.33291 dB), phase -0.392699 rad.

```
signal_processing_ai999x.solve('frequency_response', b='1, 1', f=1000.0, fs=8000.0)
# -> magnitude = 1.84776, magnitude_db = 5.33291, phase = -0.392699 rad   (SOLVED)
```

```
signal_processing_ai999x.solve('frequency_response', b=[1], a=[1, -0.9], f=0.0, fs=1000.0)
# -> magnitude = 10, magnitude_db = 20, phase = 0 rad   (SOLVED)
```

```
signal_processing_ai999x.solve('frequency_response', b=[1, 1], f=4000.0, fs=8000.0)
# -> magnitude = 0, magnitude_db = -inf, phase = 0 rad   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the magnitude response of the filter b = [1, 1] at 1 kHz with a sampling rate of 8 kHz?	SOLVED	magnitude = 1.84776, magnitude_db = 5.33291, phase = -0.392699 rad
Find the frequency response of the filter with b = [1] and a = [1, -0.9] at 0 Hz with a sampling rate of 1 kHz.	SOLVED	magnitude = 10, magnitude_db = 20, phase = 0 rad
What is the magnitude response of the filter b = [1, 1] at 5 kHz with a sampling rate of 8 kHz?	INVALID_INPUT	Frequency response of a digital filter at one frequency: the frequency must lie between 0 and fs/2.

question	outcome	result
What is the magnitude response of the filter b = [1, 1] at 1 kHz?	NEEDS_INPUT	frequency_response needs fs (sampling rate). Example: solve('frequency_response', b='1, 1', f='1000 Hz', fs='8000 Hz').

Group delay of a digital filter — `group_delay`

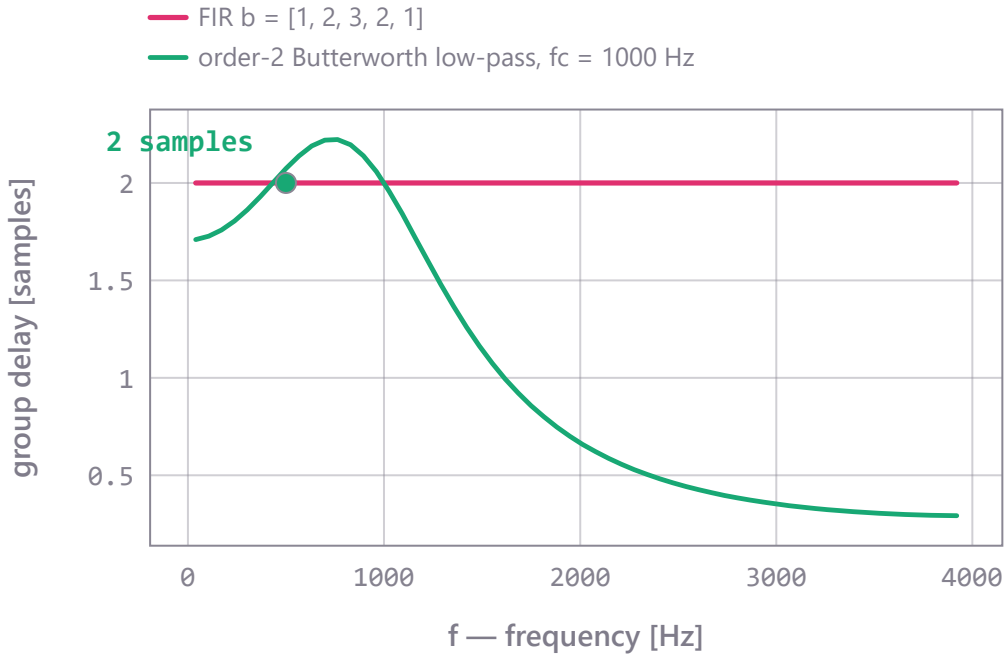
The group delay -d(phase)/dw of H(z) = B(z)/A(z) at frequency f, in samples and in seconds.

$$\tau_g(\omega) = -\frac{d\angle H(e^{j\omega})}{d\omega} = \mathrm{Re}\frac{\sum k b_k e^{-j\omega k}}{\sum b_k e^{-j\omega k}} - \mathrm{Re}\frac{\sum k a_k e^{-j\omega k}}{\sum a_k e^{-j\omega k}}$$

input	meaning	condition
`b`	numerator coefficients b	
`f`	frequency	
`fs`	sampling rate	
`a`	denominator coefficients a (1 if not given)	

output	meaning
`samples`	group delay in samples
`seconds`	group delay in seconds

- **Method:** $\tau = Re$[sum k b_k z^-k / sum b_k z^{-k}] - Re[the same for a] on the unit circle
- **Assumes:** $0 \le f \le fs/2$; no zero or pole of H on the unit circle at f
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Group delay against frequency at fs = 8000 Hz, 60 runs of the package per curve: the symmetric FIR filter b = [1, 2, 3, 2, 1] delays every frequency by the same 2 samples (linear phase), while the order-2 Butterworth low-pass designed by the package (b = [0.09763, 0.1953, 0.09763], a = [1, -0.9428, 0.3333]) delays frequencies near its cutoff most. Marked: the worked example, f = 500.0 Hz, 2 samples = 0.00025 s.

```
signal_processing_ai999x.solve('group_delay', b='1, 2, 3, 2, 1', f=500.0, fs=8000.0)
# -> samples = 2, seconds = 0.00025 s   (SOLVED)
```

```
signal_processing_ai999x.solve('group_delay', b=[1], a=[1, -0.5], f=0.0, fs=1000.0)
# -> samples = 1, seconds = 0.001 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the group delay of the FIR filter b = [1, 2, 3, 2, 1] at 500 Hz with a sampling rate of 8 kHz.	SOLVED	samples = 2, seconds = 0.00025 s
What is the group delay of the filter b = [1] and a = [1, -0.5] at 0 Hz with a sampling rate of 1 kHz?	SOLVED	samples = 1, seconds = 0.001 s

DC gain of a digital filter — `dc_gain`

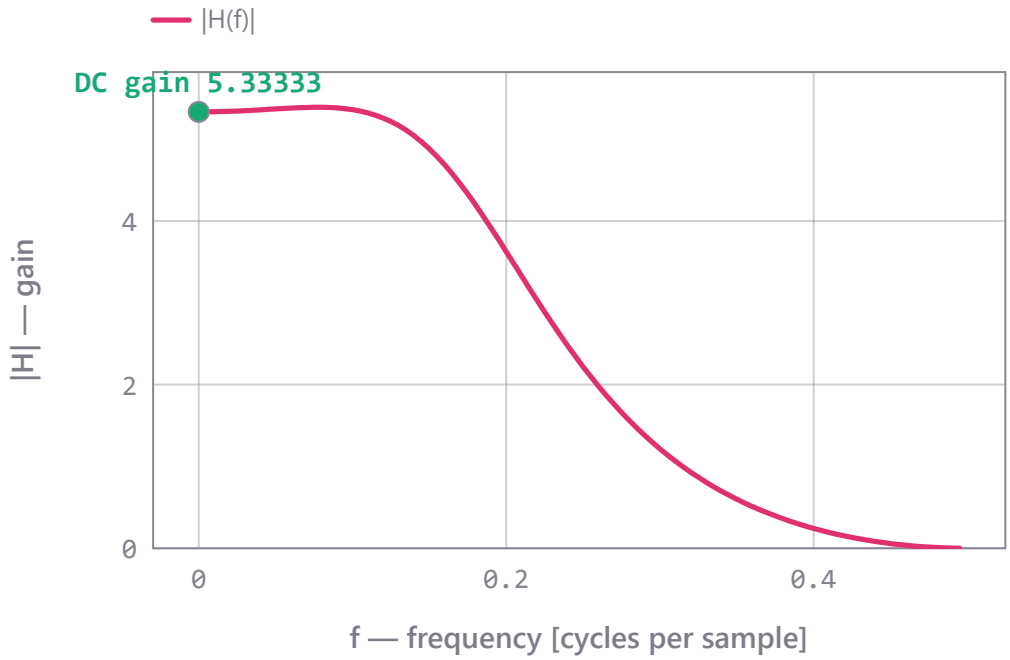
The gain H(1) = sum b / sum a of the filter at zero frequency, and its level in dB.

$$H(1) = \frac{\sum_k b_k}{\sum_k a_k}$$

input	meaning	condition
`b`	numerator coefficients b	
`a`	denominator coefficients a (1 if not given)	

output	meaning
`gain`	DC gain H(z = 1) (signed)
`gain_db`	20 log10 |H(1)| in dB

- **Method:** H(1) = sum b_k / sum a_k
- **Assumes:** sum of $a \neq 0$ (no pole at z = 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The gain of the filter b = [1, 2, 1], a = [1, -0.5, 0.25] from 0 to half the sampling rate (101 frequency-response runs of the package, fs = 1): at f = 0 it equals the returned DC gain 5.33333 (14.54 dB), marked.

```
signal_processing_ai999x.solve('dc_gain', b='1, 2, 1', a='1, -0.5, 0.25')
# -> gain = 5.33333, gain_db = 14.54   (SOLVED)
```

```
signal_processing_ai999x.solve('dc_gain', b=[0.25, 0.25, 0.25, 0.25])
# -> gain = 1, gain_db = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the DC gain of the filter with b = [1, 2, 1] and a = [1, -0.5, 0.25]?	SOLVED	gain = 5.33333, gain_db = 14.54
What is the DC gain of the filter with b = [1] and a = [1, -1]?	INVALID_INPUT	DC gain of a digital filter: the sum of the denominator coefficients is 0 (a pole at z = 1): the DC gain is infinite.

Magnitude of an analog Butterworth low-pass response — `butterworth_magnitude`

|H(f)| = $1/sqrt(1 + (f/fc)^{2N})$ of an N-th order Butterworth low-pass filter, and the attenuation in dB.

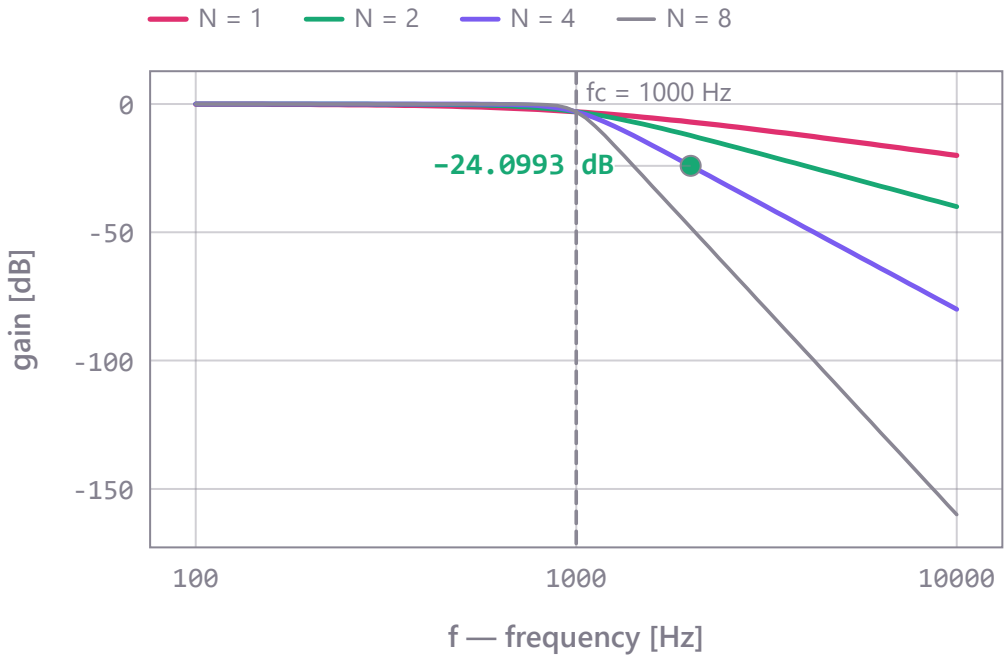
$$|H(f)| = \frac{1}{\sqrt{1 + (f/f_c)^{2N}}}$$

input	meaning	condition
`N`	filter order	
`fc`	cutoff (-3 dB) frequency	
`f`	frequency	

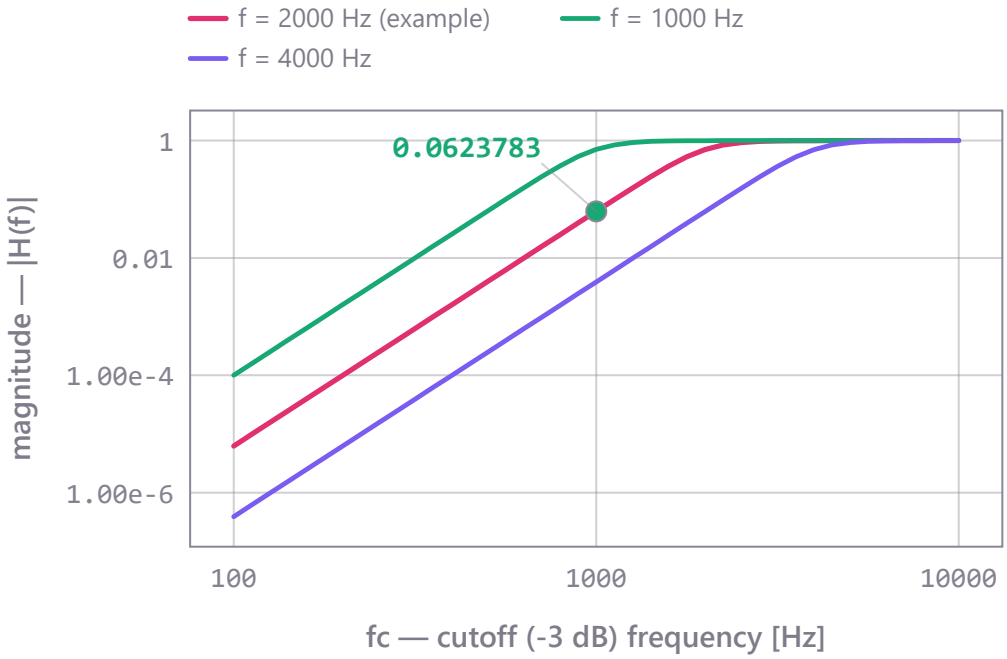
output	meaning
`magnitude`	|H(f)|

output	meaning
`attenuation_db`	attenuation in dB (positive)

- **Method:** the Butterworth magnitude-squared function
- **Assumes:** $N \geq 1$; $fc > 0$; $f \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Gain of Butterworth low-pass filters of order N = 1, 2, 4 and 8 with cutoff fc = 1000 Hz (each curve 81 frequencies computed by the package, 0.1 fc to 10 fc on a log axis); every order is 3 dB down at fc and falls by 20N dB per decade. Marked: f = 2000 Hz, N = 4, gain = −24.0993 dB.



magnitude (|H(f)|) against fc (cutoff (-3 dB) frequency) from 100 Hz to 10000 Hz on a logarithmic axis, logarithmic output axis. One curve for each value of f (frequency): 2000 Hz, 1000 Hz, 4000 Hz. Other inputs as in the worked example: N = 4. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, fc = 1000 Hz, magnitude = 0.0623783.

```
signal_processing_ai999x.solve('butterworth_magnitude', N=4, fc=1000.0, f=2000.0)
# -> magnitude = 0.0623783, attenuation_db = 24.0993   (SOLVED)
```

```
signal_processing_ai999x.solve('butterworth_magnitude', N=3, fc=10000.0, f=30000.0)
# -> magnitude = 0.0370117, attenuation_db = 28.6332   (SOLVED)
```

```
signal_processing_ai999x.solve('butterworth_magnitude', N=5, fc=1000.0, f=1000.0)
# -> magnitude = 0.707107, attenuation_db = 3.0103   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the magnitude response of a 4th-order Butterworth low-pass filter with a cutoff frequency of 1 kHz at 2 kHz?	SOLVED	magnitude = 0.0623783, attenuation_db = 24.0993
Find the attenuation of a Butterworth filter of order 3 with a cutoff of 10 kHz at 30 kHz.	SOLVED	magnitude = 0.0370117, attenuation_db = 28.6332
Find the attenuation of a Butterworth filter of order 3 at 30 kHz.	NEEDS_INPUT	butterworth_magnitude needs fc (cutoff (-3 dB) frequency). Example: solve('butterworth_magnitude', N=4, fc='1000 Hz', f=

Minimum order of a analog Butterworth filter — `butterworth_order_analog`

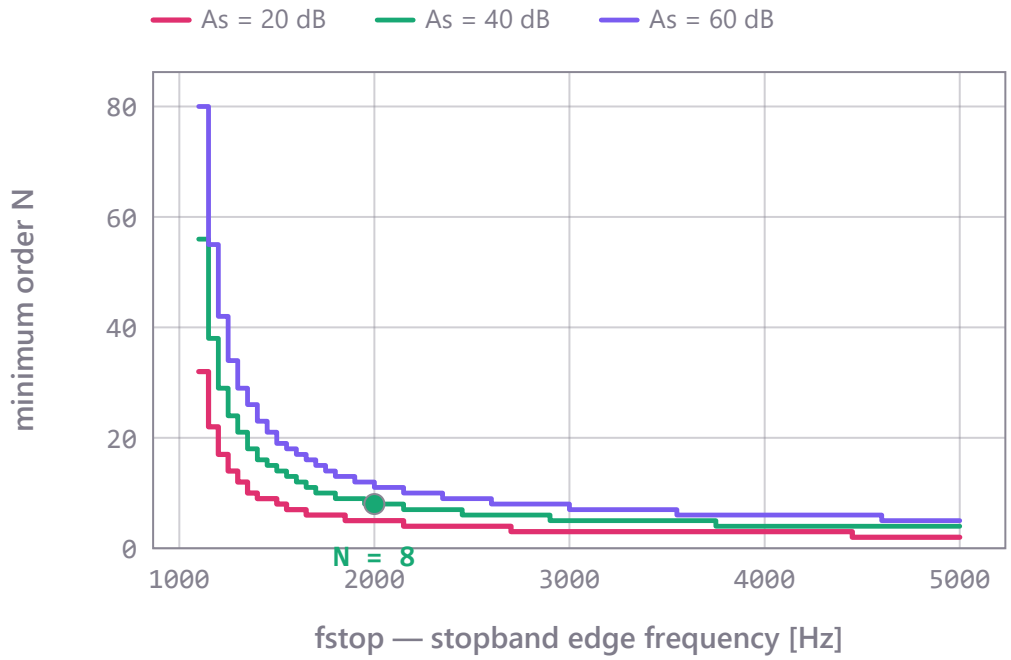
The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB.

$$N \geq \frac{\log_{10} \frac{10^{A_s/10}-1}{10^{R_p/10}-1}}{2\log_{10}(\omega_s/\omega_p)}$$

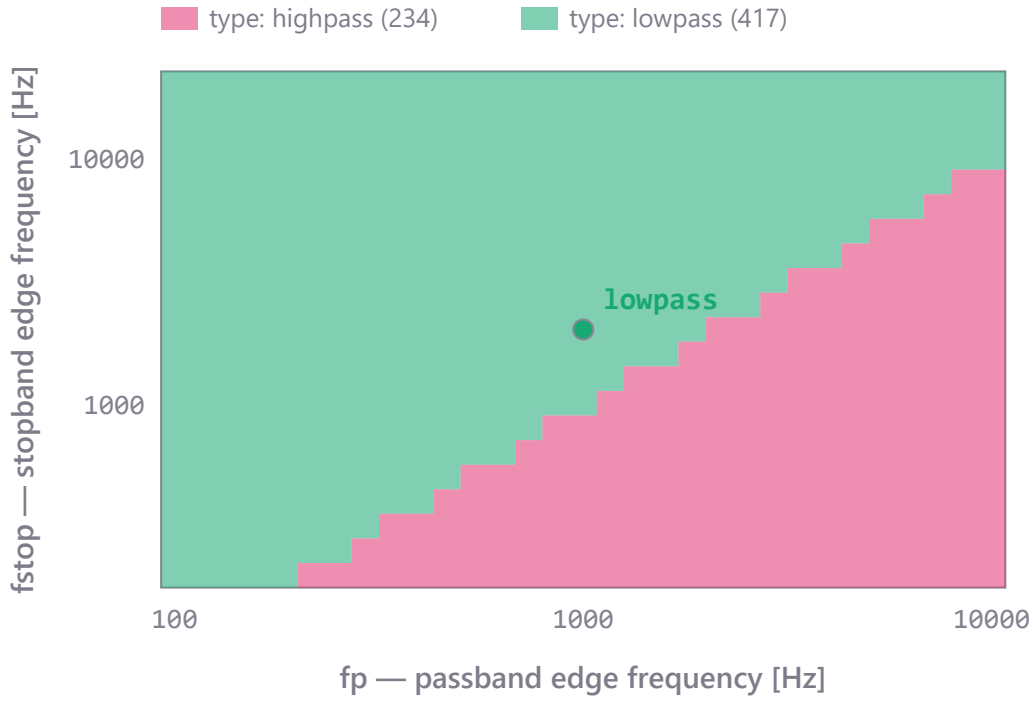
input	meaning	condition
`fp`	passband edge frequency	
`fstop`	stopband edge frequency	
`Rp`	maximum passband attenuation (ripple) in dB	
`As`	minimum stopband attenuation in dB	

output	meaning
`order`	minimum filter order
`type`	lowpass (fp < fstop) or highpass (fp > fstop)

- **Method:** N = ceil(log10($(10^{As/10}-1)/(10^{Rp/10}-1)$) / (2 log10(ws/wp)))
- **Assumes:** $0 < Rp < As$; fp != fstop
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The smallest Butterworth order meeting a passband edge fp = 1000 Hz with at most Rp = 1 dB ripple and a stopband attenuation As at fstop, for fstop from 1100 to 5000 Hz (79 runs of the package per curve, drawn as steps): a sharper transition or more attenuation needs a higher order. Marked: the worked example fstop = 2000.0 Hz, As = 40.0 dB, N = 8 (lowpass).



lowpass (fp < fstop) or highpass (fp > fstop) (type) over fp (passband edge frequency) from 100 Hz to 10000 Hz and fstop (stopband edge frequency) from 200 Hz to 20000 Hz (logarithmic axes): each of the 31 × 21 cells is one run of signal-processing-ai999x, coloured by the verdict it returned — highpass: 234 cells, lowpass: 417 cells. Other inputs as in the worked example: Rp = 1.0, As = 40.0. Marked: the worked example, fp = 1000 Hz, fstop = 2000 Hz → lowpass.

```
signal_processing_ai999x.solve('butterworth_order_analog', fp=1000.0, fstop=2000.0, Rp=1.0,
As=40.0)
# -> order = 8, type = lowpass   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What Butterworth filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz with at least 40 dB attenuation?	SOLVED	order = 8, type = lowpass
What Butterworth filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz?	NEEDS_INPUT	butterworth_order_analog needs As (minimum stopband attenuation in dB). Example: solve('butterworth_order_analog', fp='1

Minimum order of a digital (bilinear) Butterworth filter — `butterworth_order_digital`

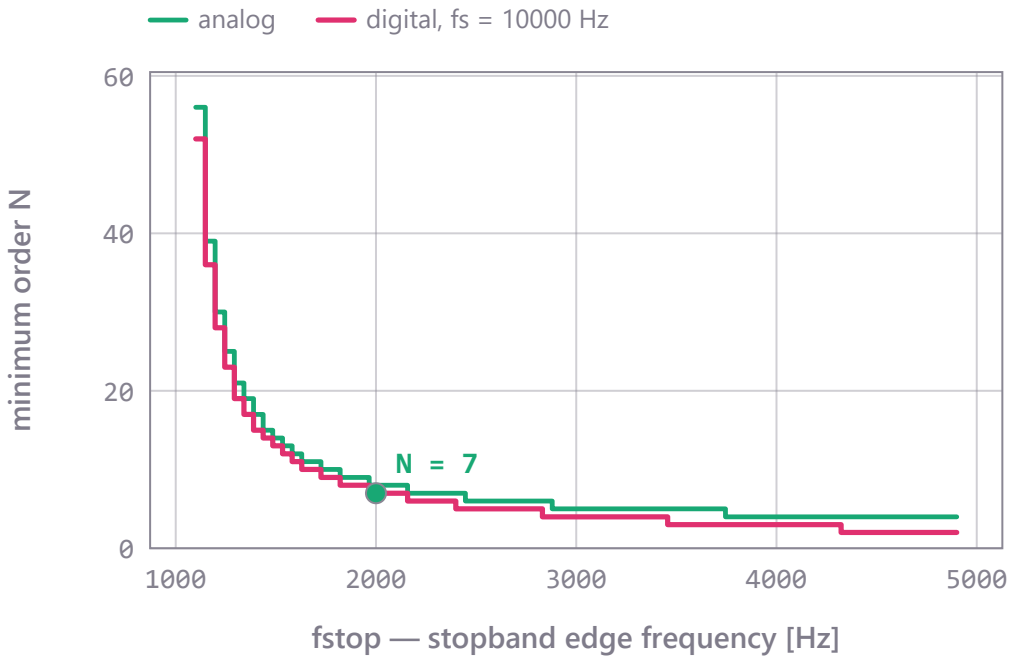
The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB (digital design by the bilinear transform: edges pre-warped with tan(pi f / fs)).

$$N \geq \frac{\log_{10} \frac{10^{A_s/10}-1}{10^{R_p/10}-1}}{2\log_{10}(\omega_s/\omega_p)}$$

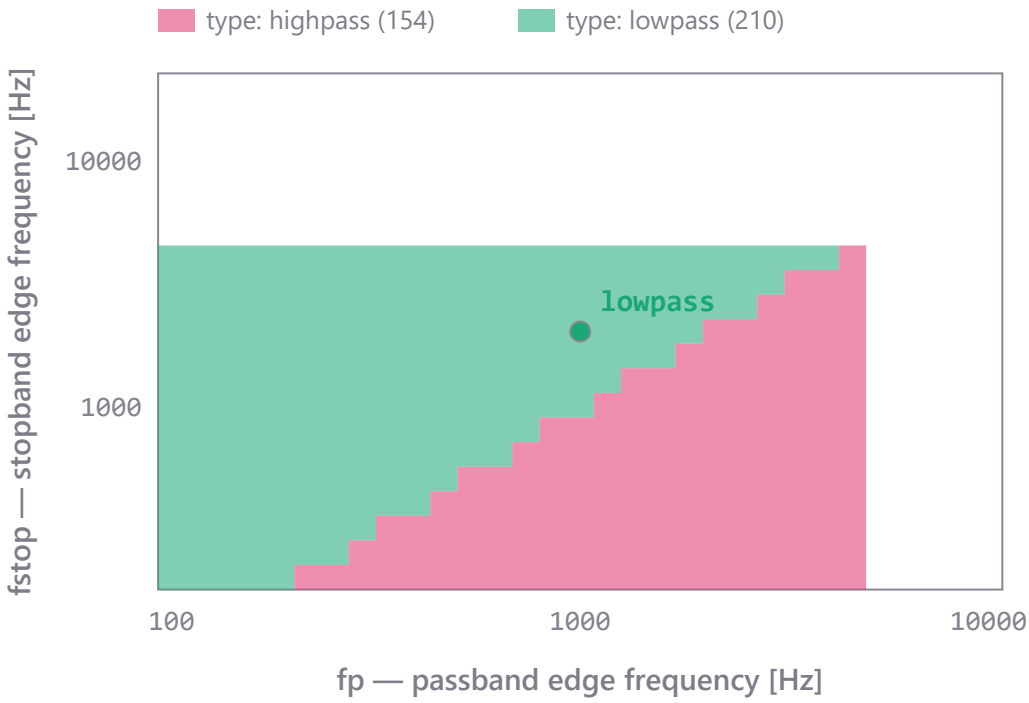
input	meaning	condition
`fp`	passband edge frequency	
`fstop`	stopband edge frequency	
`fs`	sampling rate	
`Rp`	maximum passband attenuation (ripple) in dB	
`As`	minimum stopband attenuation in dB	

output	meaning
`order`	minimum filter order
`type`	lowpass (fp < fstop) or highpass (fp > fstop)

- **Method:** N = ceil(log10($(10^{As/10}-1)/(10^{Rp/10}-1)$) / (2 log10(ws/wp))), with $wp = tan$(pi fp/fs), $ws = tan$(pi fstop/fs)
- **Assumes:** $0 < Rp < As$; fp != fstop; both edges below $fs/2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The smallest Butterworth order for fp = 1000 Hz, Rp = 1.0 dB, As = 40.0 dB as the stopband edge moves toward fs/2 = 5000 Hz: the digital design (bilinear transform at fs = 10000 Hz) against the analog one, 80 runs of the package each — frequency warping near fs/2 lets the digital filter use a lower order. Marked: fstop = 2000.0 Hz, N = 7.



lowpass (fp < fstop) or highpass (fp > fstop) (type) over fp (passband edge frequency) from 100 Hz to 10000 Hz and fstop (stopband edge frequency) from 200 Hz to 20000 Hz (logarithmic axes): each of the 31 × 21 cells is one run of signal-processing-ai999x, coloured by the verdict it returned — highpass: 154 cells, lowpass: 210 cells; blank cells: inputs the capability refuses. Other inputs as in the worked example: fs = 10000.0 Hz, Rp = 1.0, As = 40.0. Marked: the worked example, fp = 1000 Hz, fstop = 2000 Hz → lowpass.

```
signal_processing_ai999x.solve('butterworth_order_digital', fp=1000.0, fstop=2000.0, Rp=1.0,
As=40.0, fs=10000.0)
# -> order = 7, type = lowpass   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What Butterworth filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz with at least 40 dB attenuation at a sampling rate of 10 kHz?	SOLVED	order = 7, type = lowpass
What Butterworth filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz at a sampling rate of 10 kHz?	NEEDS_INPUT	butterworth_order_digital needs As (minimum stopband attenuation in dB). Example: solve('butterworth_order_digital', fp=

Minimum order of a analog Chebyshev type I filter — `chebyshev1_order_analog`

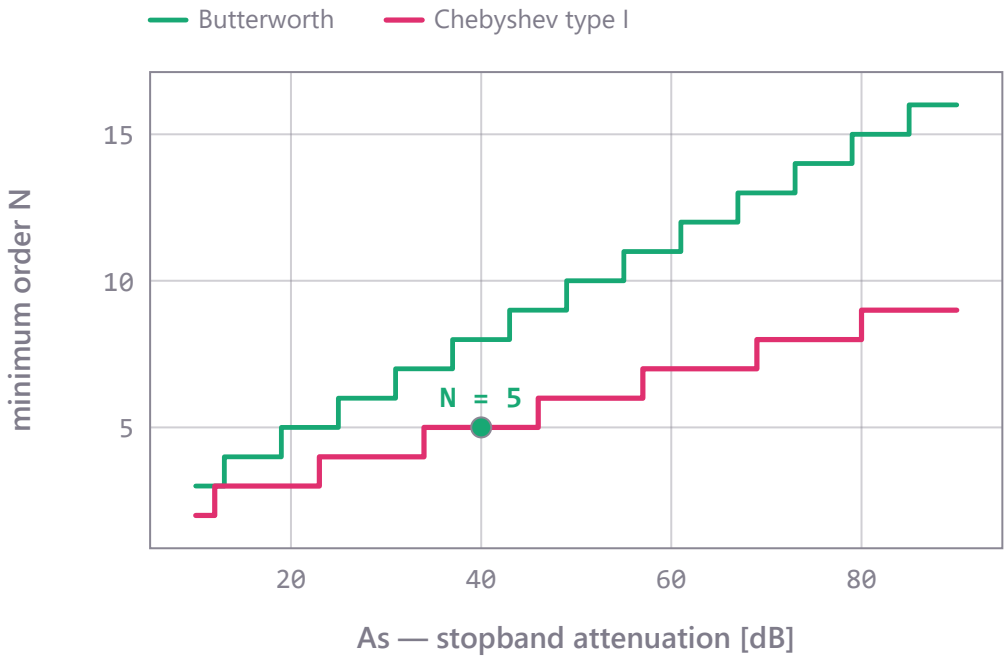
The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB.

$$N \geq \frac{\cosh^{-1}\sqrt{\frac{10^{A_s/10}-1}{10^{R_p/10}-1}}}{\cosh^{-1}(\omega_s/\omega_p)}$$

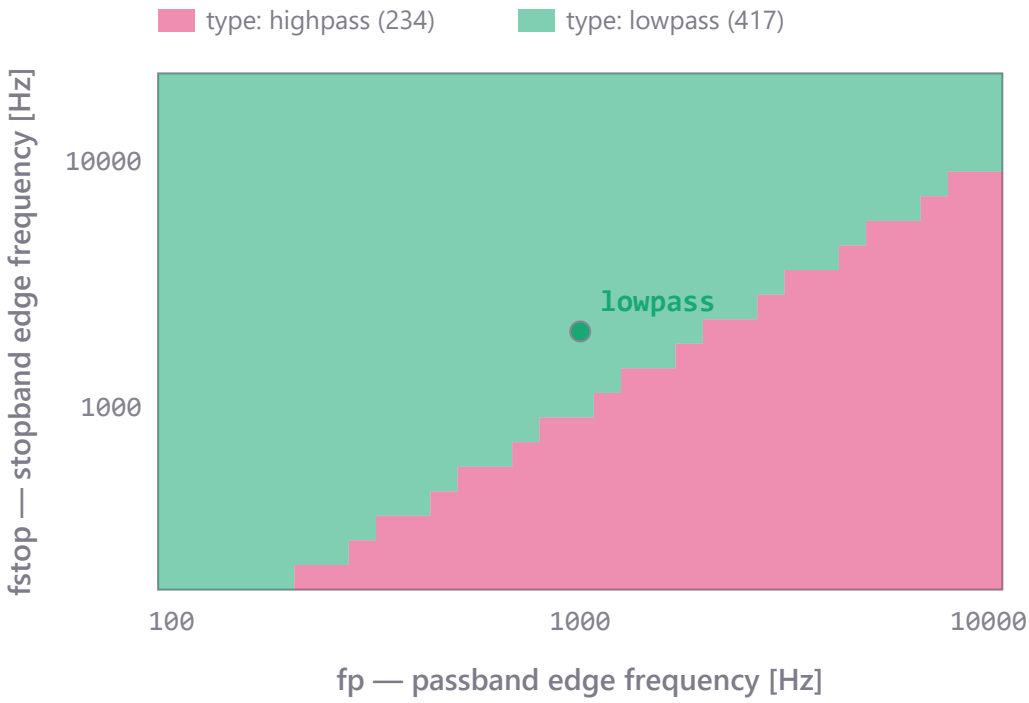
input	meaning	condition
`fp`	passband edge frequency	
`fstop`	stopband edge frequency	
`Rp`	maximum passband attenuation (ripple) in dB	
`As`	minimum stopband attenuation in dB	

output	meaning
`order`	minimum filter order
`type`	lowpass (fp < fstop) or highpass (fp > fstop)

- **Method:** N = ceil(acosh(sqrt($(10^{As/10}-1)/(10^{Rp/10}-1)$)) / acosh(ws/wp))
- **Assumes:** $0 < Rp < As$; fp != fstop
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The smallest analog filter order for passband edge fp = 1000.0 Hz (ripple Rp = 1.0 dB) and stopband edge fstop = 2000.0 Hz, as the stopband attenuation As grows from 10 to 90 dB: Chebyshev type I against Butterworth, 81 runs of the package each, drawn as steps — the equiripple passband buys a lower order. Marked: As = 40.0 dB, Chebyshev N = 5.



lowpass (fp < fstop) or highpass (fp > fstop) (type) over fp (passband edge frequency) from 100 Hz to 10000 Hz and fstop (stopband edge frequency) from 200 Hz to 20000 Hz (logarithmic axes): each of the 31 × 21 cells is one run of signal-processing-ai999x, coloured by the verdict it returned — highpass: 234 cells, lowpass: 417 cells. Other inputs as in the worked example: Rp = 1.0, As = 40.0. Marked: the worked example, fp = 1000 Hz, fstop = 2000 Hz → lowpass.

```
signal_processing_ai999x.solve('chebyshev1_order_analog', fp=1000.0, fstop=2000.0, Rp=1.0, As=40.0)
# -> order = 5, type = lowpass   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What Chebyshev filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz with at least 40 dB attenuation?	SOLVED	order = 5, type = lowpass
What Chebyshev filter order is needed for a passband edge of 1 kHz with at most 1 dB ripple and a stopband edge of 2 kHz?	NEEDS_INPUT	chebyshev1_order_analog needs As (minimum stopband attenuation in dB). Example: solve('chebyshev1_order_analog', fp='100

Poles and stability of a second-order all-pole system — `second_order_poles`

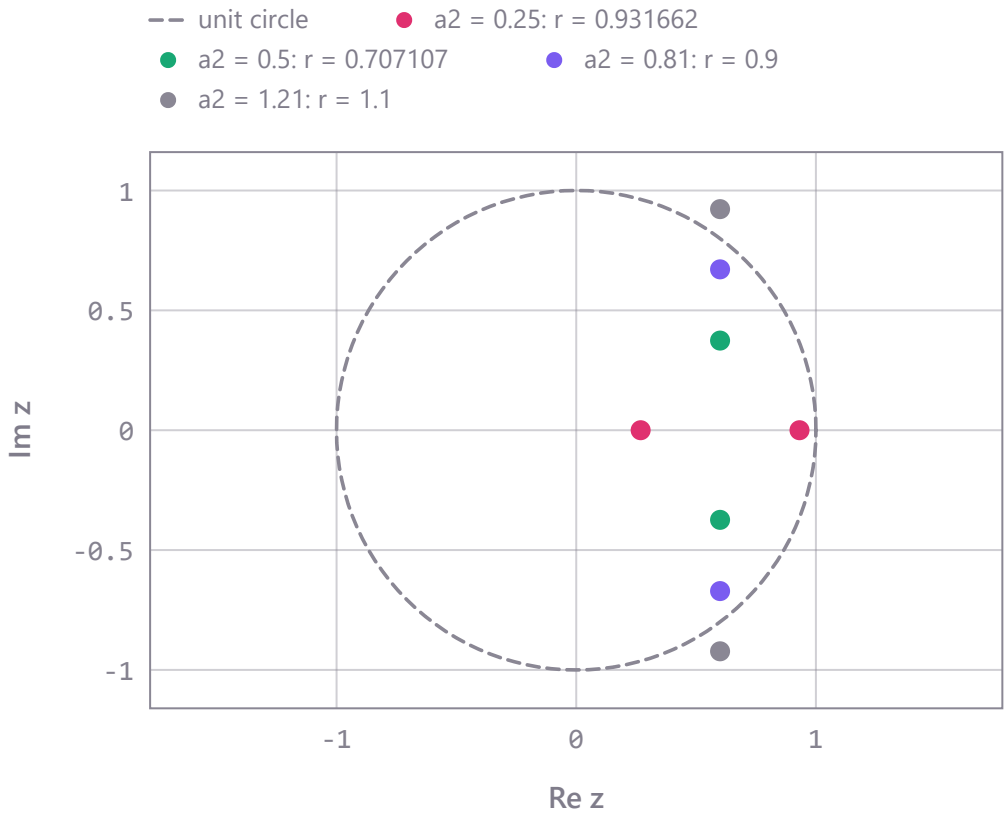
The poles of H(z) $= 1/(1 + a_1\ z^{-1} + a_2$ z^-2) (roots of $z^2 + a_1$ z + a2), their largest radius and whether the causal system is stable.

$$z^2 + a_1 z + a_2 = 0, \quad \text{stable} \iff |p_i| < 1$$

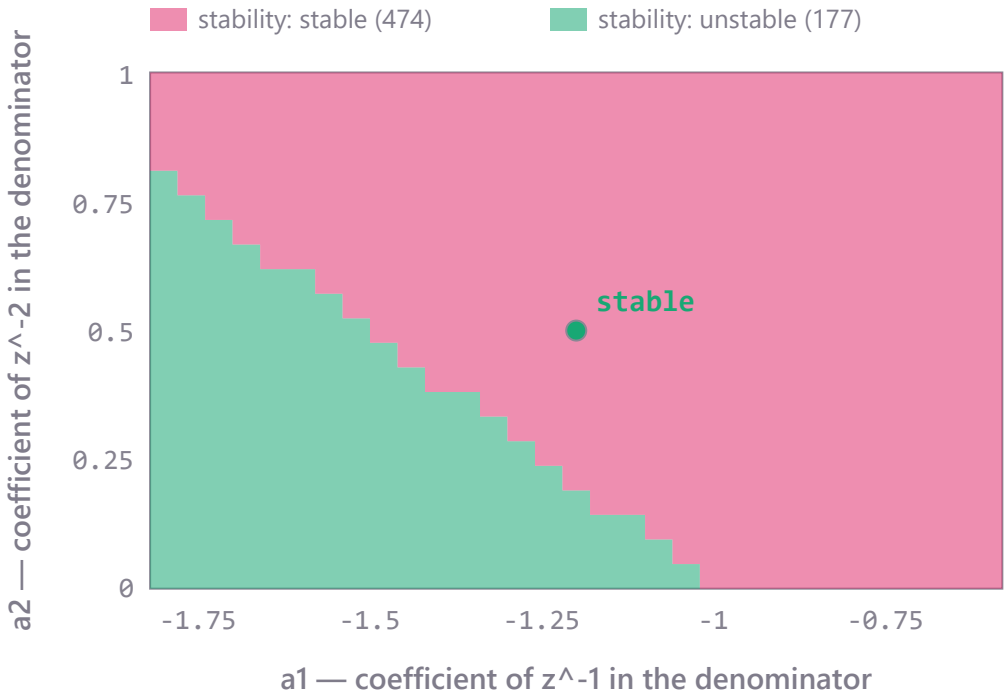
input	meaning	condition
`a1`	coefficient of z^{-1} in the denominator	
`a2`	coefficient of z^{-2} in the denominator	$a_2 \neq 0$

output	meaning
`radius`	largest pole radius |p|
`p1`	first pole (smaller real, or positive imaginary part)
`p2`	second pole
`stability`	stable if every pole lies inside the unit circle

- **Method:** roots of $z^2 + a_1\ z + a_2$ by the numerically stable quadratic formula
- **Assumes:** $a_2 \neq 0$; a causal system (stable iff \|p\| < 1 for every pole)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Poles of $1/(1 + a1\ z^{-1} + a2\ z^{-2})$ returned by the package for a1 = -1.2 and four values of a2 (the worked example is a2 = 0.5): a2 = 0.25: 0.268338 ± 0j, radius 0.931662, stable; a2 = 0.5: 0.6 ± 0.374166j, radius 0.707107, stable; a2 = 0.81: 0.6 ± 0.67082j, radius 0.9, stable; a2 = 1.21: 0.6 ± 0.921954j, radius 1.1, unstable. The pole radius is √a2 for complex poles; outside the unit circle the filter is unstable.



stable if every pole lies inside the unit circle (stability) over a1 (coefficient of z^-1 in the denominator) from -1.8 to -0.6 and a2 (coefficient of z^-2 in the denominator) from 0.02 to 0.98: each of the 31 × 21 cells is one run of signal-processing-ai999x, coloured by the verdict it returned — stable: 474 cells, unstable: 177 cells. Marked: the worked example, a1 = -1.2, a2 = 0.5 → stable.

```
signal_processing_ai999x.solve('second_order_poles', a1=-1.2, a2=0.5)
# -> radius = 0.707107, p1 = 0.6 + 0.374166i, p2 = 0.6 - 0.374166i, stability = stable   (SOLVED)
```

```
signal_processing_ai999x.solve('second_order_poles', a1=-2.5, a2=1.0)
# -> radius = 2, p1 = 0.5, p2 = 2, stability = unstable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the poles of H(z) = 1/(1 - 1.2z^-1 + 0.5z^-2) and say whether the system is stable.	SOLVED	radius = 0.707107, p1 = 0.6 + 0.374166i, p2 = 0.6 - 0.374166i, stability = stable
Find the poles of the second-order system with a1 = -2.5 and a2 = 1.	SOLVED	radius = 2, p1 = 0.5, p2 = 2, stability = unstable
Find the poles of H(z) = 1/(1 + 0.81z^-2).	SOLVED	radius = 0.9, p1 = 0 + 0.9i, p2 = 0 - 0.9i, stability = stable
Find the poles of the second-order system with a1 = -1.2.	NEEDS_INPUT	second_order_poles needs a2 (coefficient of z^-2 in the denominator). Example: solve('second_order_poles', a1=-1.2, a2=0

Poles, zeros and stability of a digital filter — `poles_zeros`

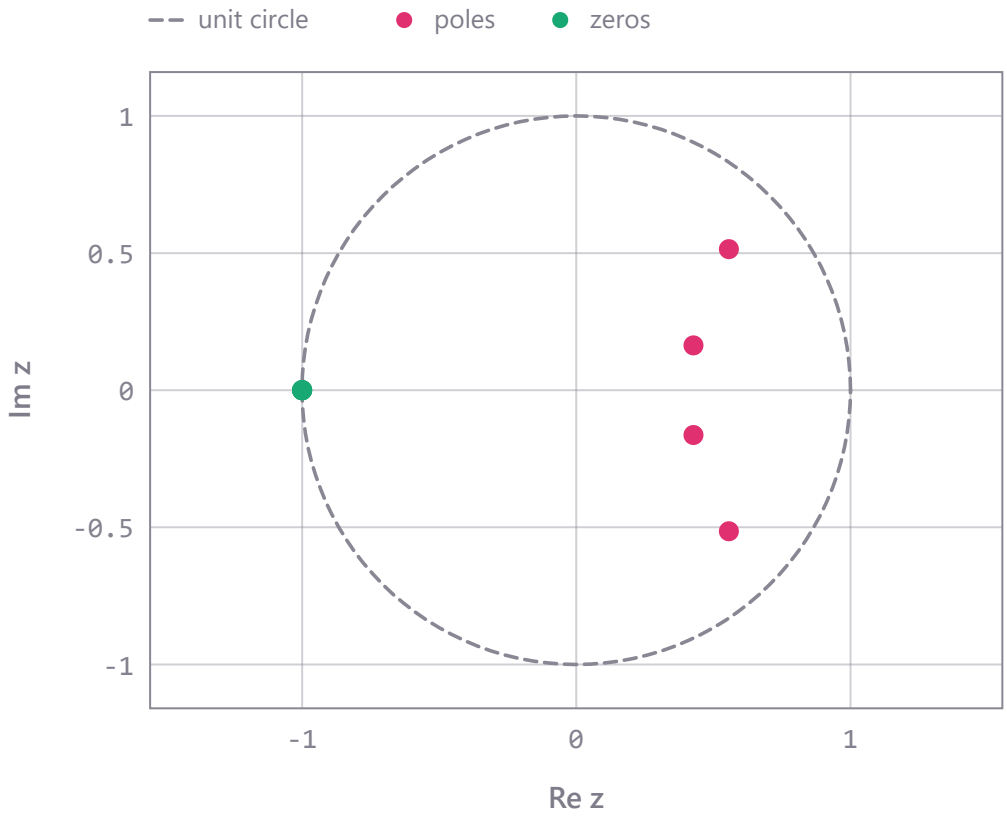
The zeros and poles of H(z) = sum b_k z^-k / sum a_k z^{-k} (up to 4th order), the largest pole radius and whether the causal filter is stable.

$$H(z) = \frac{\sum_k b_k z^{-k}}{\sum_k a_k z^{-k}} = \frac{b_0 \prod (1 - z_i z^{-1})}{a_0 \prod (1 - p_i z^{-1})}$$

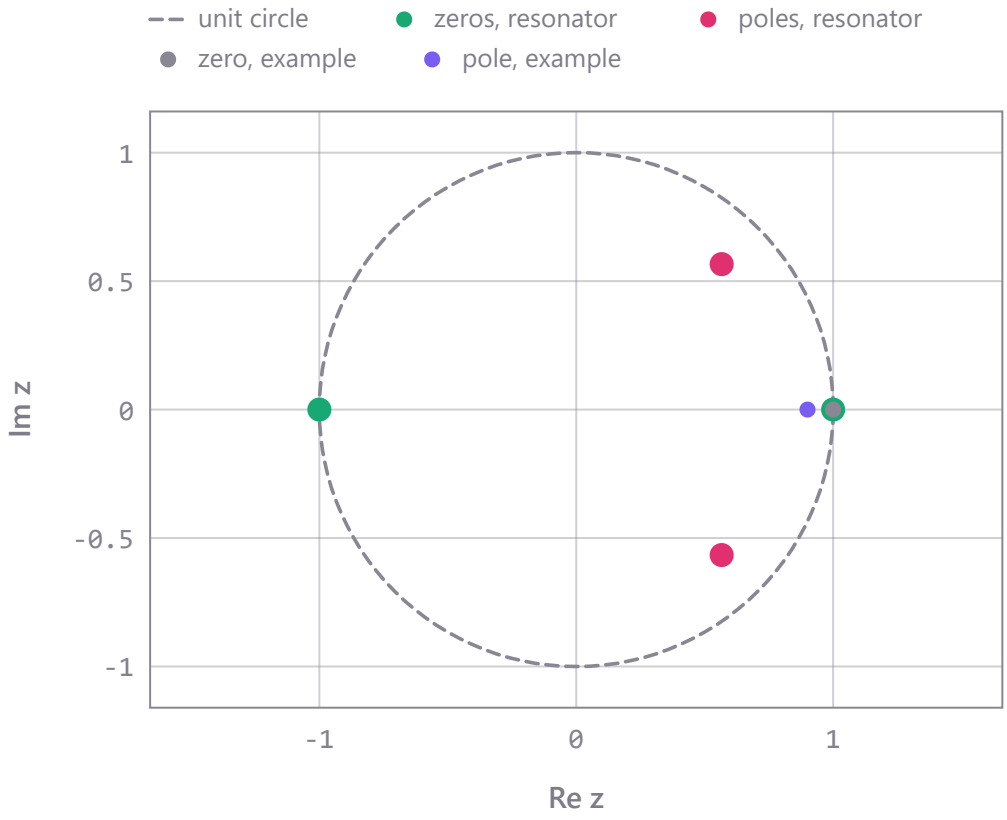
input	meaning	condition
`b`	numerator coefficients b (up to 5)	
`a`	denominator coefficients a (up to 5, a[0] != 0)	

output	meaning
`max_pole_radius`	largest pole radius
`zeros`	zeros (real ascending, then complex)
`poles`	poles (real ascending, then complex)
`stability`	stable if every pole lies inside the unit circle

- **Method:** polynomial roots (closed form up to degree 2; Durand-Kerner iteration with Newton polishing above)
- **Assumes:** a[0] != 0; orders up to 4; both polynomials in powers of z^{-1} (padded to equal length)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Poles and zeros returned by the package for the order-4 Butterworth low-pass filter (fc = 1000 Hz, fs = 8000 Hz) designed by it: 4 poles, largest radius 0.757669 (inside the unit circle: stable), and 4 zeros clustered at z = −1 (each within 0.000351186 of it).



Poles and zeros returned by the package for a two-pole resonator b = [1, 0, -1], a = [1, -1.131, 0.64] (an input chosen for the drawing): poles at radius 0.8 (stable), zeros at z = ±1; and for the worked example b = [1, -1], a = [1, -0.9]: pole 0.9, zero 1 (stable).

```
signal_processing_ai999x.solve('poles_zeros', b='1, -1', a='1, -0.9')
# -> max_pole_radius = 0.9, zeros = [1.0], poles = [0.9], stability = stable   (SOLVED)
```

```
signal_processing_ai999x.solve('poles_zeros', b=[1, 0, -1], a=[1, 0, 0.81])
# -> max_pole_radius = 0.9, zeros = [-1.0, 1.0], poles = [0.9j, -0.9j], stability = stable
(SOLVED)
```

```
signal_processing_ai999x.solve('poles_zeros', b=[1], a=[1, -1.5, 0.5])
# -> max_pole_radius = 1, zeros = [0.0, 0.0], poles = [0.5, 1.0], stability = unstable   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the poles and zeros of the filter with b = [1, -1] and a = [1, -0.9].	SOLVED	max_pole_radius = 0.9, zeros = [1.0], poles = [0.9], stability = stable
Find the zeros and poles of the filter with b = [1, 0, -1] and a = [1, 0, 0.81].	SOLVED	max_pole_radius = 0.9, zeros = [-1.0, 1.0], poles = [0.9j, -0.9j], stability = stable
Find the poles and zeros of the filter with b = [1, -1] and a = [0, 1].	INVALID_INPUT	Poles, zeros and stability of a digital filter: a[0] must not be 0.

Bilinear transform of a first-order low-pass filter — `bilinear_rc_lowpass`

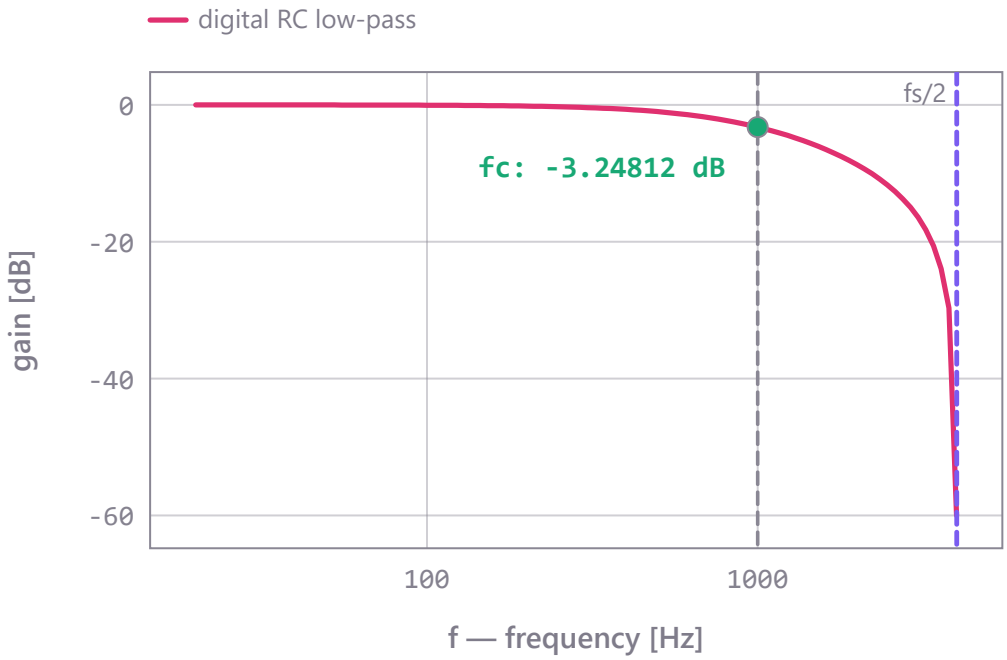
The digital filter H(z) = $(b_0 + b_1\, z^{-1})/(1 + a_1$ z^-1) obtained from H(s) = wc/(s + wc) by the bilinear transform $s = 2$ fs $(1 - z^{-1})/(1 + z^{-1})$, without pre-warping.

$$H(z) = \frac{\omega_c(1 + z^{-1})}{(\omega_c + 2f_s) + (\omega_c - 2f_s)z^{-1}}$$

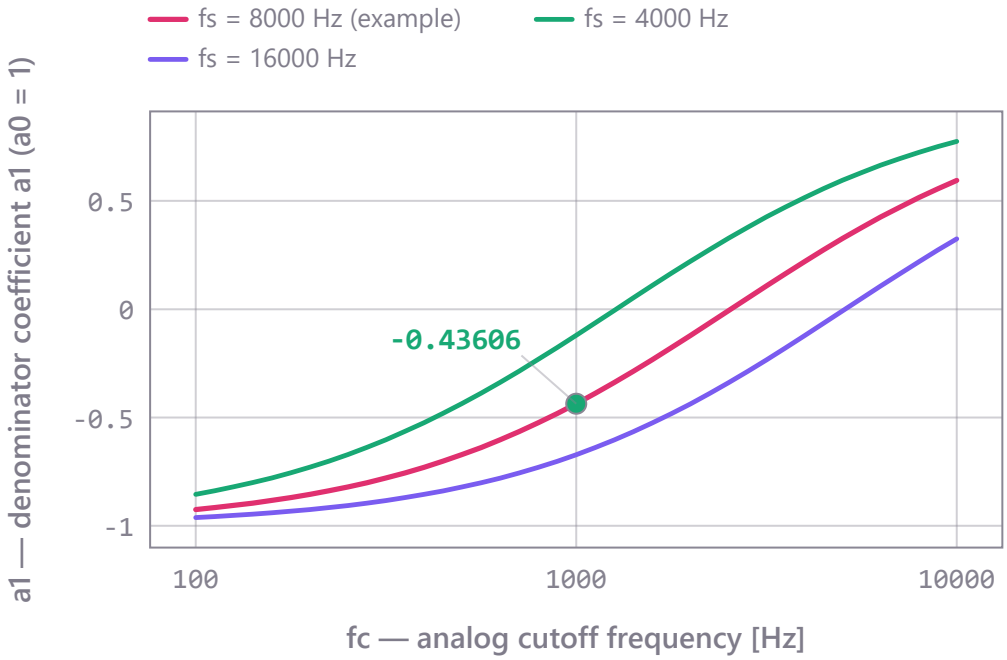
input	meaning	condition
`fc`	analog cutoff frequency	
`fs`	sampling rate	

output	meaning
`b0`	numerator coefficient b0
`b1`	numerator coefficient b1
`a1`	denominator coefficient a1 $(a_0 = 1)$

- **Method:** substitute $s = 2$ fs $(1 - z^{-1})/(1 + z^{-1})$ and normalize $a_0 = 1$
- **Assumes:** $fc > 0$; $fs > 0$; no frequency pre-warping
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The gain of the first-order digital low-pass obtained from an RC filter by the bilinear transform for fc = 1000 Hz at fs = 8000 Hz: the package returns b0 = 0.28197, b1 = 0.28197, a1 = -0.43606, and the response $H(z) = (b0 + b1\ z^{-1})/(1 + a1\ z^{-1})$ is 100 frequency-response runs from fs/400 to fs/2 (log axis). Marked: -3.24812 dB at fc; the gain falls to zero at fs/2.



a1 (denominator coefficient a1 (a0 = 1)) against fc (analog cutoff frequency) from 100 Hz to 10000 Hz on a logarithmic axis. One curve for each value of fs (sampling rate): 8000 Hz, 4000 Hz, 16000 Hz. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, fc = 1000 Hz, a1 = -0.43606.

```
signal_processing_ai999x.solve('bilinear_rc_lowpass', fc=1000.0, fs=8000.0)
# -> b0 = 0.28197, b1 = 0.28197, a1 = -0.43606   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the bilinear transform to a first-order low-pass filter with a cutoff frequency of 1 kHz at a sampling rate of 8 kHz.	SOLVED	b0 = 0.28197, b1 = 0.28197, a1 = -0.43606

question	outcome	result
Apply the bilinear transform to a first-order low-pass filter with a cutoff frequency of 1 kHz.	NEEDS_INPUT	bilinear_rc_lowpass needs fs (sampling rate). Example: solve('bilinear_rc_lowpass', fc='1000 Hz', fs='8000 Hz').

Main-lobe width of a spectral window — `window_mainlobe_width`

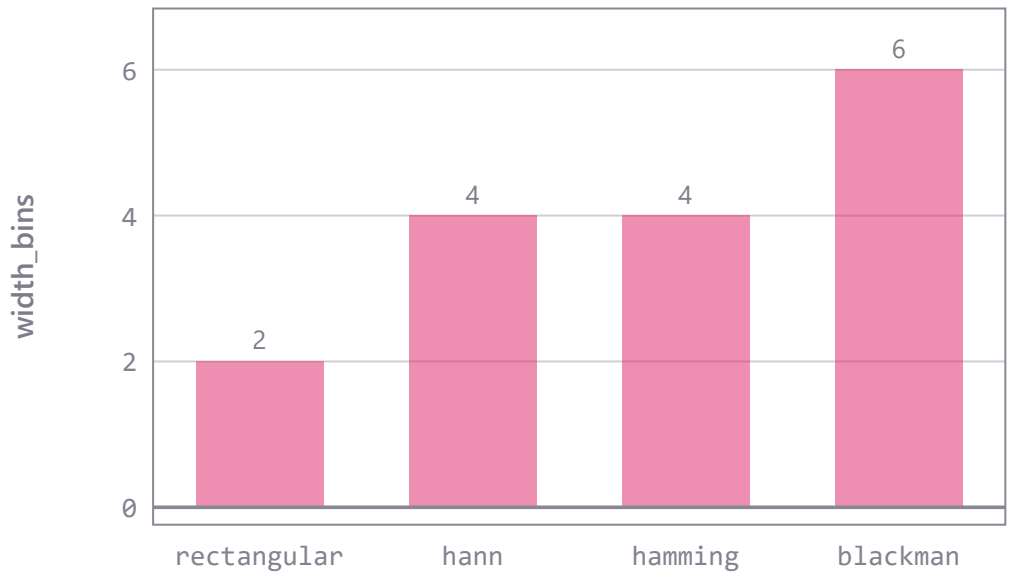
The null-to-null main-lobe width of the rectangular, Hann, Hamming or Blackman window (DFT-even form), in bins and, with fs, in hertz.

$$\Delta f_{\mathrm{ML}} = k\,\frac{f_s}{N},\ k = 2\ (\mathrm{rect}),\ 4\ (\mathrm{Hann, Hamming}),\ 6\ (\mathrm{Blackman})$$

input	meaning	condition
`window`	window type	
`N`	window length	
`fs`	sampling rate (for the width in Hz)	

output	meaning
`width_bins`	null-to-null main-lobe width in DFT bins
`first_null_bins`	position of the first null in bins
`width_hz`	main-lobe width in Hz (fs given)

- **Method:** tabulated widths 2, 4, 4, 6 bins (the first zero of each window's transform), times fs/N
- **Assumes:** $N \geq 8$; DFT-even (periodic) windows for spectral analysis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



width_bins (null-to-null main-lobe width in DFT bins) for each option of window (window type), one run of signal-processing-ai999x per option: rectangular: 2, hann: 4, hamming: 4, blackman: 6. The worked example uses hann. Other inputs as in the worked example: N = 256, fs = 8000.0 Hz.

```
signal_processing_ai999x.solve('window_mainlobe_width', window='hann', N=256, fs=8000.0)
# -> width_bins = 4, first_null_bins = 2, width_hz = 125 Hz   (SOLVED)
```

```
signal_processing_ai999x.solve('window_mainlobe_width', window='rectangular', N=64)
# -> width_bins = 2, first_null_bins = 1   (SOLVED)
```

```
signal_processing_ai999x.solve('window_mainlobe_width', window='blackman', N=100, fs=1000.0)
# -> width_bins = 6, first_null_bins = 3, width_hz = 60 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the main-lobe width of a 256-point Hann window at a sampling rate of 8 kHz?	SOLVED	width_bins = 4, first_null_bins = 2, width_hz = 125 Hz
What is the main-lobe width of a 64-point rectangular window?	SOLVED	width_bins = 2, first_null_bins = 1
What is the main-lobe width of a 4-point Hamming window?	INVALID_INPUT	N: must be at least 8, got 4.
What is the main-lobe width of a 64-point window?	NEEDS_INPUT	window_mainlobe_width needs window (window type). Example: solve('window_mainlobe_width', window='hann', N=256, fs='8000

Coefficients of a window function — `window_coefficients`

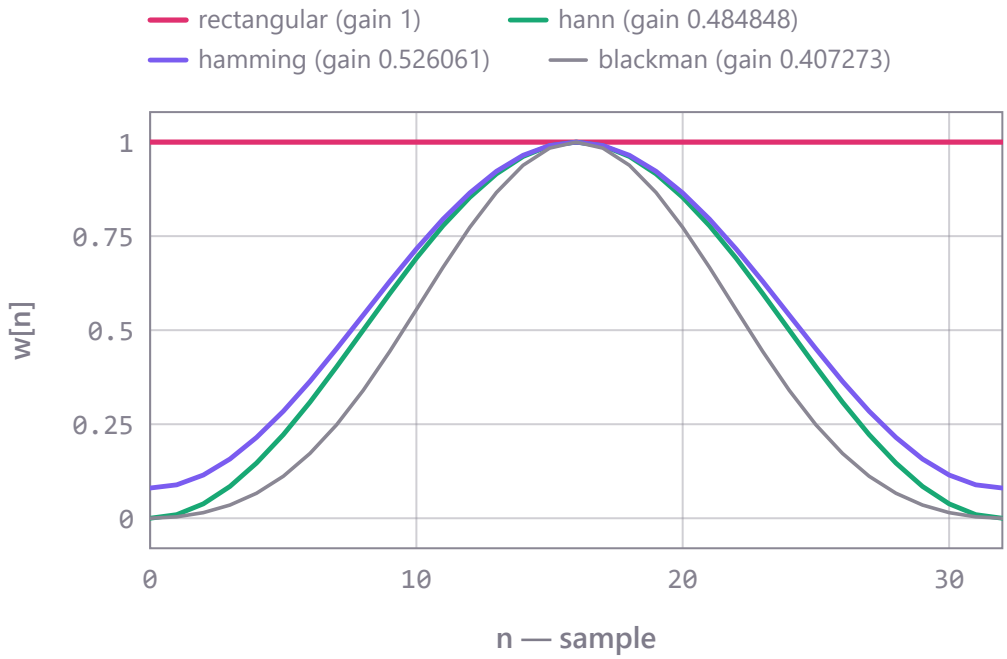
The samples w[$0..N-1$] of a rectangular, Hann, Hamming or Blackman window (symmetric for filter design, periodic for spectral analysis) and its coherent gain.

$$w[n] = \sum_k (-1)^k a_k \cos\frac{2\pi k n}{M}$$

input	meaning	condition
`window`	window type	
`N`	window length	
`form`	symmetric (default) or periodic	

output	meaning
`w`	window samples w[$0..N-1$]
`coherent_gain`	coherent gain: mean of w

- **Method:** generalized cosine sums: w[n] = sum_k $(-1)^k$ a_k cos(2 pi k n / M), $M = N-1$ (symmetric) or N (periodic)
- **Assumes:** $N \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The four window shapes of length N = 33 (symmetric form) returned by the package, each a run of it; the legend gives each window's coherent gain (rectangular 1, hann 0.484848, hamming 0.526061, blackman 0.407273). The worked example is the 4-point periodic hann window.

```
signal_processing_ai999x.solve('window_coefficients', window='hann', N=4, form='periodic')
# -> w = [0.0, 0.49999999999999994, 1.0, 0.5000000000000001], coherent_gain = 0.5   (SOLVED)
```

```
signal_processing_ai999x.solve('window_coefficients', window='hann', N=5)
# -> w = [0.0, 0.49999999999999994, 1.0, 0.5000000000000001, 0.0], coherent_gain = 0.4   (SOLVED)
```

```
signal_processing_ai999x.solve('window_coefficients', window='hamming', N=3)
# -> w = [0.08000000000000002, 1.0, 0.08000000000000002], coherent_gain = 0.386667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Give the coefficients of a 4-point periodic Hann window.	SOLVED	w = [0.0, 0.49999999999999994, 1.0, 0.5000000000000001], coherent_gain = 0.5
List the coefficients of a 3-point Hamming window.	SOLVED	w = [0.08000000000000002, 1.0, 0.08000000000000002], coherent_gain = 0.386667
List the coefficients of a Hamming window.	NEEDS_INPUT	window_coefficients needs N (window length). Example: solve('window_coefficients', window='hann', N=4, form='periodic').

Harmonic amplitude of a square, triangle or sawtooth wave — `fourier_series_harmonic`

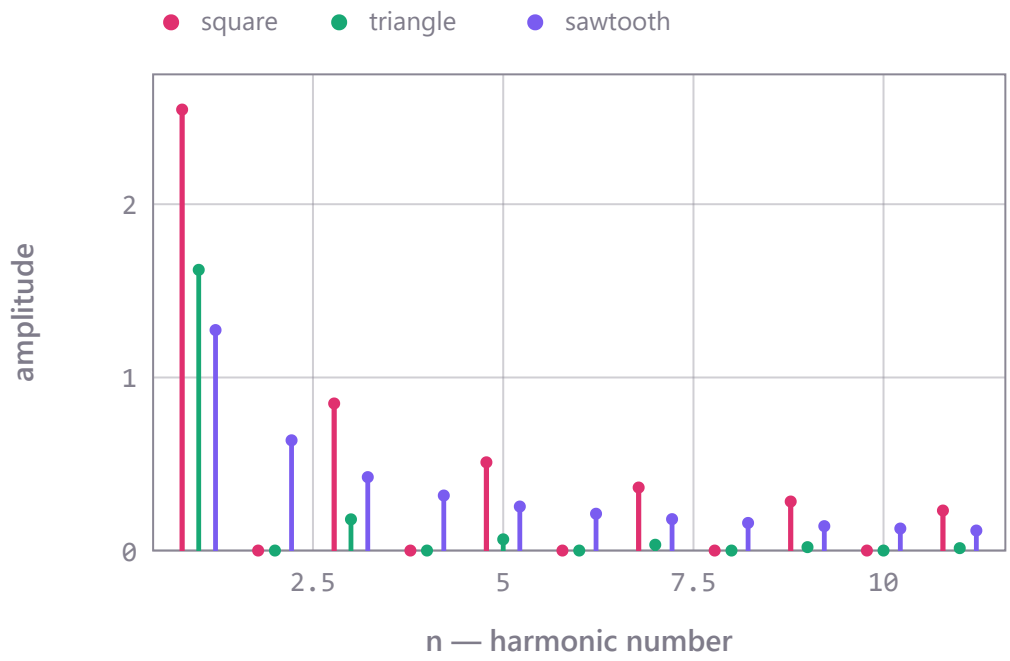
The amplitude of the n-th harmonic in the Fourier series of a zero-mean square, triangle or sawtooth wave of peak amplitude A, relative to the fundamental, and its RMS value.

$$c_n^{\mathrm{sq}} = \frac{4A}{n\pi},\ c_n^{\mathrm{tri}} = \frac{8A}{\pi^2 n^2}\ (n \text{ odd}),\quad c_n^{\mathrm{saw}} = \frac{2A}{n\pi}$$

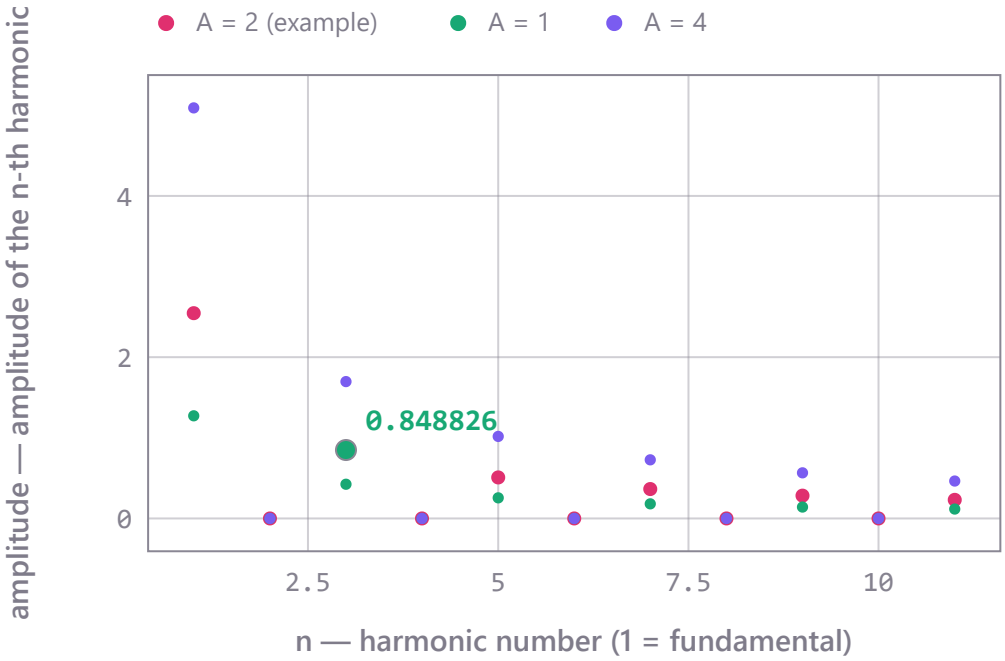
input	meaning	condition
`waveform`	waveform	
`A`	peak amplitude of the wave	
`n`	harmonic number (1 = fundamental)	

output	meaning
`amplitude`	amplitude of the n-th harmonic
`relative`	amplitude relative to the fundamental
`rms`	RMS value of the n-th harmonic

- **Method:** Fourier coefficients: square 4A/(n pi) (odd n), triangle 8A/(pi^2 n^2) (odd n), sawtooth 2A/(n pi)
- **Assumes:** zero-mean waveform with peak A; 50 % duty square wave; symmetric triangle; linear sawtooth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Fourier-series harmonic amplitudes of three waves of peak amplitude A = 2 (harmonics 1 … 11, each a run of the package; the stems of each harmonic are side by side): the square wave has odd harmonics falling as 1/n, the triangle odd harmonics as $1/n^2$, the sawtooth every harmonic as 1/n. square: fundamental 2.54648, 3rd 0.848826; triangle: fundamental 1.62114, 3rd 0.180127; sawtooth: fundamental 1.27324, 3rd 0.424413.



amplitude (amplitude of the n-th harmonic) against n (harmonic number (1 = fundamental)) from 1 to 11. One curve for each value of A (peak amplitude of the wave): 2, 1, 4. Other inputs as in the worked example: waveform = square. 11 points per curve computed by signal-processing-ai999x; marked: the worked example, n = 3, amplitude = 0.848826.

```
signal_processing_ai999x.solve('fourier_series_harmonic', waveform='square', A=2, n=3)
# -> amplitude = 0.848826, relative = 0.333333, rms = 0.600211   (SOLVED)
```

```
signal_processing_ai999x.solve('fourier_series_harmonic', waveform='square', A=1, n=2)
# -> amplitude = 0, relative = 0, rms = 0   (SOLVED)
```

```
signal_processing_ai999x.solve('fourier_series_harmonic', waveform='triangle', A=1, n=5)
# -> amplitude = 0.0324228, relative = 0.04, rms = 0.0229264   (SOLVED)
```

```
signal_processing_ai999x.solve('fourier_series_harmonic', waveform='sawtooth', A=1, n=4)
# -> amplitude = 0.159155, relative = 0.25, rms = 0.11254   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the amplitude of the 3rd harmonic of a square wave with amplitude 2?	SOLVED	amplitude = 0.848826, relative = 0.333333, rms = 0.600211
Find the 5th harmonic amplitude of a triangle wave with peak amplitude 1.	SOLVED	amplitude = 0.0324228, relative = 0.04, rms = 0.0229264
What is the amplitude of the 2nd harmonic of a sawtooth wave with amplitude 1?	SOLVED	amplitude = 0.31831, relative = 0.5, rms = 0.225079
What is the amplitude of the 3rd harmonic of a square wave?	NEEDS_INPUT	fourier_series_harmonic needs A (peak amplitude of the wave). Example: solve('fourier_series_harmonic', waveform='square

Fourier transform of a rectangular pulse — `rect_pulse_spectrum`

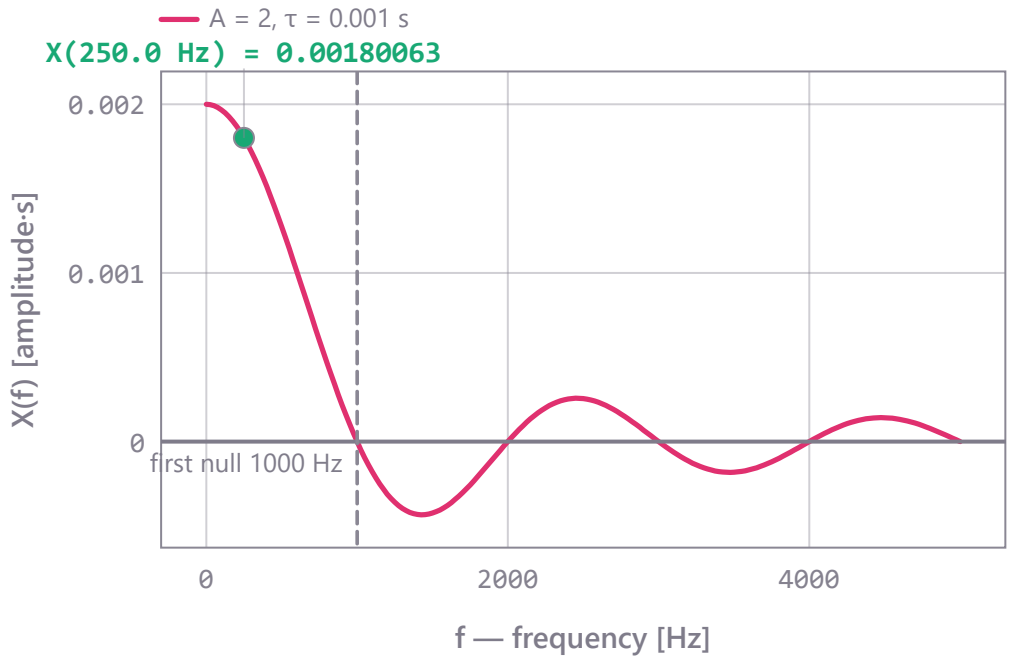
X(f) = A tau sinc(f tau) of a rectangular pulse of amplitude A and width tau centred at $t = 0$, its magnitude, and the first spectral null $1/\tau$.

$$X(f) = A\tau\,\mathrm{sinc}(f\tau) = A\tau\frac{\sin(\pi f\tau)}{\pi f\tau}$$

input	meaning	condition
`A`	pulse amplitude	
`tau`	pulse width	
`f`	frequency	

output	meaning
`X`	X(f) (real, in amplitude-seconds)
`magnitude`	|X(f)|
`first_null`	first zero of the spectrum, $1/\tau$

- **Method:** X(f) = A tau sin(pi f tau)/(pi f tau)
- **Assumes:** $A \neq 0$; $\tau > 0$; the pulse centred at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Fourier transform of a rectangular pulse of amplitude A = 2 and width τ = 0.001 s, X(f) = Aτ·sinc(fτ) (even in f), from 0 to 5 times the returned first null 1000 Hz (201 runs of the package): a main lobe of height Aτ and side lobes of alternating sign. Marked: the worked example f = 250.0 Hz.

```
signal_processing_ai999x.solve('rect_pulse_spectrum', A=2, tau=0.001, f=250.0)
# -> X = 0.00180063, magnitude = 0.00180063, first_null = 1000 Hz   (SOLVED)
```

```
signal_processing_ai999x.solve('rect_pulse_spectrum', A=1, tau=2e-06, f=0.0)
# -> X = 2e-06, magnitude = 2e-06, first_null = 500000 Hz   (SOLVED)
```

```
signal_processing_ai999x.solve('rect_pulse_spectrum', A=1, tau=0.001, f=1000.0)
# -> X = 3.89817e-20, magnitude = 3.89817e-20, first_null = 1000 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the Fourier transform of a rectangular pulse of amplitude 2 and width 1 ms at 250 Hz?	SOLVED	X = 0.00180063, magnitude = 0.00180063, first_null = 1000 Hz
Find the spectrum of a rectangular pulse of amplitude 1 and width 1 ms at 1500 Hz.	SOLVED	X = -0.000212207, magnitude = 0.000212207, first_null = 1000 Hz
Find the spectrum of a rectangular pulse of amplitude 1 at 1500 Hz.	NEEDS_INPUT	rect_pulse_spectrum needs tau (pulse width). Example: solve('rect_pulse_spectrum', A=2, tau='0.001 s', f='250 Hz').

Fourier transform of a decaying exponential — `exponential_pulse_spectrum`

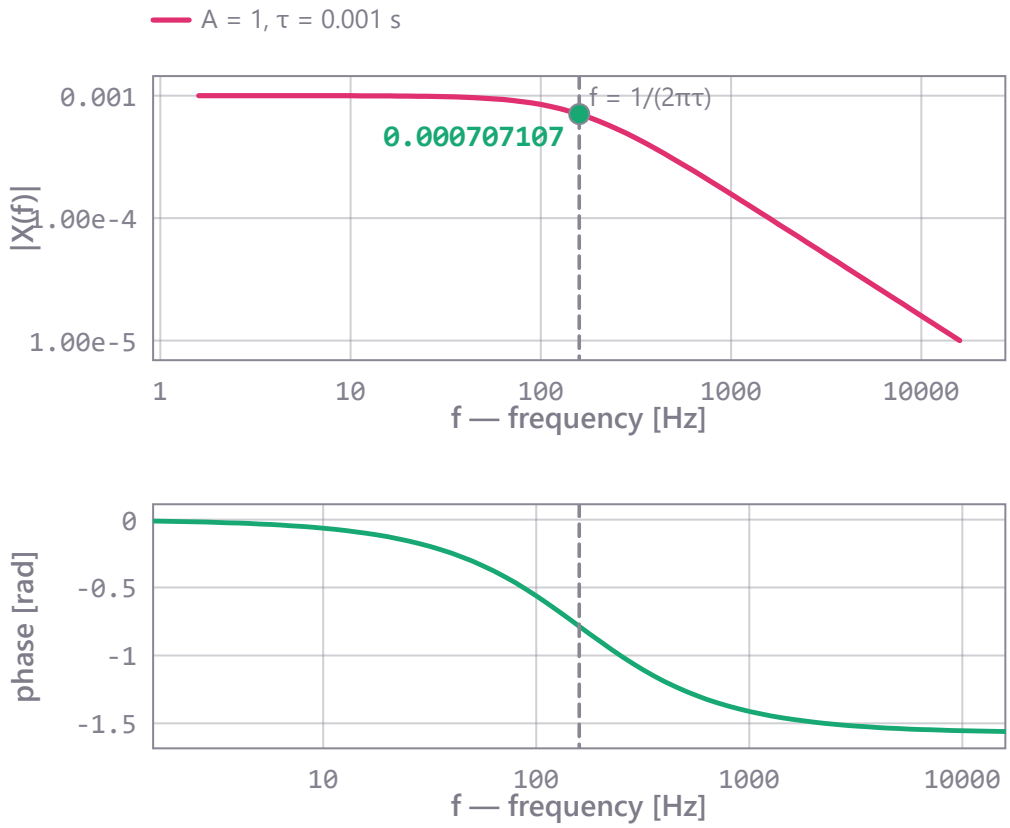
X(f) = A tau / (1 + j 2 pi f tau) of x(t) = A exp(-t/tau) for $t \geq 0$: magnitude, phase, real and imaginary parts.

$$X(f) = \frac{A\tau}{1 + j2\pi f\tau}$$

input	meaning	condition
`A`	initial amplitude	
`tau`	time constant	
`f`	frequency	

output	meaning
`magnitude`	|X(f)|
`phase`	phase of X(f) in radians
`re`	real part
`im`	imaginary part

- **Method:** X(f) = integral_0^inf A e^{-t/tau} e^{-j 2 pi f t} dt in closed form
- **Assumes:** $A > 0$; $\tau > 0$; x(t) = 0 for $t < 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Magnitude and phase of the spectrum of A·e^(−t/τ) for t ≥ 0 with A = 1, τ = 0.001 s, from 1/100 to 100 times f = 1/(2πτ) = 159.155 Hz (81 runs of the package, log axes): flat at Aτ below that frequency, falling 20 dB per decade above, the phase going from 0 to −π/2. Marked: the worked example f = 159.155 Hz, |X| = 0.000707107 (phase -0.785398 rad).

```
signal_processing_ai999x.solve('exponential_pulse_spectrum', A=1, tau=0.001, f=159.15494309189535)
# -> magnitude = 0.000707107, phase = -0.785398 rad, re = 0.0005, im = -0.0005   (SOLVED)
```

```
signal_processing_ai999x.solve('exponential_pulse_spectrum', A=2, tau=0.5, f=0.0)
# -> magnitude = 1, phase = 0 rad, re = 1, im = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the Fourier transform of a decaying exponential with amplitude 2 and time constant 0.5 s at 0 Hz?	SOLVED	magnitude = 1, phase = 0 rad, re = 1, im = 0
What is the Fourier transform of a decaying exponential with amplitude 1 and time constant 1 ms at 159.15 Hz?	SOLVED	magnitude = 0.000707118, phase = -0.785383 rad, re = 0.000500016, im = -0.0005
What is the Fourier transform of a decaying exponential with amplitude 1 at 100 Hz?	NEEDS_INPUT	exponential_pulse_spectrum needs tau (time constant). Example: solve('exponential_pulse_spectrum', A=1, tau='0.001 s', f

Total harmonic distortion — `thd`

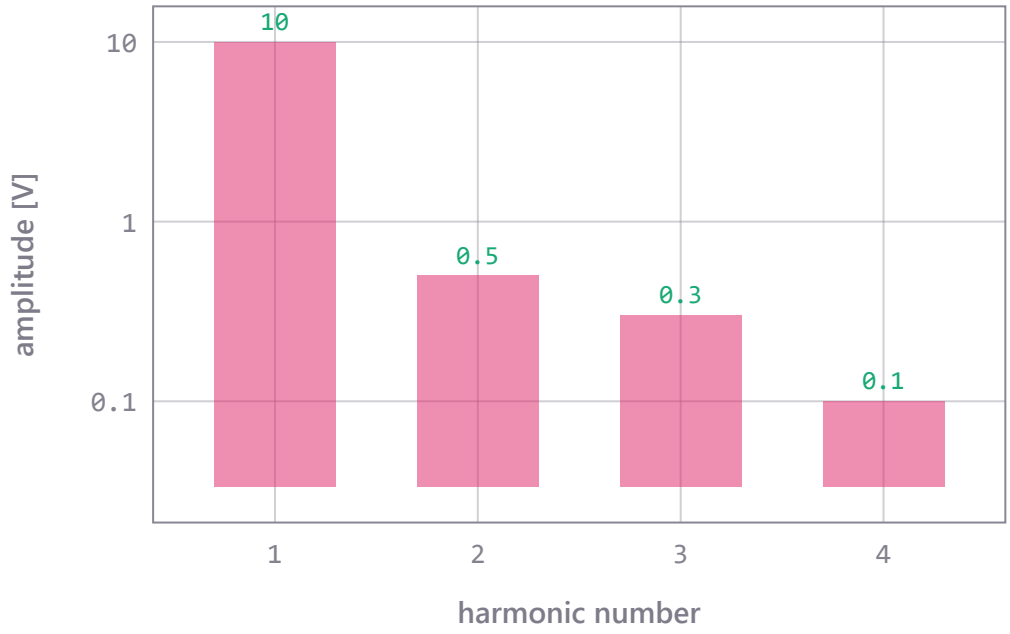
THD = sqrt(V2^2 + V3^2 + ...)/V1 from the amplitudes of the fundamental and its harmonics, as a ratio, in percent and in dB.

$$\mathrm{THD} = \frac{\sqrt{\sum_{n\geq 2} V_n^2}}{V_1}$$

input	meaning	condition
`V`	amplitudes [V1, V2, V3, ...] of the fundamental and harmonics	

output	meaning
`thd`	THD as a ratio
`percent`	THD in percent
`db`	THD in dB

- **Method:** root-sum-square of the harmonics over the fundamental
- **Assumes:** $V_1 > 0$; amplitudes >= 0 (RMS or peak, all alike)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Harmonic amplitudes V1 … V4 = [10, 0.5, 0.3, 0.1] (input; the fundamental first, log axis). The package returns THD = √(V2² + …)/V1 = 0.0591608 (5.91608 %, -24.5593 dB).

```
signal_processing_ai999x.solve('thd', V='10, 0.5, 0.3, 0.1')
# -> thd = 0.0591608, percent = 5.91608, db = -24.5593   (SOLVED)
```

```
signal_processing_ai999x.solve('thd', V=[1, 0.3, 0.4])
# -> thd = 0.5, percent = 50, db = -6.0206   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the total harmonic distortion of a signal with harmonic amplitudes [10, 0.5, 0.3, 0.1]?	SOLVED	thd = 0.0591608, percent = 5.91608, db = -24.5593

question	outcome	result
Find the THD for the amplitudes [1, 0.3, 0.4].	SOLVED	thd = 0.5, percent = 50, db = -6.0206
Find the THD for the amplitudes [0, 0.3, 0.4].	INVALID_INPUT	Total harmonic distortion: the amplitudes must be >= 0 and the fundamental > 0.

Analytic signal, Hilbert transform and envelope — `analytic_envelope`

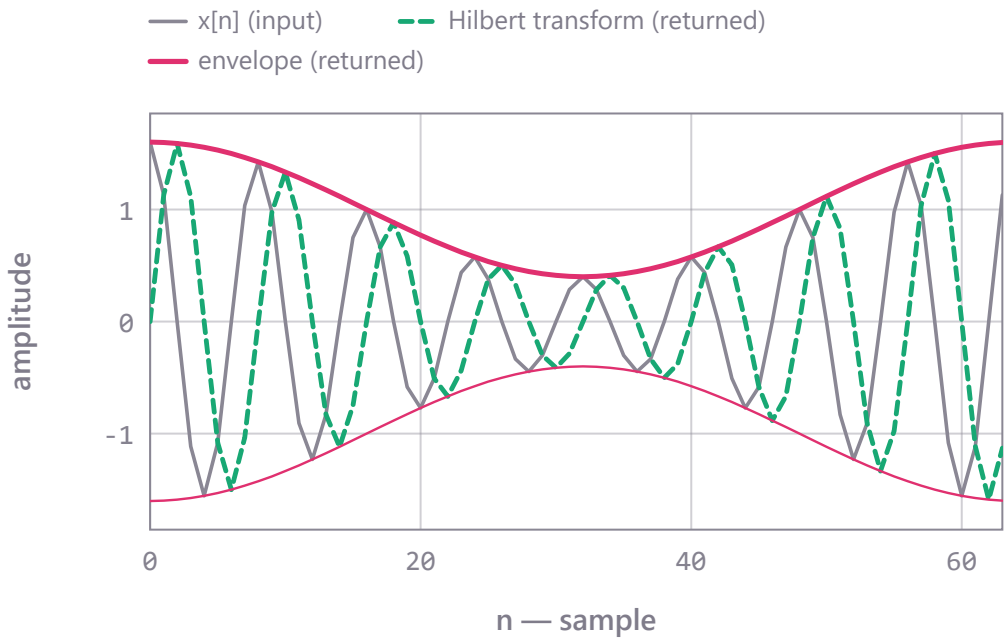
The discrete Hilbert transform of a real sequence and its envelope |x + j H{x}|, computed through the DFT (negative frequencies removed, positive ones doubled).

$$x_a[n] = \mathrm{IDFT}\{X[k]\,h[k]\}, \quad \text{envelope} = |x_a[n]|$$

input	meaning	condition
`x`	real samples	

output	meaning
`envelope`	envelope |x_a[n]|
`hilbert`	Hilbert transform (imaginary part of the analytic signal)

- **Method:** FFT, multiply by h[k] = 1, 2, ..., 2, (1), 0, ..., 0, inverse FFT
- **Assumes:** the sequence treated as one period (DFT-based transform)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



An amplitude-modulated tone chosen for the drawing, x[n] = (1 + 0.6 cos 2πn/64)·cos 2π·8n/64, n = 0 … 63, its Hilbert transform and the envelope |x + j·H{x}| returned by the package (drawn at ± the envelope): the envelope traces the slow modulation, between 0.4 and 1.6. The worked example x = [1, 0, -1, 0] has a constant envelope of 1.

```
signal_processing_ai999x.solve('analytic_envelope', x='1, 0, -1, 0')
# -> envelope = [1.0, 1.0, 1.0, 1.0], hilbert = [0.0, 1.0, 0.0, -1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Hilbert transform and envelope of x = [1, 0, -1, 0].	SOLVED	envelope = [1.0, 1.0, 1.0, 1.0], hilbert = [0.0, 1.0, 0.0, -1.0]
Find the envelope of the analytic signal of x = [2, 0, -2, 0].	SOLVED	envelope = [2.0, 2.0, 2.0, 2.0], hilbert = [0.0, 2.0, 0.0, -2.0]

Periodogram (power spectral density estimate) — `periodogram`

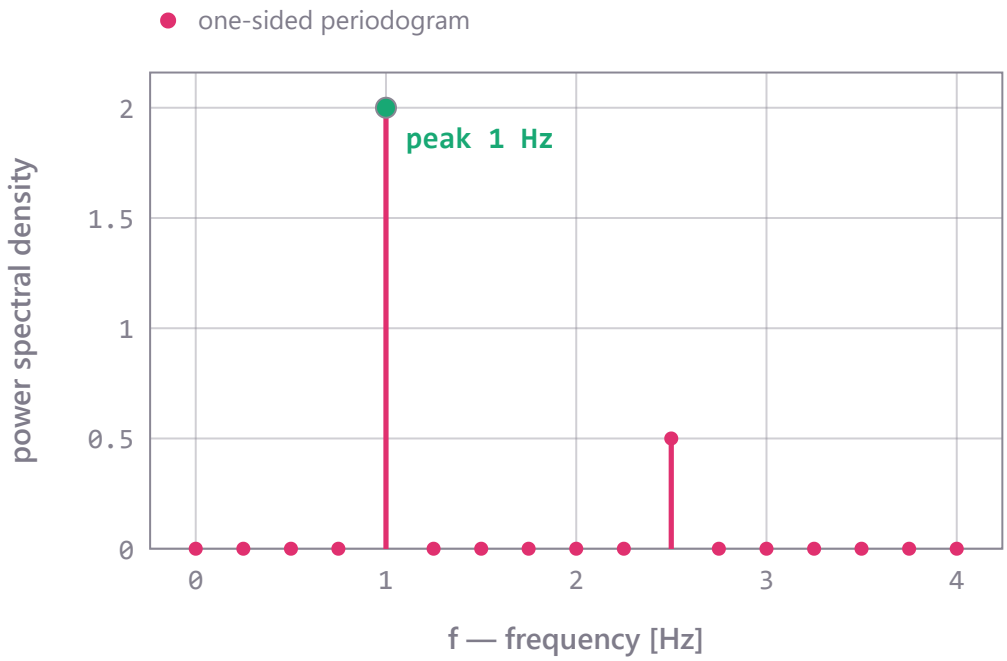
The one-sided periodogram PSD estimate (rectangular window, mean removed) of a sequence sampled at fs, its frequency grid and the frequency of its peak.

$$P[k] = \frac{2\,|X[k]|^2}{f_s N} \quad (0 < k < N/2)$$

input	meaning	condition
`x`	samples	
`fs`	sampling rate	

output	meaning
`peak_frequency`	frequency of the largest PSD value
`freqs`	frequencies k fs / N, $k = 0 \,..\, N/2$
`psd`	one-sided power spectral density (power per hertz)

- **Method:** FFT of the mean-removed samples; |X[k]|^2 / (fs N), doubled except at DC and Nyquist
- **Assumes:** mean removed first; rectangular window; one-sided density scaling
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The periodogram returned by the package for 32 samples at fs = 8 Hz of the chosen signal x[n] = sin(2π·1·n/fs) + 0.5 sin(2π·2.5·n/fs): 17 frequencies 0 … fs/2 in steps of fs/32 = 0.25 Hz; the two tones appear at 1 Hz and 2.5 Hz, the second a quarter of the first's power. Marked: the returned peak frequency 1 Hz.

```
signal_processing_ai999x.solve('periodogram', x='1, 0, -1, 0, 1, 0, -1, 0', fs=8.0)
# -> peak_frequency = 2 Hz, freqs = [0.0, 1.0, 2.0, 3.0, 4.0], psd = [0.0, 0.0, 0.5, 0.0, 0.0]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the periodogram of x = [1, 0, -1, 0, 1, 0, -1, 0] at a sampling rate of 8 Hz.	SOLVED	peak_frequency = 2 Hz, freqs = [0.0, 1.0, 2.0, 3.0, 4.0], psd = [0.0, 0.0, 0.5, 0.0, 0.0]
Compute the periodogram of x = [1, 0, -1, 0, 1, 0, -1, 0].	NEEDS_INPUT	periodogram needs fs (sampling rate). Example: solve('periodogram', x='1, 0, -1, 0, 1, 0, -1, 0', fs='8 Hz').

Power in an AM (DSB full-carrier) signal — `am_power`

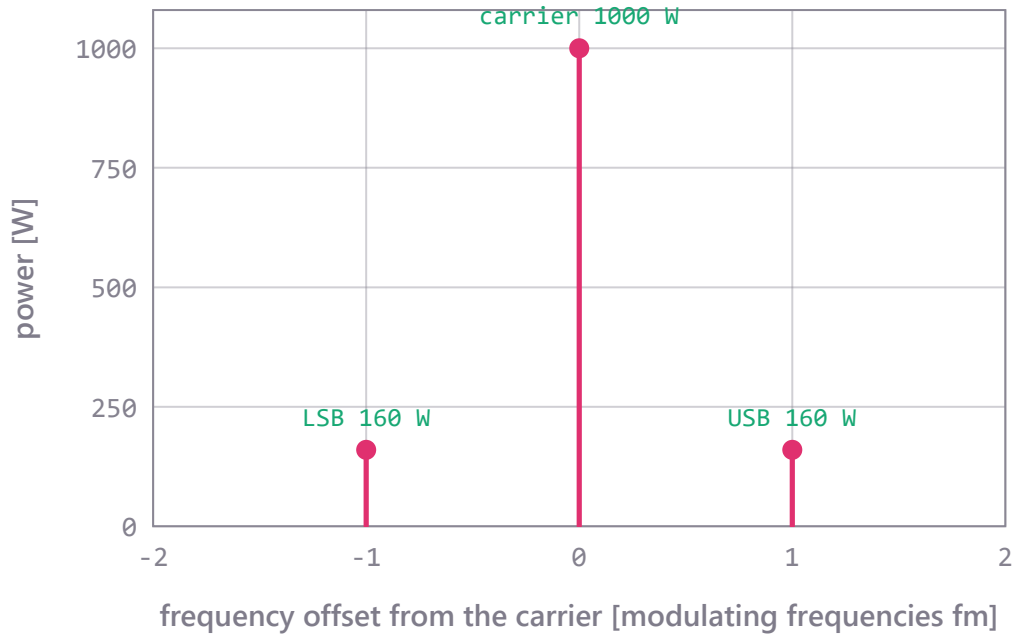
Total power Pc $(1 + m^2/2)$, sideband power, power per sideband and transmission efficiency of an AM signal with carrier power Pc and modulation index m (sinusoidal message).

$$P_t = P_c \left(1 + \frac{m^2}{2}\right)$$

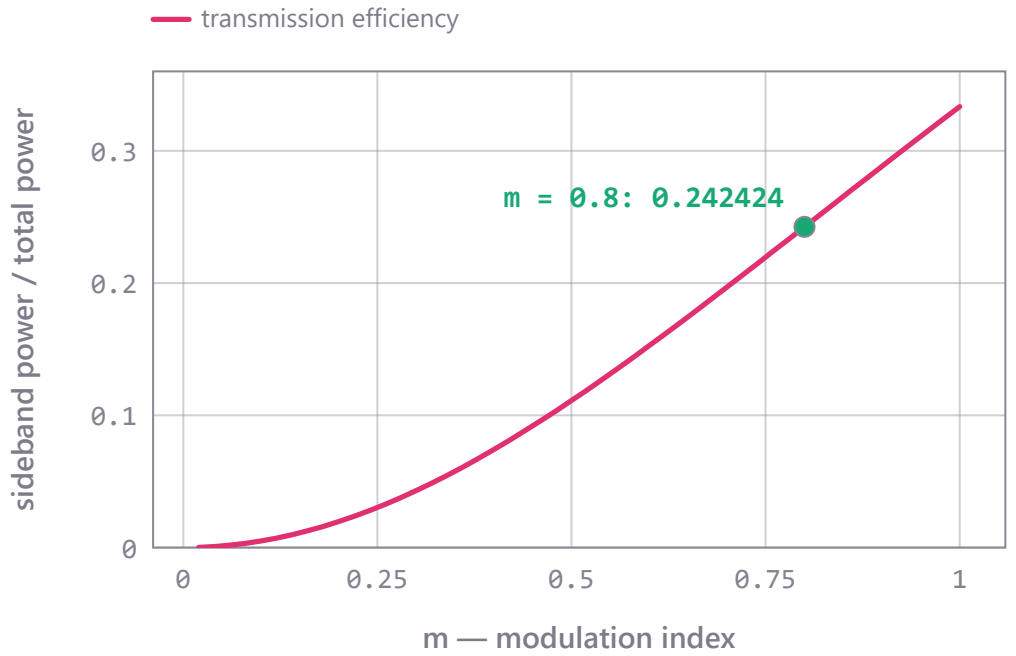
input	meaning	condition
`Pc`	carrier power	
`m`	modulation index $(0 < m \leq 1)$	

output	meaning
`Pt`	total transmitted power
`Psb`	power in both sidebands
`Pusb`	power in one sideband
`efficiency`	fraction of the power in the sidebands

- **Method:** $Pt = Pc\,(1 + m^2/2)$; each sideband Pc $m^2/4$
- **Assumes:** $0 < m \leq 1$; a single sinusoidal message
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The line spectrum of an AM signal with carrier power Pc = 1000 W and modulation index m = 0.8 (inputs): the carrier and the upper and lower sidebands at ±fm, each sideband carrying the returned Pusb = 160 W (the two are equal). Returned: total power Pt = 1320 W, both sidebands Psb = 320 W, efficiency 0.242424.



The share of the transmitted AM power that is in the sidebands, for modulation index m from 0 to 1 (51 runs of the package, Pc = 1000.0 W): at most one third, at m = 1. Marked: m = 0.8, efficiency 0.242424.

```
signal_processing_ai999x.solve('am_power', Pc=1000.0, m=0.8)
# -> Pt = 1320 W, Psb = 320 W, Pusb = 160 W, efficiency = 0.242424   (SOLVED)
```

```
signal_processing_ai999x.solve('am_power', Pc=50.0, m=1.0)
# -> Pt = 75 W, Psb = 25 W, Pusb = 12.5 W, efficiency = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
An AM transmitter has a carrier power of 1000 W and a modulation index of 0.8. What is the total transmitted power?	SOLVED	Pt = 1320 W, Psb = 320 W, Pusb = 160 W, efficiency = 0.242424
An AM signal has a carrier power of 50 W and 100% modulation. Find the total power and the efficiency.	SOLVED	Pt = 75 W, Psb = 25 W, Pusb = 12.5 W, efficiency = 0.333333
An AM transmitter has a carrier power of 1000 W and a modulation index of 1.2. What is the total transmitted power?	INVALID_INPUT	Power in an AM (DSB full-carrier) signal: the modulation index must be at most 1 (no overmodulation).
An AM transmitter has a carrier power of 1000 W. What is the total transmitted power?	NEEDS_INPUT	am_power needs m (modulation index (0 < m <= 1)). Example: solve('am_power', Pc='1000 W', m=0.8).

AM modulation index from the envelope — `am_modulation_index`

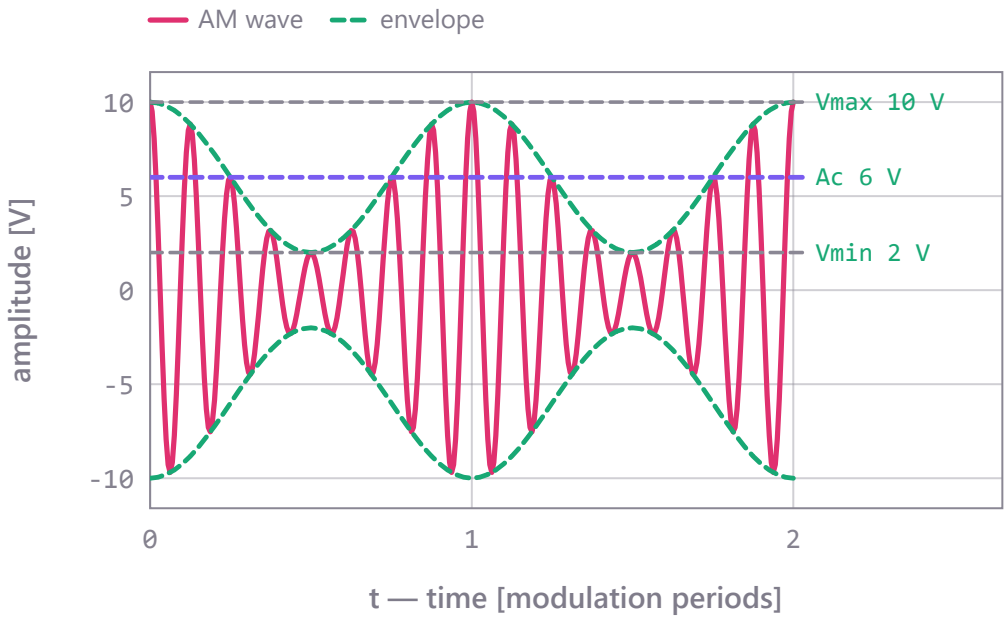
The modulation index m = (Vmax - Vmin)/(Vmax + Vmin) of an AM wave from the maximum and minimum of its envelope, and the carrier amplitude.

$$m = \frac{V_{\max} - V_{\min}}{V_{\max} + V_{\min}}$$

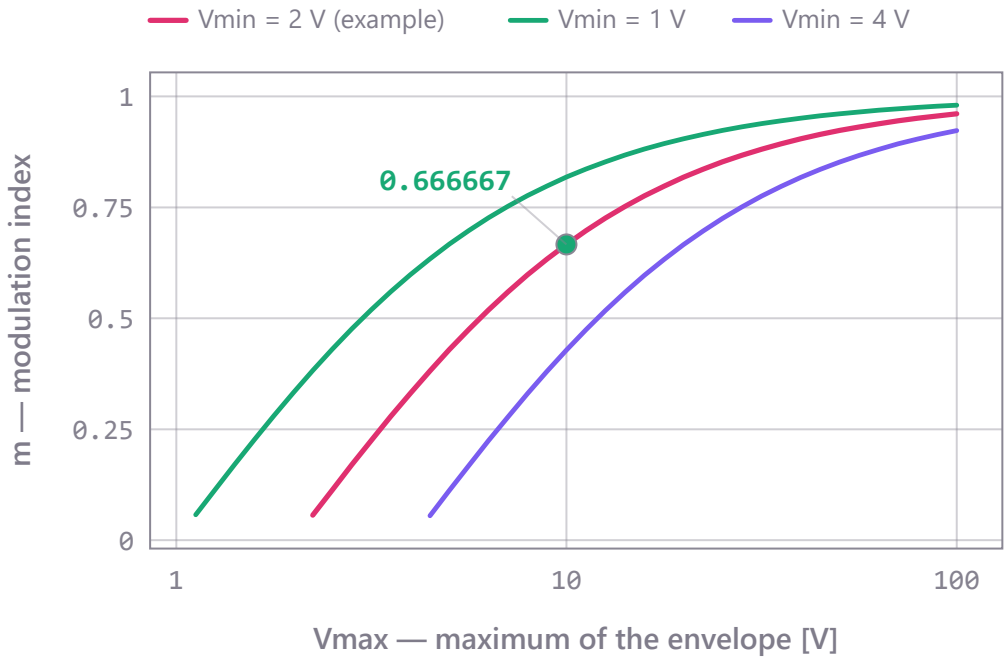
input	meaning	condition
`Vmax`	maximum of the envelope	
`Vmin`	minimum of the envelope	

output	meaning
`m`	modulation index
`Ac`	carrier amplitude (Vmax + Vmin)/2

- **Method:** m = (Vmax - Vmin)/(Vmax + Vmin)
- **Assumes:** 0 <= Vmin < Vmax; sinusoidal message
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



An AM wave with envelope maximum Vmax = 10 V and minimum Vmin = 2 V (inputs): the package returns the modulation index m = 0.666667 and carrier amplitude Ac = 6 V. The wave Ac(1 + m cos 2πt)·cos 2π·8t is drawn from those returned values (a carrier of 8 cycles per modulation period, chosen for the drawing); its envelope swings between Vmin and Vmax around Ac.



m (modulation index) against Vmax (maximum of the envelope) from 1 V to 100 V on a logarithmic axis. One curve for each value of Vmin (minimum of the envelope): 2 V, 1 V, 4 V. 34 points per curve computed by signal-processing-ai999x; marked: the worked example, Vmax = 10 V, m = 0.666667.

```
signal_processing_ai999x.solve('am_modulation_index', Vmax=10.0, Vmin=2.0)
# -> m = 0.666667, Ac = 6 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
An AM envelope has a maximum of 10 V and a minimum of 2 V. What is the modulation index?	SOLVED	m = 0.666667, Ac = 6 V

question	outcome	result
An AM envelope has a maximum of 2 V and a minimum of 10 V. What is the modulation index?	INVALID_INPUT	AM modulation index from the envelope: the maximum of the envelope must exceed its minimum.

FM spectrum: Carson bandwidth and Bessel sideband amplitudes — `fm_spectrum`

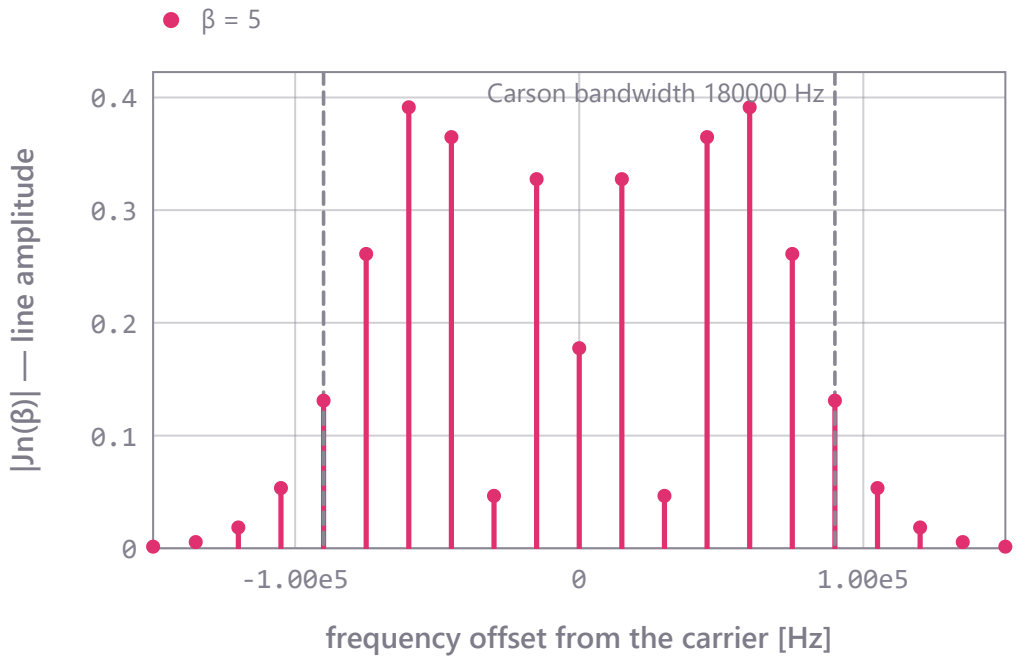
For sinusoidal FM with peak deviation df and modulating frequency fm: the index $\beta = df/fm$, Carson's bandwidth 2(df + fm), the carrier and n-th sideband amplitudes J0(beta), Jn(beta) (relative to the unmodulated carrier) and the power fraction inside Carson's bandwidth.

$$B_C = 2(\Delta f + f_m), \quad s(t) = A\sum_n J_n(\beta)\cos[2\pi(f_c + nf_m)t]$$

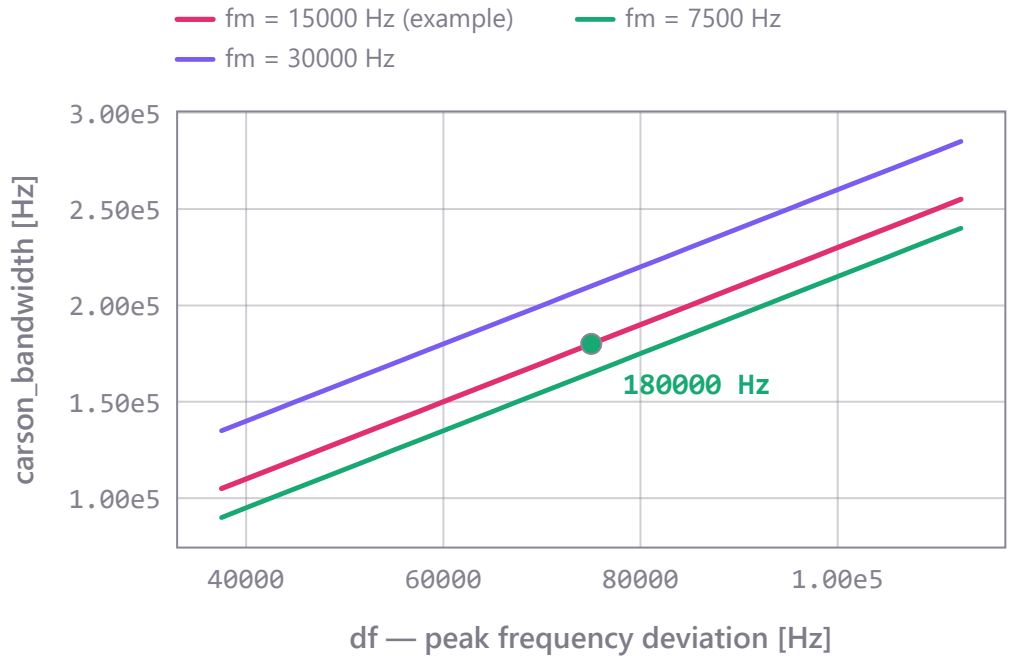
input	meaning	condition
`df`	peak frequency deviation	
`fm`	modulating (message) frequency	
`n`	sideband number (default 1)	

output	meaning
`carson_bandwidth`	Carson's rule bandwidth 2(df + fm)
`beta`	modulation index df/fm
`carrier`	carrier amplitude J0(beta)
`Jn`	amplitude of each n-th sideband Jn(beta)
`pair_power`	power fraction in the pair of n-th sidebands
`power_in_carson`	power fraction within Carson's bandwidth

- **Method:** Bessel functions of the first kind (30-digit evaluation); Carson's rule
- **Assumes:** sinusoidal message; $\beta \le 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The line spectrum of FM with peak deviation Δf = 75000 Hz and modulating frequency fm = 15000 Hz: modulation index β = 5 and line amplitudes |Jn(β)| at ±n·fm for n = 0 ... 10 (n = 0 is the carrier, -0.177597), each a run of the package. Dashed: the returned Carson bandwidth 180000 Hz centred on the carrier (±90000 Hz), which holds 0.993559 of the power.



carson_bandwidth (Carson's rule bandwidth 2(df + fm)) against df (peak frequency deviation) from 37500 Hz to 112500 Hz. One curve for each value of fm (modulating (message) frequency): 15000 Hz, 7500 Hz, 30000 Hz. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, df = 75000 Hz, carson_bandwidth = 180000 Hz.

```
signal_processing_ai999x.solve('fm_spectrum', df=75000.0, fm=15000.0)
# -> carson_bandwidth = 180000 Hz, beta = 5, carrier = -0.177597, Jn = -0.327579, pair_power =
0.214616, power_in_carson = 0.993559   (SOLVED)
```

```
signal_processing_ai999x.solve('fm_spectrum', df=2405.0, fm=1000.0, n=1)
# -> carson_bandwidth = 6810 Hz, beta = 2.405, carrier = -9.0558e-05, Jn = 0.51911, pair_power =
0.53895, power_in_carson = 0.99105   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
An FM signal has a peak frequency deviation of 75 kHz and a modulating frequency of 15 kHz. Find the Carson bandwidth.	SOLVED	carson_bandwidth = 180000 Hz, beta = 5, carrier = -0.177597, Jn = -0.327579, pair_power = 0.214616, power_in_carson = 0.993559
An FM signal has a frequency deviation of 5 kHz and a modulating frequency of 1 kHz. What is the amplitude of the 2nd sideband?	SOLVED	carson_bandwidth = 12000 Hz, beta = 5, carrier = -0.177597, Jn = 0.0465651, pair_power = 0.00433662, power_in_carson = 0.993559
An FM signal has a peak frequency deviation of 75 kHz. Find the Carson bandwidth.	NEEDS_INPUT	fm_spectrum needs fm (modulating (message) frequency). Example: solve('fm_spectrum', df='75000 Hz', fm='15000 Hz').

Shannon capacity of an AWGN channel — `shannon_capacity`

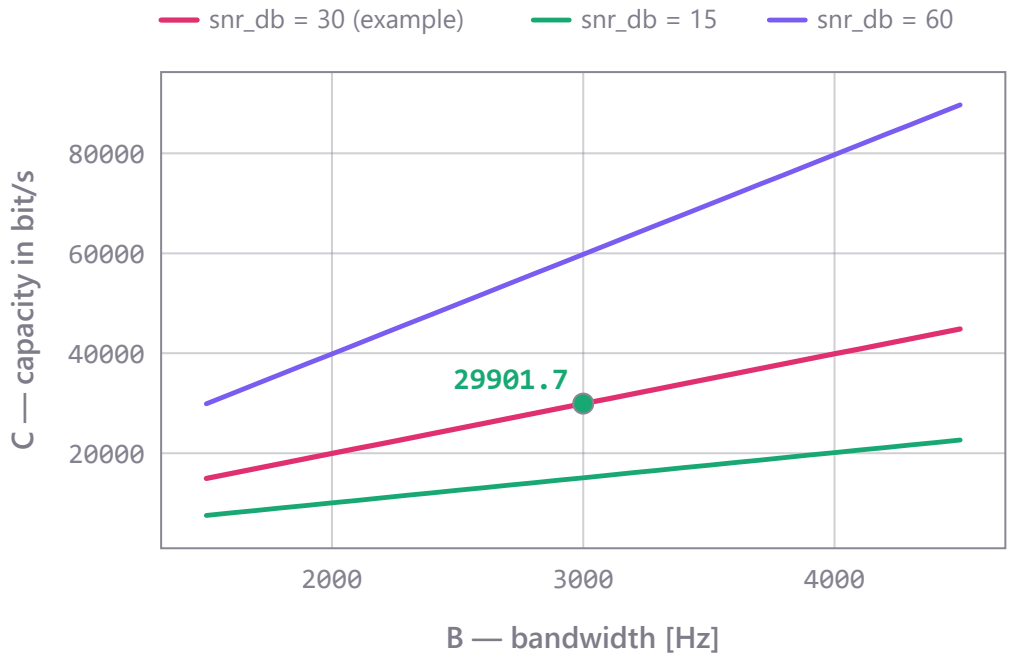
The channel capacity $C = B$ log2(1 + SNR) of a band-limited additive white Gaussian noise channel and the spectral efficiency C/B.

$$C = B \log_2(1 + \mathrm{SNR})$$

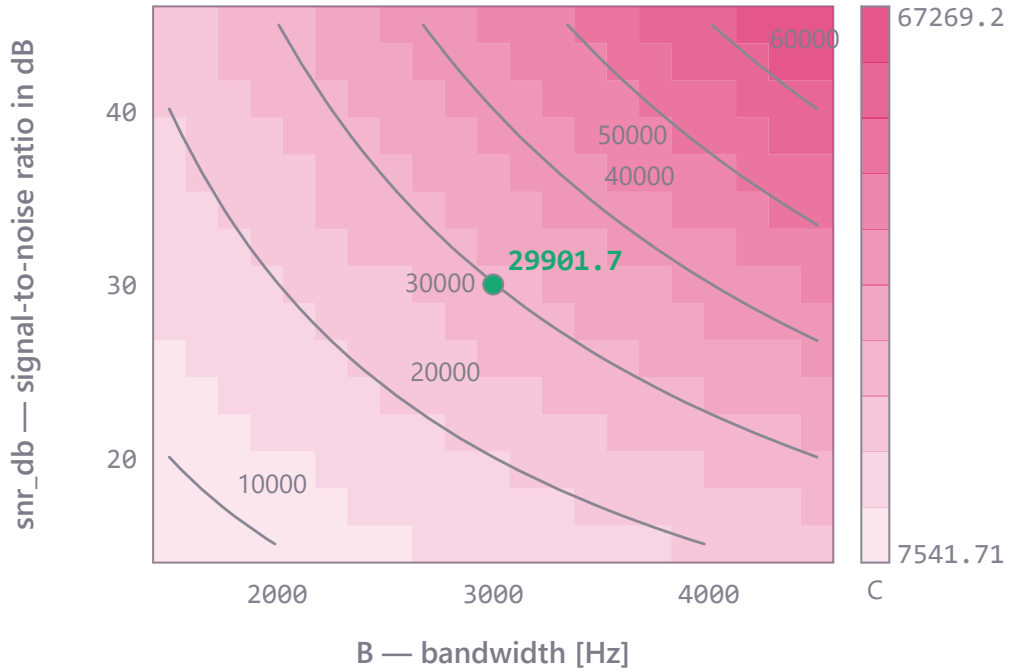
input	meaning	condition
`B`	bandwidth	
`snr_db`	signal-to-noise ratio in dB	

output	meaning
`C`	capacity in bit/s
`efficiency`	spectral efficiency in bit/s per Hz

- **Method:** Shannon-Hartley theorem
- **Assumes:** additive white Gaussian noise; SNR stated in dB
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (capacity in bit/s) against B (bandwidth) from 1500 Hz to 4500 Hz. One curve for each value of snr_db (signal-to-noise ratio in dB): 30, 15, 60. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, B = 3000 Hz, C = 29901.7.



C (capacity in bit/s) over B (bandwidth) from 1500 Hz to 4500 Hz and snr_db (signal-to-noise ratio in dB) from 15 to 45: each of the 21 × 15 cells is one run of signal-processing-ai999x, shaded by its value (light 7541.71 to dark 67269.2); iso-lines at C = 10000, 20000, 30000, 40000, 50000, 60000. Marked: the worked example, B = 3000 Hz, snr_db = 30, C = 29901.7.

```
signal_processing_ai999x.solve('shannon_capacity', B=3000.0, snr_db=30.0)
# -> C = 29901.7, efficiency = 9.96723   (SOLVED)
```

```
signal_processing_ai999x.solve('shannon_capacity', B=1000000.0, snr_db=0.0)
# -> C = 1e+06, efficiency = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the Shannon capacity of a 3 kHz channel with an SNR of 30 dB?	SOLVED	C = 29901.7, efficiency = 9.96723
Find the channel capacity for a bandwidth of 1 MHz and a signal-to-noise ratio of 0 dB.	SOLVED	C = 1e+06, efficiency = 1
What is the Shannon capacity of a 3 kHz channel with an SNR of 1000?	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: signal_processing_ai999x.template
What is the Shannon capacity of a 3 kHz channel?	NEEDS_INPUT	shannon_capacity needs snr_db (signal-to-noise ratio in dB). Example: solve('shannon_capacity', B='3000 Hz', snr_db=30).

Minimum SNR for a data rate (Shannon limit) — `shannon_min_snr`

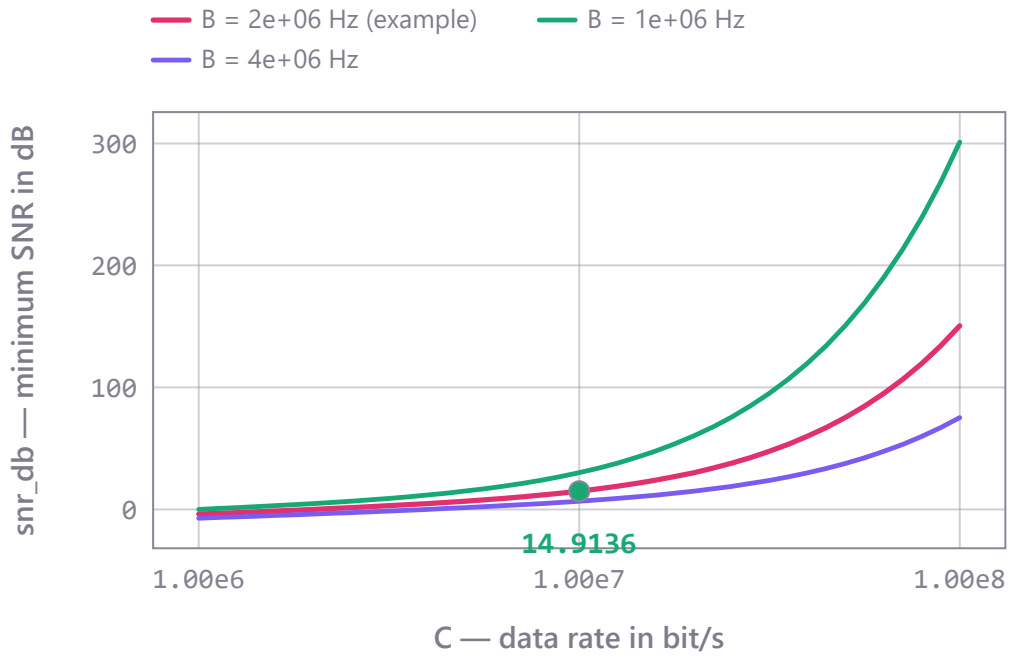
The smallest SNR = 2^(C/B) - 1 that lets an AWGN channel of bandwidth B carry C bit/s, and the corresponding minimum Eb/N_0.

$$\mathrm{SNR}_{\min} = 2^{C/B} - 1, \quad \left(\frac{E_b}{N_0}\right)_{\min} = \frac{2^{C/B} - 1}{C/B}$$

input	meaning	condition
`C`	data rate in bit/s	
`B`	bandwidth	

output	meaning
`snr_db`	minimum SNR in dB
`snr`	minimum SNR (linear)
`ebn0_min_db`	minimum Eb/N_0 in dB

- **Method:** invert $C = B$ log2(1 + SNR)
- **Assumes:** additive white Gaussian noise; $C/B \leq 200$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



snr_db (minimum SNR in dB) against C (data rate in bit/s) from 1e+06 to 1.0000e+08 on a logarithmic axis. One curve for each value of B (bandwidth): 2e+06 Hz, 1e+06 Hz, 4e+06 Hz. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, C = 1.0000e+07, snr_db = 14.9136.

```
signal_processing_ai999x.solve('shannon_min_snr', C=10000000.0, B=2000000.0)
# -> snr_db = 14.9136, snr = 31, ebn0_min_db = 7.92392   (SOLVED)
```

```
signal_processing_ai999x.solve('shannon_min_snr', C=1000000.0, B=1000000.0)
# -> snr_db = 0, snr = 1, ebn0_min_db = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What minimum SNR is needed to send 10 Mbit/s over a bandwidth of 2 MHz?	SOLVED	snr_db = 14.9136, snr = 31, ebn0_min_db = 7.92392
What minimum SNR is needed to send 10 Mbit/s?	NEEDS_INPUT	shannon_min_snr needs B (bandwidth). Example: solve('shannon_min_snr', C=1e+07, B='2e+06 Hz').

Bit error rate of BPSK (and coherent BFSK) in AWGN — `bpsk_ber`

The bit error probability of coherent BPSK, Q(sqrt(2 Eb/N0)), and of coherent orthogonal BFSK, Q(sqrt(Eb/N0)), in additive white Gaussian noise.

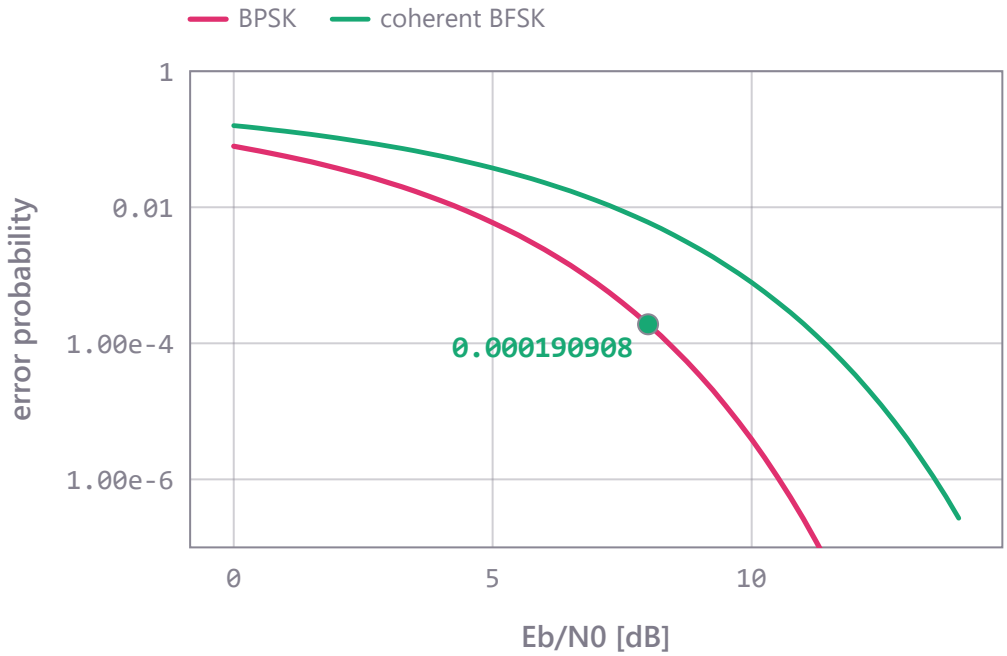
$$P_b = Q\left(\sqrt{2E_b/N_0}\right) = \tfrac{1}{2}\operatorname{erfc}\sqrt{E_b/N_0}$$

input	meaning	condition
`ebn0_db`	Eb/N_0 in dB	

output	meaning
`ber`	BPSK bit error rate

output	meaning
`ber_bfsk`	coherent BFSK bit error rate

- **Method:** $P_b = Q$(sqrt(2 Eb/N0)) = erfc(sqrt(Eb/N0))/2
- **Assumes:** coherent detection; AWGN channel; Eb/N_0 stated in dB
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bit error probability of coherent BPSK and of coherent binary FSK; FSK needs 3 dB more for the same error rate. Eb/N0 from 0 to 14 dB, one run of the package per 0.25 dB, logarithmic axis (values below 10^{-7} are off the axis). Marked: the worked example, Eb/N0 = 8.0 dB, ber = 0.000190908.

```
signal_processing_ai999x.solve('bpsk_ber', ebn0_db=8.0)
# -> ber = 0.000190908, ber_bfsk = 0.00600439   (SOLVED)
```

```
signal_processing_ai999x.solve('bpsk_ber', ebn0_db=0.0)
# -> ber = 0.0786496, ber_bfsk = 0.158655   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the bit error rate of BPSK at Eb/N0 = 8 dB?	SOLVED	ber = 0.000190908, ber_bfsk = 0.00600439
What is the bit error rate of BPSK?	SOLVED	answer = None None

Bit and symbol error rates of QPSK in AWGN — `qpsk_error_rates`

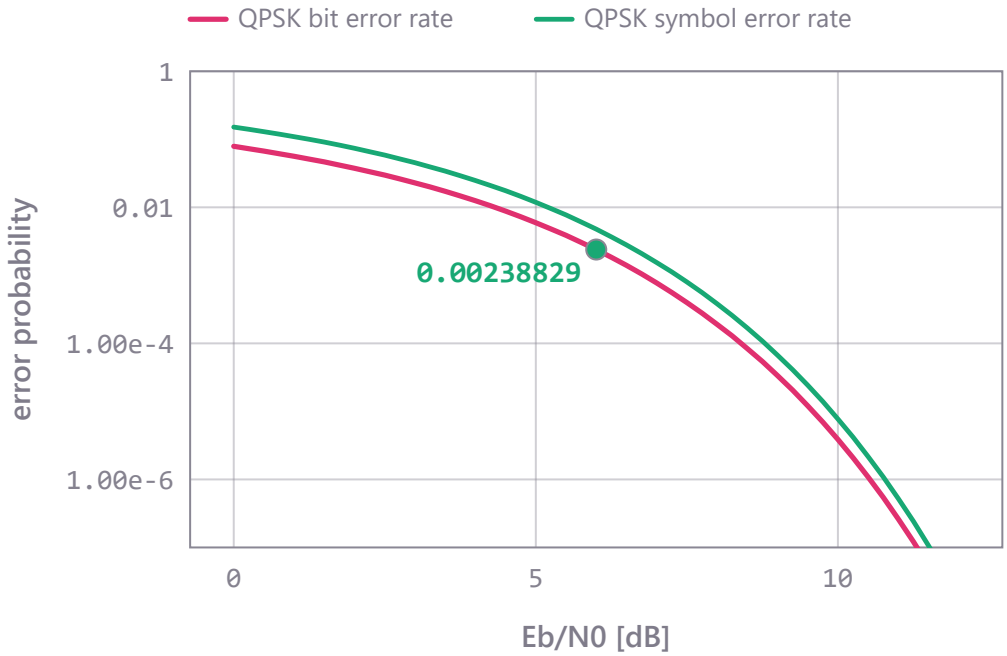
Gray-coded coherent QPSK: bit error rate Q(sqrt(2 Eb/N0)) and symbol error rate $2Q - Q^{2.}$

$$P_b = Q\left(\sqrt{2E_b/N_0}\right), \quad P_s = 2Q - Q^2$$

input	meaning	condition
`ebn0_db`	Eb/N_0 in dB	

output	meaning
`ber`	bit error rate (Gray coding)
`ser`	symbol error rate

- **Method:** independent BPSK on I and Q: $P_s = 1 - (1 - Q)^2$
- **Assumes:** coherent detection; Gray coding; AWGN channel
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bit and symbol error probabilities of Gray-coded QPSK. Eb/N0 from 0 to 14 dB, one run of the package per 0.25 dB, logarithmic axis (values below 10^{-7} are off the axis). Marked: the worked example, Eb/N0 = 6.0 dB, ber = 0.00238829.

```
signal_processing_ai999x.solve('qpsk_error_rates', ebn0_db=6.0)
# -> ber = 0.00238829, ser = 0.00477088   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What are the bit and symbol error rates of QPSK at Eb/N0 = 6 dB?	SOLVED	ber = 0.00238829, ser = 0.00477088

Bit error rates of DPSK and noncoherent BFSK — `noncoherent_ber`

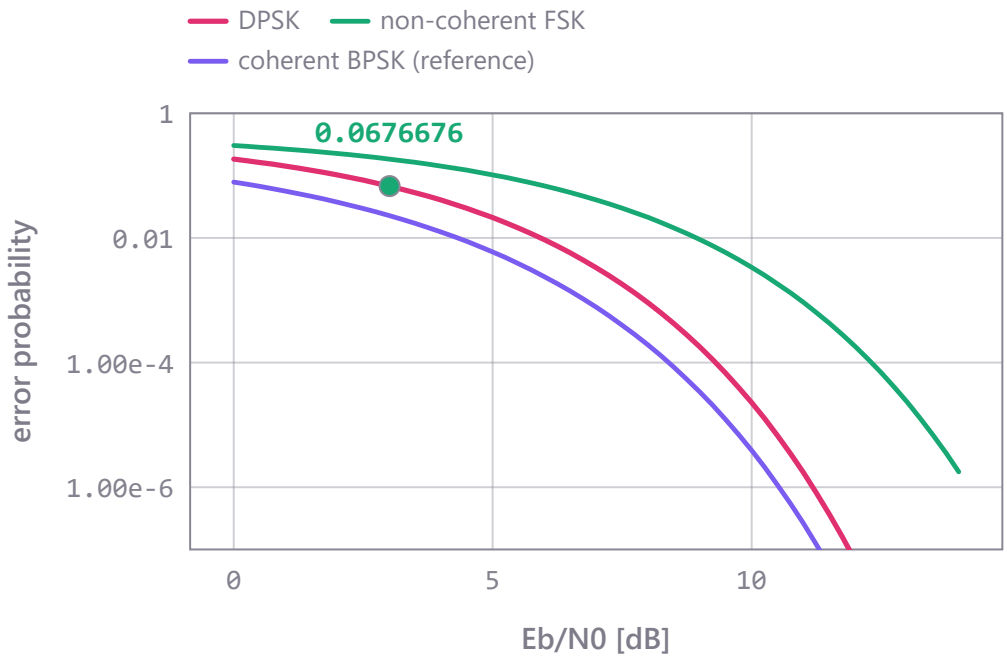
The bit error rates of differentially coherent BPSK (DPSK), exp(-Eb/N0)/2, and of noncoherent orthogonal BFSK, exp(-Eb/2N0)/2, in AWGN.

$$P_b^{\mathrm{DPSK}} = \tfrac{1}{2} e^{-E_b/N_0}, \quad P_b^{\mathrm{NCFSK}} = \tfrac{1}{2} e^{-E_b/2N_0}$$

input	meaning	condition
`ebn0_db`	Eb/N_0 in dB	

output	meaning
`ber_dpsk`	DPSK bit error rate
`ber_ncfsk`	noncoherent BFSK bit error rate

- **Method:** closed forms of noncoherent detection
- **Assumes:** AWGN channel; slow phase variation (DPSK); orthogonal tones (FSK)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bit error probability of DPSK and non-coherent binary FSK, with coherent BPSK for comparison. Eb/N0 from 0 to 14 dB, one run of the package per 0.25 dB, logarithmic axis (values below 10^{-7} are off the axis). Marked: the worked example, Eb/N0 = 3.010299956639812 dB, ber_dpsk = 0.0676676.

```
signal_processing_ai999x.solve('noncoherent_ber', ebn0_db=3.010299956639812)
# -> ber_dpsk = 0.0676676, ber_ncfsk = 0.18394   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the bit error rate of DPSK at Eb/N0 = 8 dB?	SOLVED	ber_dpsk = 0.000909404, ber_ncfsk = 0.0213237
Find the bit error rate of noncoherent FSK at Eb/N0 = 10 dB.	SOLVED	ber_dpsk = 2.27e-05, ber_ncfsk = 0.00336897

Symbol error rate of square M-QAM in AWGN — `mqam_symbol_error`

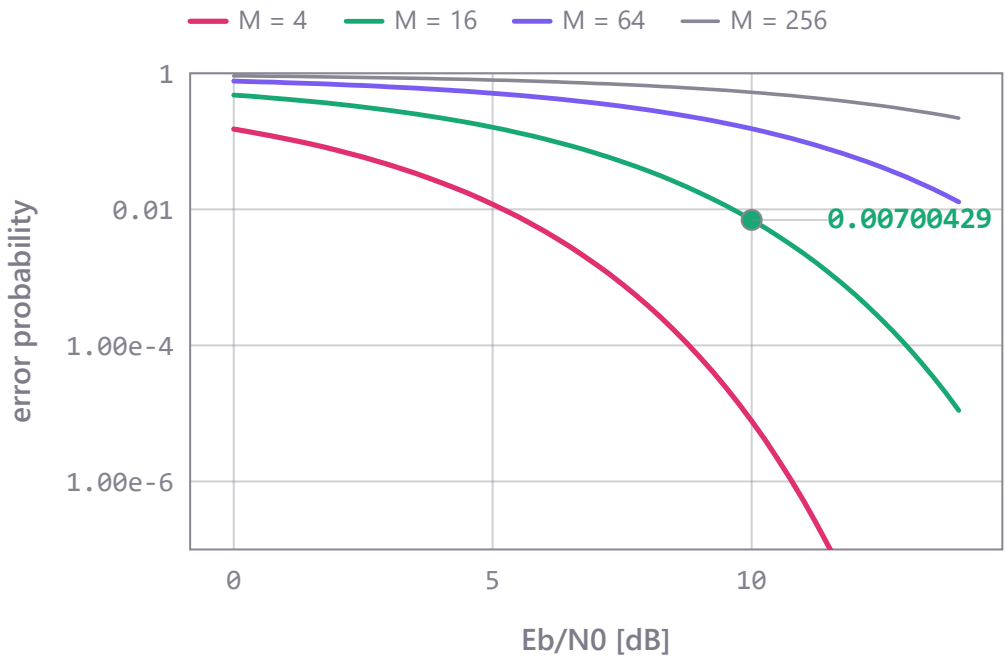
The exact symbol error rate of coherent square M-QAM (4 to 1024 points) at a given Eb/N_0, and the Gray-coded bit error rate approximation SER/log2 M.

$$P_s = 1 - \left[1 - 2\left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3E_s}{(M-1)N_0}}\right)\right]^2$$

input	meaning	condition
`M`	constellation size	
`ebn0_db`	Eb/N_0 in dB	

output	meaning
`ser`	symbol error rate
`ber_approx`	bit error rate approximation (Gray coding)

- **Method:** $P_s = 1 - (1 - P_L)^2$ with $P_L = 2(1 - 1/sqrt$(M)) Q(sqrt(3 Es / (($M - 1$) N0)))
- **Assumes:** square constellation; coherent detection; AWGN channel
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Symbol error probability of square M-QAM for M = 4, 16, 64 and 256. Eb/N0 from 0 to 14 dB, one run of the package per 0.25 dB, logarithmic axis (values below 10^{-7} are off the axis). Marked: the worked example, Eb/N0 = 10.0 dB, ser = 0.00700429.

```
signal_processing_ai999x.solve('mqam_symbol_error', M='16', ebn0_db=10.0)
# -> ser = 0.00700429, ber_approx = 0.00175107   (SOLVED)
```

```
signal_processing_ai999x.solve('mqam_symbol_error', M='4', ebn0_db=6.0)
# -> ser = 0.00477088, ber_approx = 0.00238544   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the symbol error rate of 16-QAM at Eb/N0 = 10 dB?	SOLVED	ser = 0.00700429, ber_approx = 0.00175107
What is the symbol error rate of 32-QAM at Eb/N0 = 10 dB?	INVALID_INPUT	M: one of 4, 16, 64, 256, 1024; got '32'.
What is the symbol error rate of 64-QAM?	NEEDS_INPUT	mqam_symbol_error needs ebn0_db (Eb/N0 in dB). Example: solve('mqam_symbol_error', M='16', ebn0_db=10).

Symbol error rate of M-PSK in AWGN — `mpsk_symbol_error`

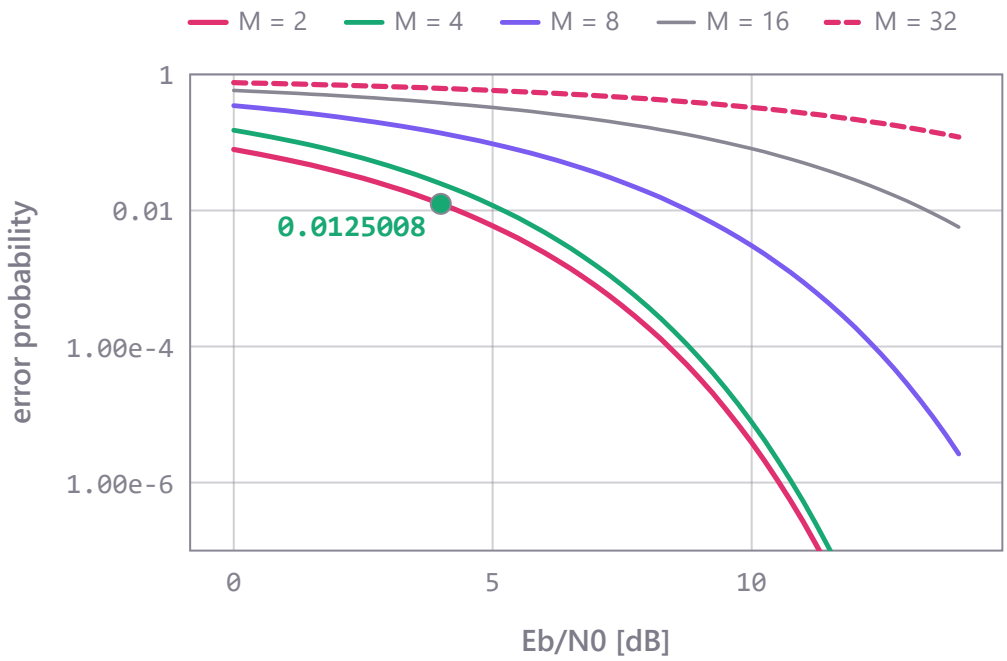
The exact symbol error rate of coherent M-PSK (M = 2 to 64) at a given Eb/N_0 (Craig's integral), and the Gray-coded bit error rate approximation.

$$P_s = \frac{1}{\pi}\int_0^{(M-1)\pi/M} \exp\left(-\frac{E_s}{N_0}\frac{\sin^2(\pi/M)}{\sin^2\theta}\right)d\theta$$

input	meaning	condition
`M`	number of phases	
`ebn0_db`	Eb/N_0 in dB	

output	meaning
`ser`	symbol error rate
`ber_approx`	bit error rate approximation SER/log2 M

- **Method:** Craig's integral evaluated to 30 digits
- **Assumes:** coherent detection; AWGN channel
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Symbol error probability of M-ary PSK for M = 2, 4, 8, 16 and 32: each doubling of M costs more Eb/N0. Eb/N0 from 0 to 14 dB, one run of the package per 0.25 dB, logarithmic axis (values below 10^{-7} are off the axis). Marked: the worked example, Eb/N0 = 4.0 dB, ser = 0.0125008.

```
signal_processing_ai999x.solve('mpsk_symbol_error', M='2', ebn0_db=4.0)
# -> ser = 0.0125008, ber_approx = 0.0125008   (SOLVED)
```

```
signal_processing_ai999x.solve('mpsk_symbol_error', M='4', ebn0_db=5.0)
# -> ser = 0.0118723, ber_approx = 0.00593614   (SOLVED)
```

```
signal_processing_ai999x.solve('mpsk_symbol_error', M='8', ebn0_db=10.0)
# -> ser = 0.00303419, ber_approx = 0.0010114   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the symbol error rate of 8-PSK at Eb/N0 = 10 dB?	SOLVED	ser = 0.00303419, ber_approx = 0.0010114
What is the symbol error rate of 8-PSK?	NEEDS_INPUT	mpsk_symbol_error needs ebn0_db (Eb/N0 in dB). Example: solve('mpsk_symbol_error', M='2', ebn0_db=4).

Noise figure of cascaded stages (Friis formula) — `noise_figure_cascade`

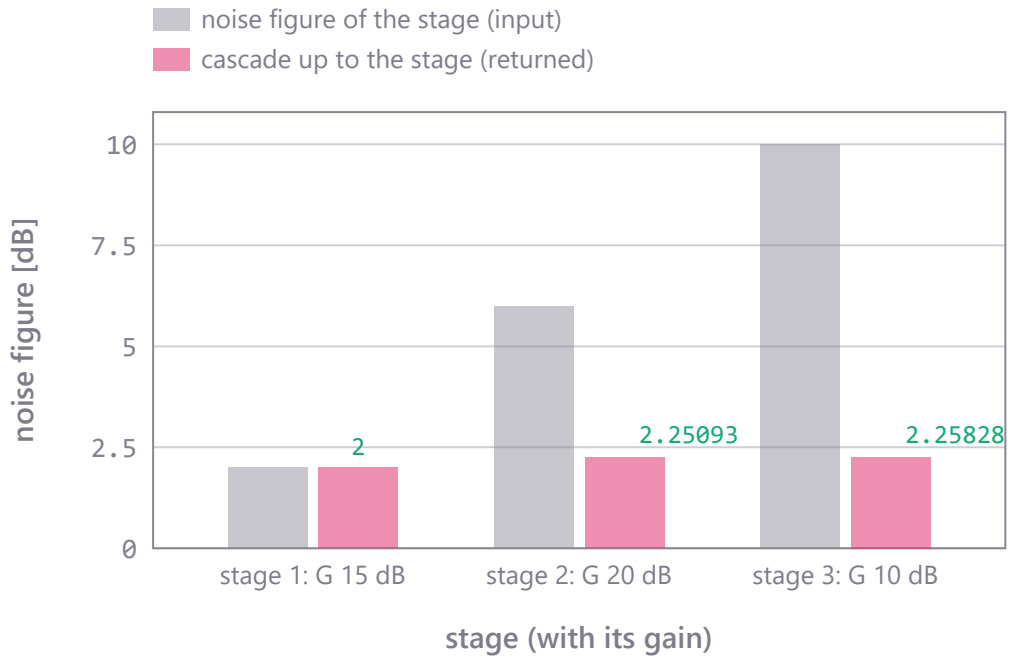
The overall noise figure, noise factor, equivalent noise temperature and gain of a chain of matched stages with given noise figures and gains.

$$F = F_1 + \frac{F_2 - 1}{G_1} + \frac{F_3 - 1}{G_1 G_2} + \cdots$$

input	meaning	condition
`nf_db`	noise figure of each stage in dB, input first	
`gain_db`	available gain of each stage in dB	

output	meaning
`nf_db`	overall noise figure in dB
`F`	overall noise factor
`Te`	equivalent input noise temperature
`gain_db`	overall gain in dB

- **Method:** Friis: F = F1 + (F2 - 1)/G1 + (F3 - 1)/(G1 G2) + ...
- **Assumes:** impedance-matched stages; noise figures referred to $T_0 = 290$ K; one gain per stage
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Friis noise cascade of 3 stages with noise figures [2, 6, 10] dB and gains [15, 20, 10] dB (inputs): the cascade figure after each stage, each a run of the package on the first stages — 2, 2.25093, 2.25828 dB. The first stage dominates; later noisy stages add little because the gain in front divides their contribution. Whole chain: F = 1.68201, Te = 197.783 K, gain 45 dB.

```
signal_processing_ai999x.solve('noise_figure_cascade', nf_db='2, 6, 10', gain_db='15, 20, 10')
# -> nf_db = 2.25828, F = 1.68201, Te = 197.783 K, gain_db = 45   (SOLVED)
```

```
signal_processing_ai999x.solve('noise_figure_cascade', nf_db=[3, 3], gain_db=[10, 10])
# -> nf_db = 3.2114, F = 2.09479, Te = 317.489 K, gain_db = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the overall noise figure of a cascade with noise figures [2, 6, 10] dB and gains [15, 20, 10] dB.	SOLVED	nf_db = 2.25828, F = 1.68201, Te = 197.783 K, gain_db = 45
Find the overall noise figure of a cascade with noise figures [2, 6] dB and gains [15] dB.	INVALID_INPUT	Noise figure of cascaded stages (Friis formula): give one noise figure and one gain per stage.

Free-space path loss — `free_space_path_loss`

The loss (4 pi d f / c)^2 between isotropic antennas a distance d apart at frequency f, in dB and as a ratio.

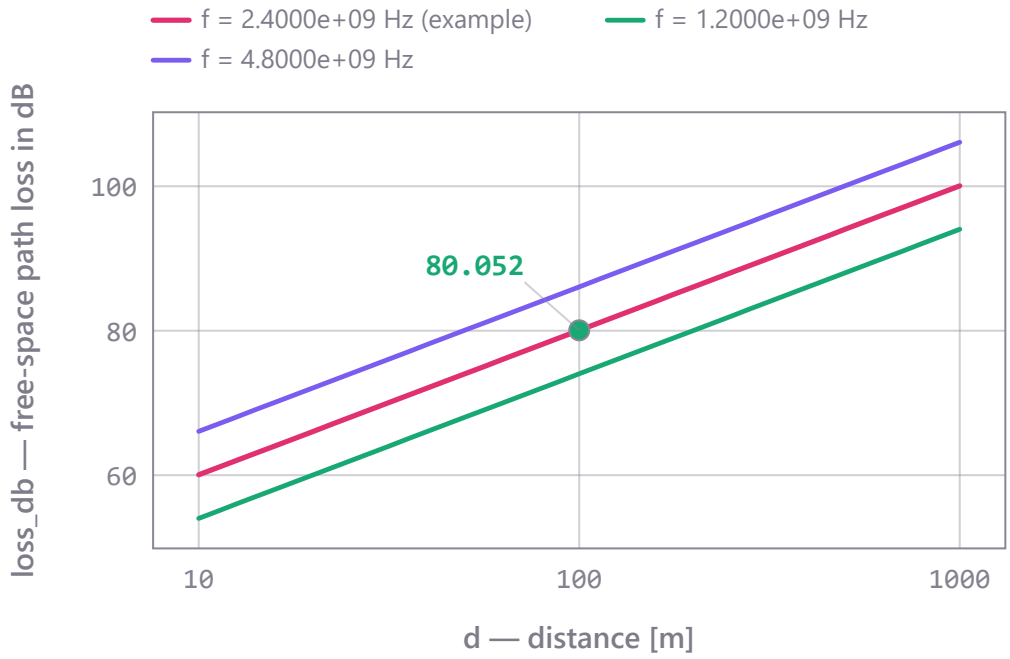
$$L_{\text{fs}} = \left(\frac{4\pi d f}{c}\right)^2$$

input	meaning	condition
`d`	distance	
`f`	frequency	

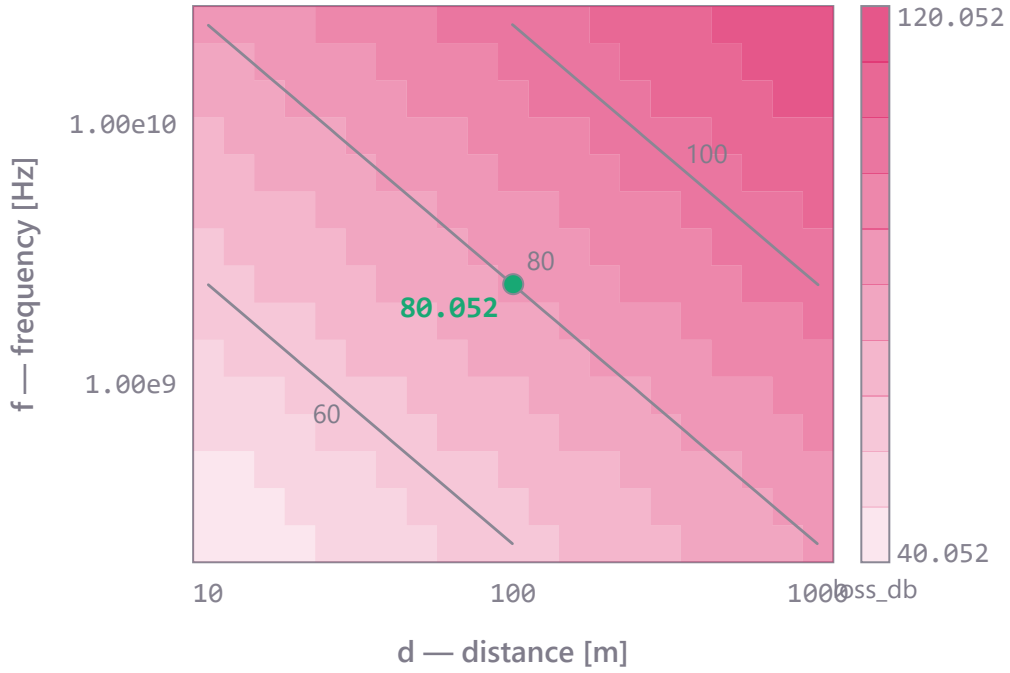
output	meaning
`loss_db`	free-space path loss in dB

output	meaning
`loss`	path loss as a power ratio
`wavelength`	wavelength

- **Method:** FSPL = (4 pi d / lambda)^2
- **Assumes:** free space, line of sight; far field (d >= one wavelength); isotropic antennas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



loss_db (free-space path loss in dB) against d (distance) from 10 m to 1000 m on a logarithmic axis. One curve for each value of f (frequency): 2.4000e+09 Hz, 1.2000e+09 Hz, 4.8000e+09 Hz. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, d = 100 m, loss_db = 80.052.



loss_db (free-space path loss in dB) over d (distance) from 10 m to 1000 m and f (frequency) from 2.4000e+08 Hz to 2.4000e+10 Hz (logarithmic axes): each of the 21 × 15 cells is one run of signal-processing-ai999x, shaded by its value (light 40.052 to dark 120.052); iso-lines at loss_db = 60, 80, 100. Marked: the worked example, d = 100 m, f = 2.4000e+09 Hz, loss_db = 80.052.

```
signal_processing_ai999x.solve('free_space_path_loss', d=100.0, f=2400000000.0)
# -> loss_db = 80.052, loss = 1.01205e+08, wavelength = 0.124914 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the free-space path loss at 2.4 GHz over a distance of 100 m?	SOLVED	loss_db = 80.052, loss = 1.01205e+08, wavelength = 0.124914 m
What is the free-space path loss at 2.4 GHz over a distance of 1 cm?	INVALID_INPUT	Free-space path loss: the distance must be at least one wavelength (far field).
What is the free-space path loss at 2.4 GHz?	NEEDS_INPUT	free_space_path_loss needs d (distance). Example: solve('free_space_path_loss', d='100 m', f='2.4e+09 Hz').

Received power of a radio link (Friis transmission equation) — `friis_received_power`

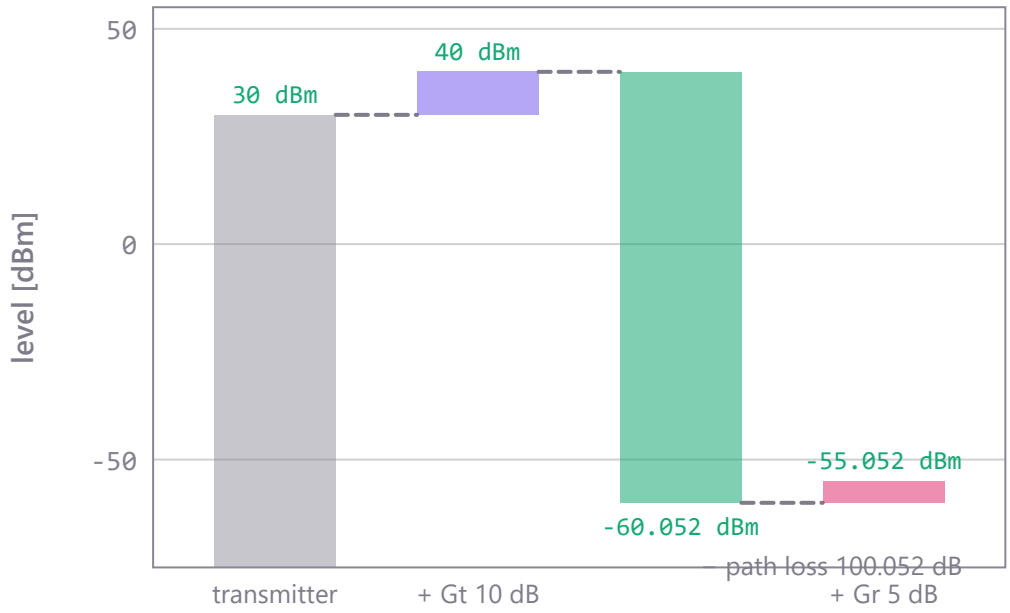
The power received over a free-space link: $Pr = Pt$ Gt Gr (lambda / 4 pi d)^2, in dBm and watts.

$$P_r = P_t G_t G_r \left(\frac{\lambda}{4\pi d}\right)^2$$

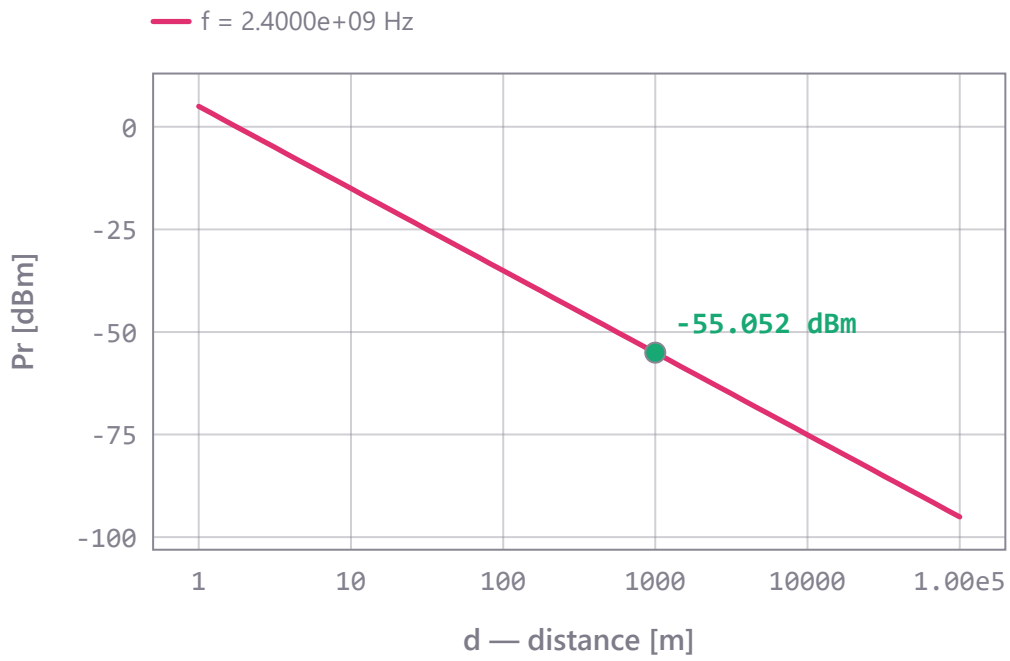
input	meaning	condition
`Pt`	transmit power	
`Gt_db`	transmit antenna gain in dBi	
`Gr_db`	receive antenna gain in dBi	
`f`	frequency	
`d`	distance	

output	meaning
`Pr_dbm`	received power in dBm
`Pr`	received power

- **Method:** link budget in dB: Pt + Gt + Gr - FSPL
- **Assumes:** free space, line of sight; far field; matched, co-polarized antennas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A free-space link budget: Pt = 1 W = 30 dBm, transmit gain 10 dBi, path loss 100.052 dB at d = 1000 m and f = 2.4000e+09 Hz (from the package's dBm and path-loss capabilities), receive gain 5 dBi; the running level ends at the package's received power Pr = -55.052 dBm = 3.1246e-09 W.



Received power against distance from 1 m to 100 km (61 runs of the package, log axis) for Pt = 1.0 W, Gt = 10.0 dBi, Gr = 5.0 dBi, f = 2400000000.0 Hz: a straight line falling 20 dB per decade of distance. Marked: d = 1000.0 m, Pr = -55.052 dBm.

```
signal_processing_ai999x.solve('friis_received_power', Pt=1.0, Gt_db=10.0, Gr_db=5.0,
f=2400000000.0, d=1000.0)
# -> Pr_dbm = -55.052, Pr = 3.12463e-09 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the received power for a transmit power of 1 W, a transmit antenna gain of 10 dBi, a receive antenna gain of 5 dBi, a frequency of 2.4 GHz and a distance of 1 km.	SOLVED	Pr_dbm = -55.052, Pr = 3.12463e-09 W
Find the received power for a transmit power of 30 dBm, a transmit antenna gain of 10 dBi, a receive antenna gain of 5 dBi, a frequency of 2.4 GHz and a distance of 1 km.	SOLVED	Pr_dbm = -55.052, Pr = 3.12463e-09 W
Find the received power for a transmit power of 1 W, a transmit antenna gain of 10 dBi, a frequency of 2.4 GHz and a distance of 1 km.	NEEDS_INPUT	friis_received_power needs Gr_db (receive antenna gain in dBi). Example: solve('friis_received_power', Pt='1 W', Gt_db=1

Gain of a parabolic dish (aperture) antenna — `dish_antenna_gain`

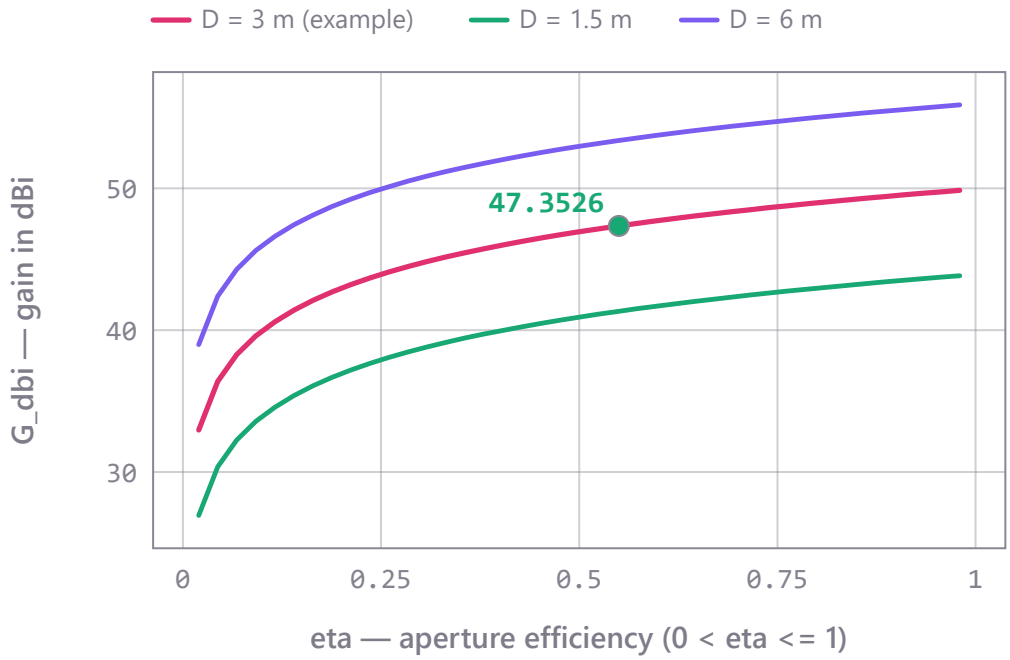
The gain eta (pi D / lambda)^2 of a circular aperture antenna of diameter D and aperture efficiency eta, in dBi, and its effective aperture.

$$G = \eta\left(\frac{\pi D}{\lambda}\right)^2$$

input	meaning	condition
`D`	dish diameter	
`f`	frequency	
`eta`	aperture efficiency $(0 < \eta \leq 1)$	

output	meaning
`G_dbi`	gain in dBi
`G`	gain as a ratio
`Ae`	effective aperture

- **Method:** $G = \eta$ (pi D / lambda)^2 = 4 pi Ae/λ^2
- **Assumes:** $0 < \eta \leq 1$; D >= one wavelength
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G_dbi (gain in dBi) against eta (aperture efficiency (0 < eta <= 1)) from 0.02 to 0.98. One curve for each value of D (dish diameter): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: f = 10000000000.0 Hz. 41 points per curve computed by signal-processing-ai999x; marked: the worked example, eta = 0.55, G_dbi = 47.3526.

```
signal_processing_ai999x.solve('dish_antenna_gain', D=3.0, f=10000000000.0, eta=0.55)
# -> G_dbi = 47.3526, G = 54358, Ae = 3.88772 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the gain of a parabolic dish with a diameter of 3 m at 10 GHz and an aperture efficiency of 55%?	SOLVED	G_dbi = 47.3526, G = 54358, Ae = 3.88772 m^2
What is the gain of a parabolic dish with a diameter of 3 m at 10 GHz?	NEEDS_INPUT	dish_antenna_gain needs eta (aperture efficiency (0 < eta <= 1)). Example: solve('dish_antenna_gain', D='3 m', f='1e+10

Received echo power (radar range equation) — `radar_received_power`

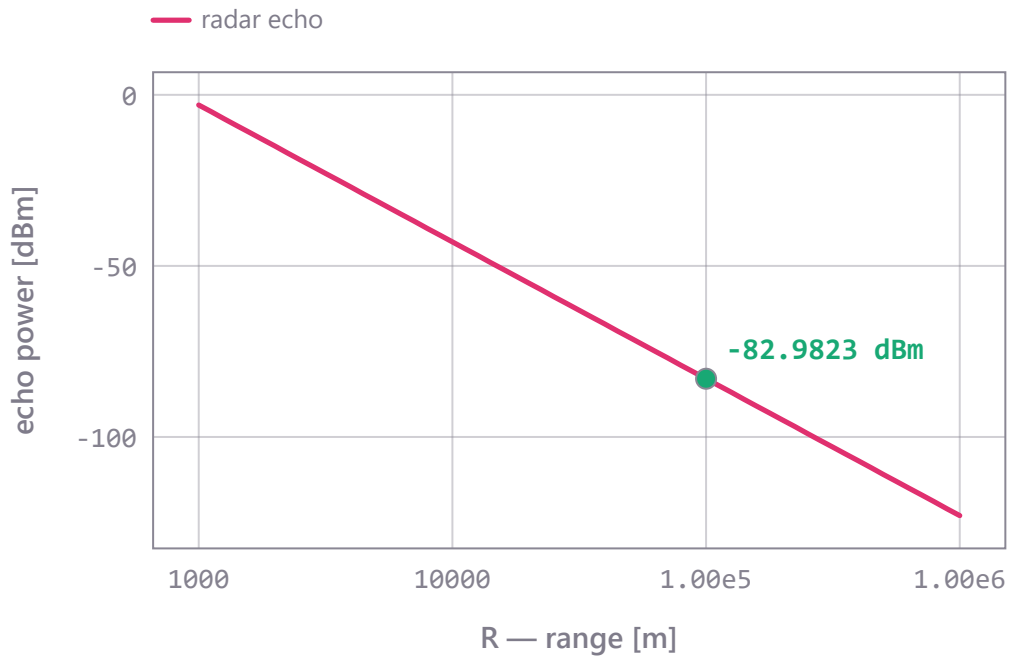
The echo power $Pr = Pt\,G^2\,\lambda^2$ sigma / ((4 pi)^3 R^4) of a monostatic radar from a target of cross-section sigma at range R.

$$P_r = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4}$$

input	meaning	condition
`Pt`	peak transmit power	
`G_db`	antenna gain in dBi (transmit and receive)	
`f`	frequency	
`sigma`	radar cross-section	
`R`	range to the target	

output	meaning
`Pr`	received echo power
`Pr_dbm`	received echo power in dBm

- **Method:** radar range equation
- **Assumes:** monostatic radar, one antenna; free space, no losses; far field
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Echo power against range from 1 km to 1000 km (61 runs of the package, log axis) for Pt = 1000000.0 W, antenna gain 40.0 dBi, f = 3000000000.0 Hz, target cross-section σ = 1.0 m^2: 40 dB lost per decade of range (the $1/R^4$ law). Marked: R = 100000.0 m, Pr = -82.9823 dBm = 5.0323e-12 W.

```
signal_processing_ai999x.solve('radar_received_power', Pt=1000000.0, G_db=40.0, f=3000000000.0,
sigma=1.0, R=100000.0)
# -> Pr = 5.03233e-12 W, Pr_dbm = -82.9823   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A radar with a peak power of 1 MW and an antenna gain of 40 dBi operates at 3 GHz. What is the received echo power from a target with a radar cross-section of 1 m^2 at a range of 100 km?	SOLVED	Pr = 5.03233e-12 W, Pr_dbm = -82.9823
A radar with a peak power of 1 MW operates at 3 GHz. What is the received echo power from a target with a radar cross-section of 1 m^2 at a range of 100 km?	NEEDS_INPUT	radar_received_power needs G_db (antenna gain in dBi (transmit and receive)). Example: solve('radar_received_power', Pt=

Maximum detection range of a radar — `radar_max_range`

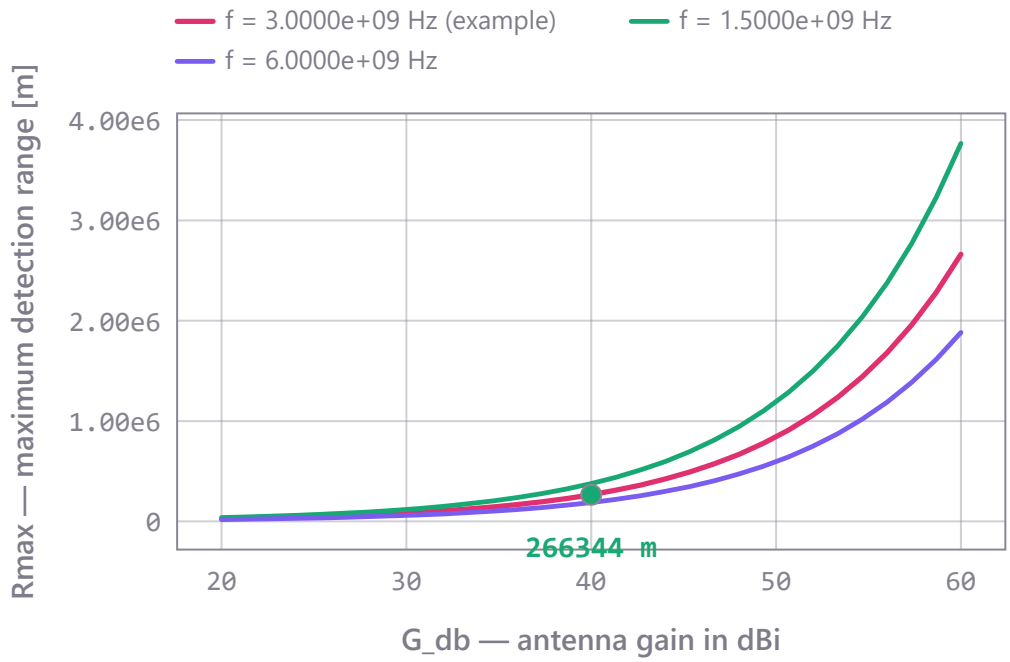
The largest range R = (Pt G^2 λ^2 sigma / ((4 pi)^3 Smin))^$(1/4)$ at which the echo still reaches the minimum detectable signal.

$$R_{\max} = \left(\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 S_{\min}} \right)^{1/4}$$

input	meaning	condition
`Pt`	peak transmit power	
`G_db`	antenna gain in dBi	
`f`	frequency	
`sigma`	radar cross-section	
`Pmin`	minimum detectable signal power	

output	meaning
`Rmax`	maximum detection range

- **Method:** radar range equation solved for R
- **Assumes:** monostatic radar; free space, no losses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rmax (maximum detection range) against G_db (antenna gain in dBi) from 20 to 60. One curve for each value of f (frequency): 3.0000e+09 Hz, 1.5000e+09 Hz, 6.0000e+09 Hz. Other inputs as in the worked example: Pt = 1000000.0 W, sigma = 1.0 m^2, Pmin = 1e-13 W. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, G_db = 40, Rmax = 266344 m.

```
signal_processing_ai999x.solve('radar_max_range', Pt=1000000.0, G_db=40.0, f=3000000000.0,
sigma=1.0, Pmin=1e-13)
# -> Rmax = 266344 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the maximum range of a radar with a peak power of 1 MW, an antenna gain of 40 dBi, a frequency of 3 GHz, a radar cross-section of 1 m^2 and a minimum detectable signal of 1e-13 W?	SOLVED	Rmax = 266344 m

Doppler shift of a radar echo — `radar_doppler`

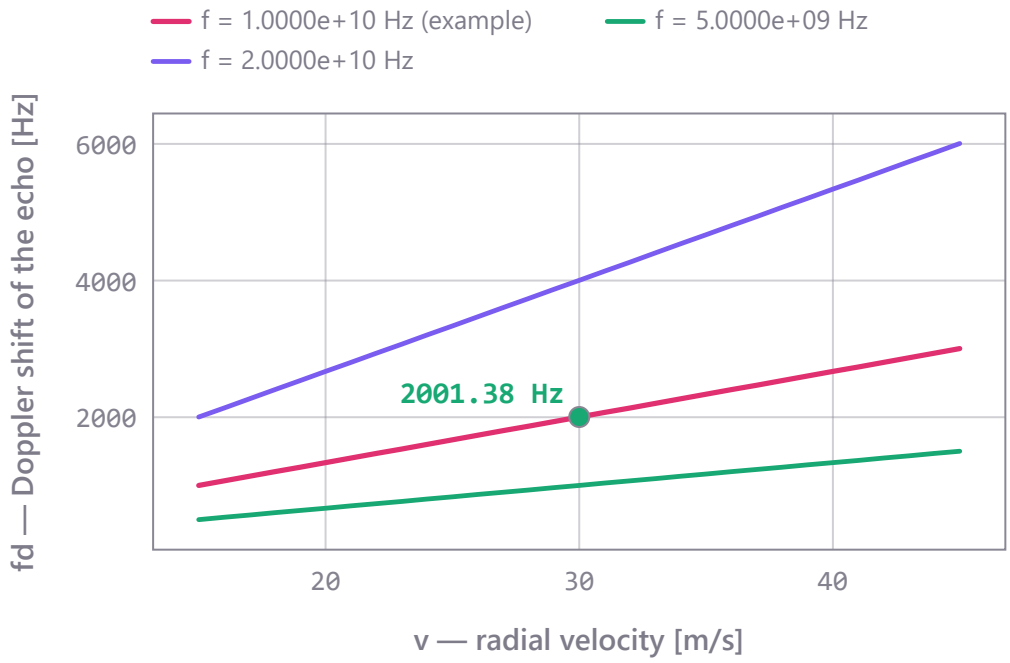
The two-way Doppler shift of the echo from a target closing at radial velocity v (negative when receding): $fd = 2$ v f / (c - v), essentially 2 v / lambda.

$$f_d = \frac{2vf}{c - v} \approx \frac{2v}{\lambda}$$

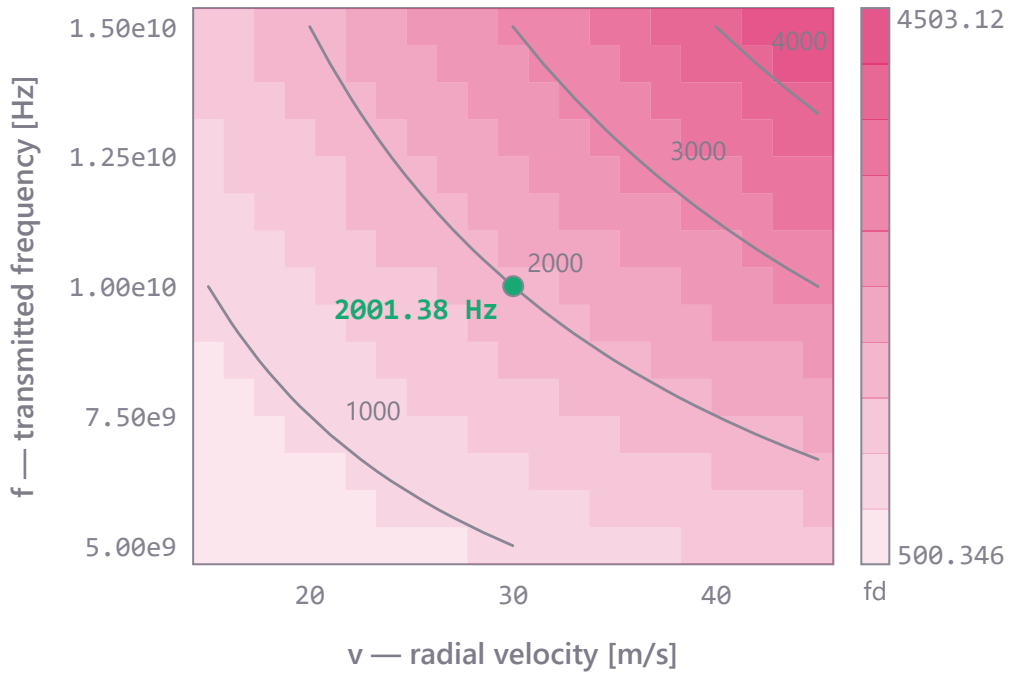
input	meaning	condition
`v`	radial velocity, positive when approaching	
`f`	transmitted frequency	

output	meaning
`fd`	Doppler shift of the echo
`wavelength`	wavelength

- **Method:** two-way Doppler of a reflector: $f_r = f$ (c + v)/(c - v)
- **Assumes:** monostatic radar; |v| much smaller than c
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fd (Doppler shift of the echo) against v (radial velocity, positive when approaching) from 15 m/s to 45 m/s. One curve for each value of f (transmitted frequency): 1.0000e+10 Hz, 5.0000e+09 Hz, 2.0000e+10 Hz. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, v = 30 m/s, fd = 2001.38 Hz.



fd (Doppler shift of the echo) over v (radial velocity, positive when approaching) from 15 m/s to 45 m/s and f (transmitted frequency) from 5.0000e+09 Hz to 1.5000e+10 Hz: each of the 21 × 15 cells is one run of signal-processing-ai999x, shaded by its value (light 500.346 Hz to dark 4503.12 Hz); iso-lines at fd = 1000, 2000, 3000, 4000 Hz. Marked: the worked example, v = 30 m/s, f = 1.0000e+10 Hz, fd = 2001.38 Hz.

```
signal_processing_ai999x.solve('radar_doppler', v=30.0, f=10000000000.0)
# -> fd = 2001.38 Hz, wavelength = 0.0299792 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A 10 GHz radar sees a target approaching at 30 m/s. What is the Doppler shift?	SOLVED	fd = 2001.38 Hz, wavelength = 0.0299792 m
What is the Doppler shift of the radar echo from a target approaching at 30 m/s?	NEEDS_INPUT	radar_doppler needs f (transmitted frequency). Example: solve('radar_doppler', v='30 m/s', f='1e+10 Hz').

Reflection coefficient, VSWR and return loss of a load — `reflection_coefficient`

For a load ZL = RL + jXL on a line of characteristic impedance Z0: the reflection coefficient (magnitude, angle, parts), VSWR, return loss and mismatch loss.

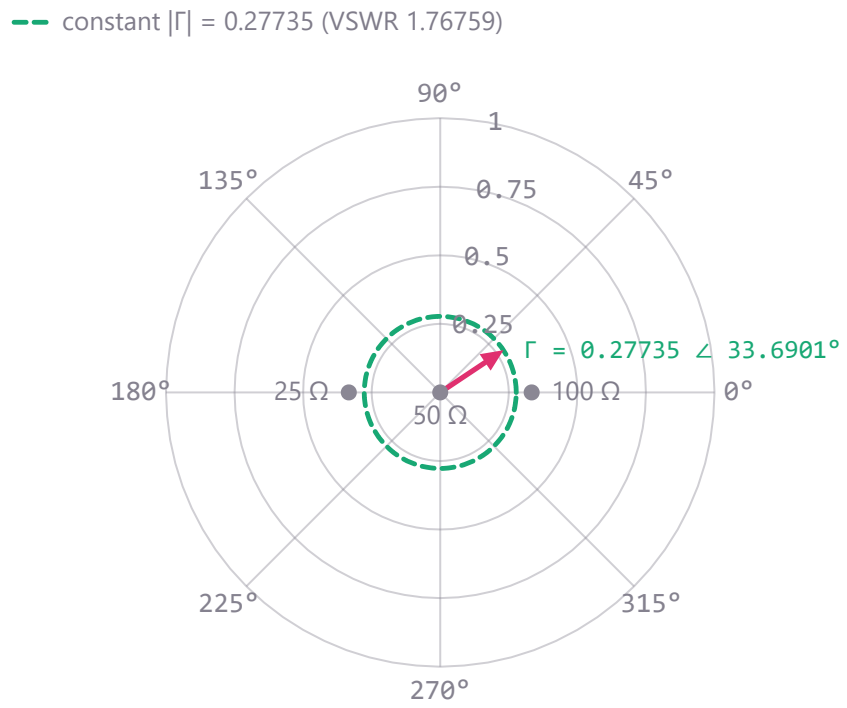
$$\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}, \quad \mathrm{VSWR} = \frac{1 + |\Gamma|}{1 - |\Gamma|}$$

input	meaning	condition
`RL`	load resistance	
`Z0`	characteristic impedance of the line	
`XL`	load reactance (0 if not given)	

output	meaning
`gamma_mag`	|Gamma|

output	meaning
`gamma_angle_deg`	angle of Gamma in degrees
`gamma_re`	real part of Gamma
`gamma_im`	imaginary part of Gamma
`vswr`	voltage standing wave ratio
`return_loss_db`	return loss in dB
`mismatch_loss_db`	mismatch loss in dB

- **Method:** $Gamma = (ZL - Z_0)/(ZL + Z_0)$; VSWR = (1 + |Gamma|)/(1 - |Gamma|)
- **Assumes:** $RL > 0$; Z0 real (lossless line)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The reflection coefficient Γ of the load ZL = 75.0 + j25.0 Ω on a Z0 = 50.0 Ω line in the unit circle of the Γ-plane: the package returns Γ = 0.230769 + j0.153846 (|Γ| = 0.27735, angle 33.6901°), VSWR 1.76759, return loss 11.1394 dB. Dashed: the circle of loads with the same |Γ|. Grey dots: purely resistive loads for comparison (25 Ω: Γ = -0.333333; 50 Ω: Γ = 0; 100 Ω: Γ = 0.333333); the centre is a matched load.

```
signal_processing_ai999x.solve('reflection_coefficient', RL=75.0, XL=25.0, Z0=50.0)
# -> gamma_mag = 0.27735, gamma_angle_deg = 33.6901, gamma_re = 0.230769, gamma_im = 0.153846, vswr
= 1.76759, return_loss_db = 11.1394, mismatch_loss_db = 0.347621   (SOLVED)
```

```
signal_processing_ai999x.solve('reflection_coefficient', RL=100.0, Z0=50.0)
# -> gamma_mag = 0.333333, gamma_angle_deg = 0, gamma_re = 0.333333, gamma_im = 0, vswr = 2,
return_loss_db = 9.54243, mismatch_loss_db = 0.511525   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A 50 ohm line is terminated in a load of 75 + j25 ohm. Find the reflection coefficient and the VSWR.	SOLVED	gamma_mag = 0.27735, gamma_angle_deg = 33.6901, gamma_re = 0.230769, gamma_im = 0.153846, vswr = 1.76759, return_loss_db = 11.1394, mismatch_loss_db = 0.347621
What is the VSWR of a 100 ohm load on a 50 ohm line?	SOLVED	gamma_mag = 0.333333, gamma_angle_deg = 0, gamma_re = 0.333333, gamma_im = 0, vswr = 2, return_loss_db = 9.54243, mismatch_loss_db = 0.511525
A 50 ohm line is terminated in a load of -10 ohm. Find the reflection coefficient.	INVALID_INPUT	Reflection coefficient, VSWR and return loss of a load: load resistance must be a positive finite number.

VSWR and return loss from the reflection coefficient — `vswr_from_reflection`

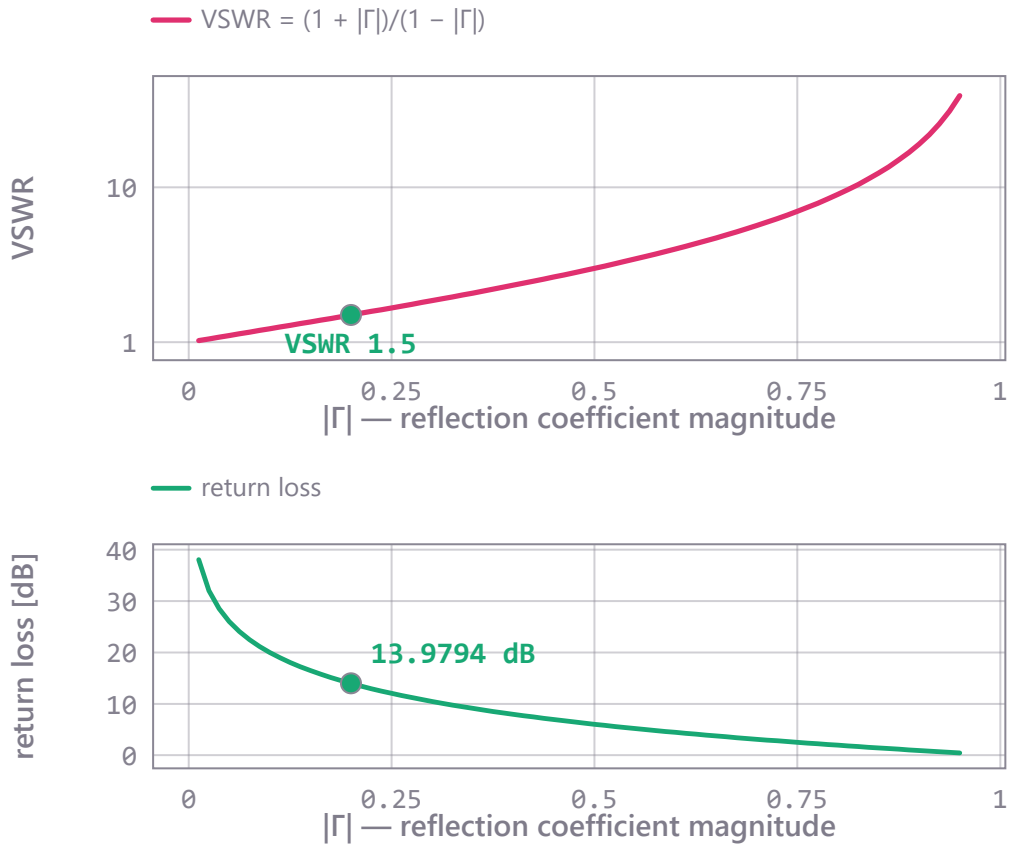
The VSWR (1 + |Gamma|)/(1 - |Gamma|), the return loss -20 log10 |Gamma| and the mismatch loss from the magnitude of the reflection coefficient.

$$\mathrm{VSWR} = \frac{1 + |\Gamma|}{1 - |\Gamma|}$$

input	meaning	condition
`gamma`	magnitude of the reflection coefficient (0 <= |Gamma| < 1)	

output	meaning
`vswr`	voltage standing wave ratio
`return_loss_db`	return loss in dB
`mismatch_loss_db`	mismatch loss in dB

- **Method:** VSWR = (1 + |Gamma|)/(1 - |Gamma|)
- **Assumes:** 0 <= |Gamma| < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Voltage standing-wave ratio (log axis) and return loss for reflection magnitudes 0 to 0.95, 77 runs of the package (return loss is infinite at |Γ| = 0 and left out there). Marked: the worked example |Γ| = 0.2: VSWR 1.5, return loss 13.9794 dB, mismatch loss 0.177288 dB.

```
signal_processing_ai999x.solve('vswr_from_reflection', gamma=0.2)
# -> vswr = 1.5, return_loss_db = 13.9794, mismatch_loss_db = 0.177288   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the VSWR for a reflection coefficient of 0.2?	SOLVED	vswr = 1.5, return_loss_db = 13.9794, mismatch_loss_db = 0.177288
What is the VSWR for a reflection coefficient of 1.2?	INVALID_INPUT	VSWR and return loss from the reflection coefficient: |Gamma| must be below 1.

Reflection coefficient and return loss from the VSWR — `reflection_from_vswr`

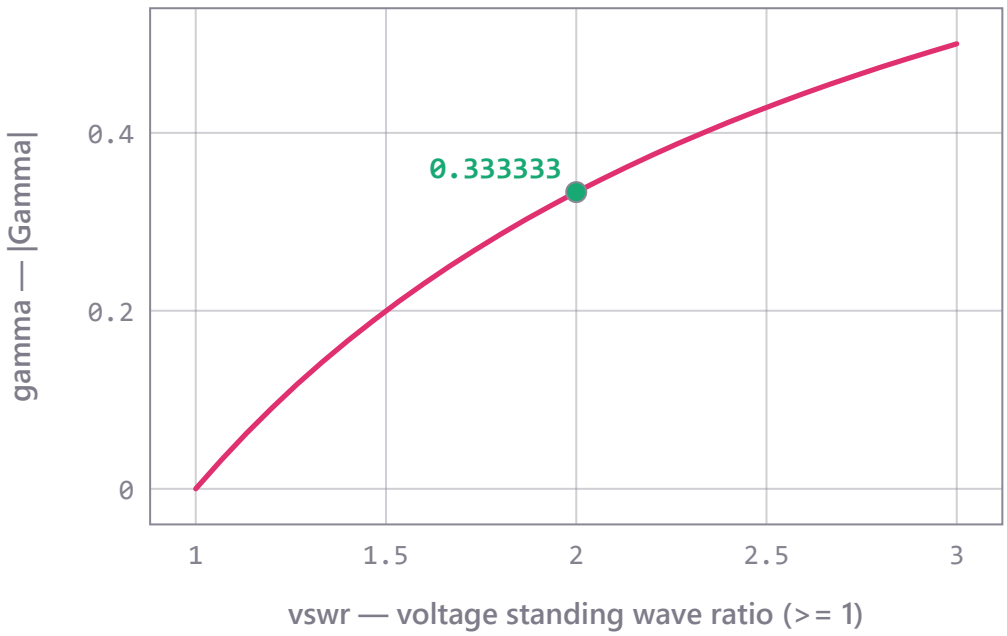
The reflection coefficient magnitude (VSWR - 1)/(VSWR + 1), the return loss and the mismatch loss for a given VSWR.

$$|\Gamma| = \frac{\mathrm{VSWR} - 1}{\mathrm{VSWR} + 1}$$

input	meaning	condition
`vswr`	voltage standing wave ratio (>= 1)	

output	meaning
`gamma`	|Gamma|
`return_loss_db`	return loss in dB
`mismatch_loss_db`	mismatch loss in dB

- **Method:** |Gamma| = (VSWR - 1)/(VSWR + 1)
- **Assumes:** VSWR >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gamma (|Gamma|) against vswr (voltage standing wave ratio (>= 1)) from 1 to 3. 31 points computed by signal-processing-ai999x; marked: the worked example, vswr = 2, gamma = 0.333333.

```
signal_processing_ai999x.solve('reflection_from_vswr', vswr=2.0)
# -> gamma = 0.333333, return_loss_db = 9.54243, mismatch_loss_db = 0.511525   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What reflection coefficient and return loss correspond to a VSWR of 2?	SOLVED	gamma = 0.333333, return_loss_db = 9.54243, mismatch_loss_db = 0.511525
What reflection coefficient corresponds to a VSWR of 0.5?	INVALID_INPUT	Reflection coefficient and return loss from the VSWR: the VSWR is at least 1.

Input impedance of a terminated lossless line — `input_impedance`

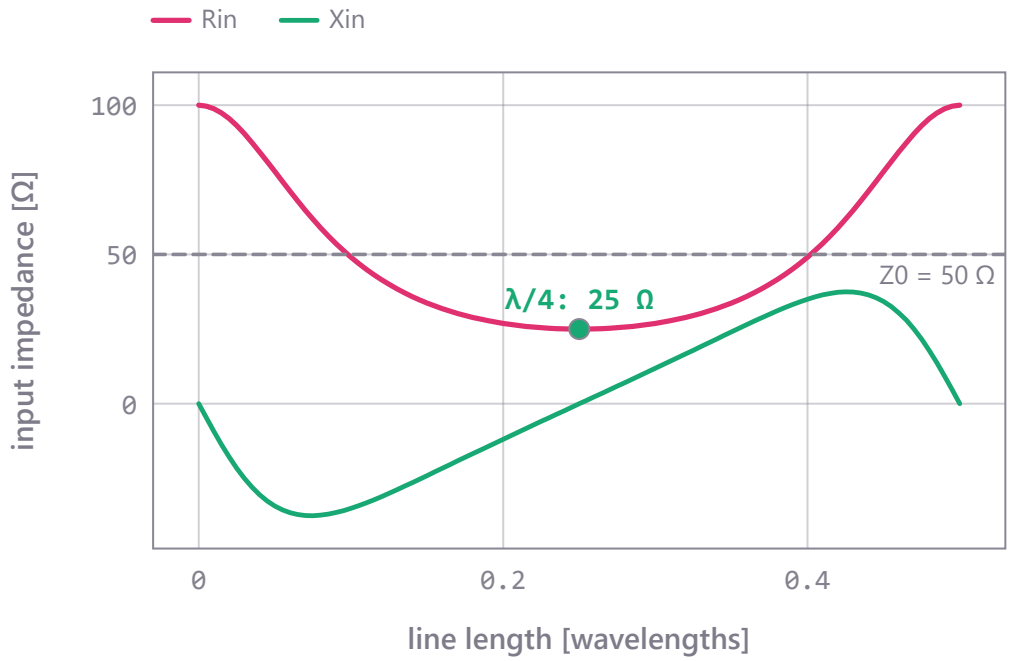
The impedance seen at the input of a lossless line of characteristic impedance Z0 and electrical length l (in wavelengths) terminated in ZL = RL + jXL.

$$Z_{in} = Z_0 \frac{Z_L + jZ_0 \tan \beta l}{Z_0 + jZ_L \tan \beta l}$$

input	meaning	condition
`RL`	load resistance	
`Z0`	characteristic impedance	
`length`	line length in wavelengths	
`XL`	load reactance (0 if not given)	

output	meaning
`Rin`	input resistance
`Xin`	input reactance
`abs_zin`	|Zin|

- **Method:** Gamma moved along the line: $Gamma_in = Gamma_L$ e^{-j 4 pi l / lambda}, Zin = Z0 $(1 + Gamma_in)/(1 - Gamma_in)$
- **Assumes:** lossless line; $RL > 0$; length >= 0 in wavelengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Input impedance of a lossless line of characteristic impedance Z0 = 50 Ω terminated in RL = 100 Ω, for lengths 0 to half a wavelength (101 runs of the package): the impedance repeats every λ/2, and a quarter-wave line turns RL into $Z0^2/RL$. Marked: the worked example length 0.25 λ, Rin = 25 Ω, Xin = 0 Ω.

```
signal_processing_ai999x.solve('input_impedance', RL=100.0, Z0=50.0, length=0.25)
# -> Rin = 25 ohm, Xin = 0 ohm, abs_zin = 25 ohm   (SOLVED)
```

```
signal_processing_ai999x.solve('input_impedance', RL=100.0, Z0=50.0, length=0.125)
# -> Rin = 40 ohm, Xin = -30 ohm, abs_zin = 50 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the input impedance of a 0.125 wavelength 50 ohm line terminated in a load of 100 + j0 ohm?	SOLVED	Rin = 40 ohm, Xin = -30 ohm, abs_zin = 50 ohm
What is the input impedance of a 0.25 wavelength 50 ohm line terminated in a 100 ohm load?	SOLVED	Rin = 25 ohm, Xin = 0 ohm, abs_zin = 25 ohm
What is the input impedance of a 50 ohm line terminated in a 100 ohm load?	NEEDS_INPUT	input_impedance needs length (line length in wavelengths). Example: solve('input_impedance', RL='100 ohm', Z0='50 ohm',

Quarter-wave impedance transformer — `quarter_wave_transformer`

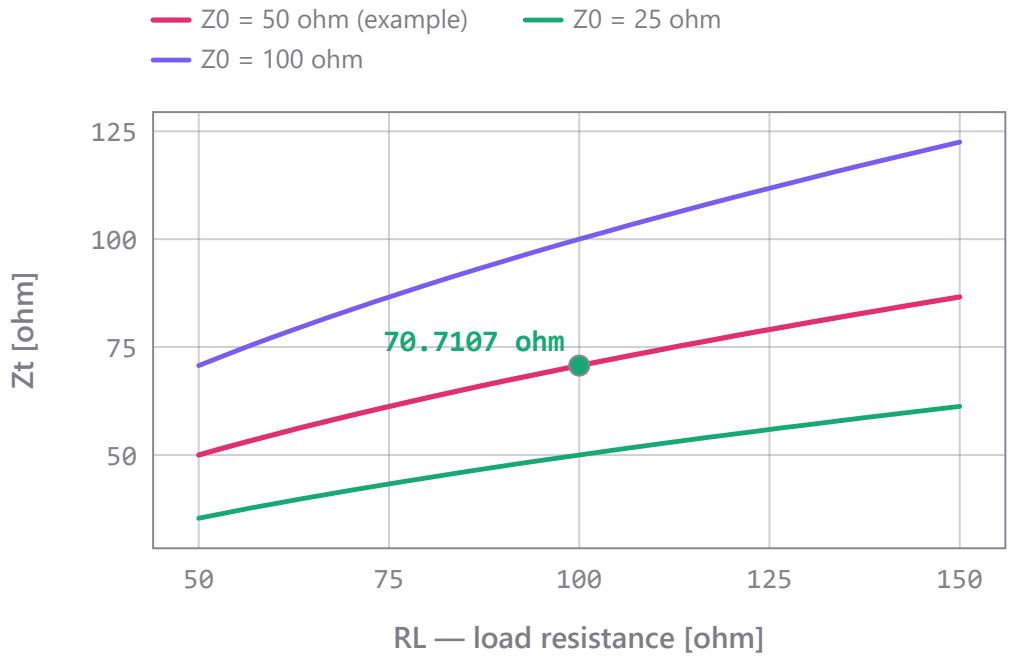
The characteristic impedance sqrt(RL Z0) of the quarter-wave section that matches a resistive load RL to a line Z0, and its physical length at frequency f.

$$Z_t = \sqrt{R_L Z_0}, \quad \ell = \frac{v_f\, c}{4f}$$

input	meaning	condition
`RL`	load resistance	
`Z0`	impedance to match to	
`f`	design frequency	
`vf`	velocity factor of the section (default 1)	

output	meaning
`Zt`	characteristic impedance of the transformer
`length`	physical length of the section (f given)

- **Method:** Zin of a quarter-wave line is Zt^2/RL; set it equal to Z0
- **Assumes:** resistive load; lossless lines; exact match at the design frequency only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Zt (characteristic impedance of the transformer) against RL (load resistance) from 50 ohm to 150 ohm. One curve for each value of Z0 (impedance to match to): 50 ohm, 25 ohm, 100 ohm. Other inputs as in the worked example: f = 1000000000.0 Hz. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, RL = 100 ohm, Zt = 70.7107 ohm.

```
signal_processing_ai999x.solve('quarter_wave_transformer', RL=100.0, Z0=50.0, f=1000000000.0)
# -> Zt = 70.7107 ohm, length = 0.0749481 m   (SOLVED)
```

```
signal_processing_ai999x.solve('quarter_wave_transformer', RL=300.0, Z0=75.0, f=100000000.0,
vf=0.66)
# -> Zt = 150 ohm, length = 0.494658 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Design a quarter-wave transformer to match a 100 ohm load to a 50 ohm line at 1 GHz.	SOLVED	Zt = 70.7107 ohm, length = 0.0749481 m
What impedance must a quarter-wave transformer have to match a 300 ohm load to a 75 ohm line?	SOLVED	Zt = 150 ohm
Design a quarter-wave transformer to match a 100 ohm load.	NEEDS_INPUT	quarter_wave_transformer needs Z0 (impedance to match to). Example: solve('quarter_wave_transformer', RL='100 ohm', Z0='

Characteristic impedance of a coaxial line — `coax_impedance`

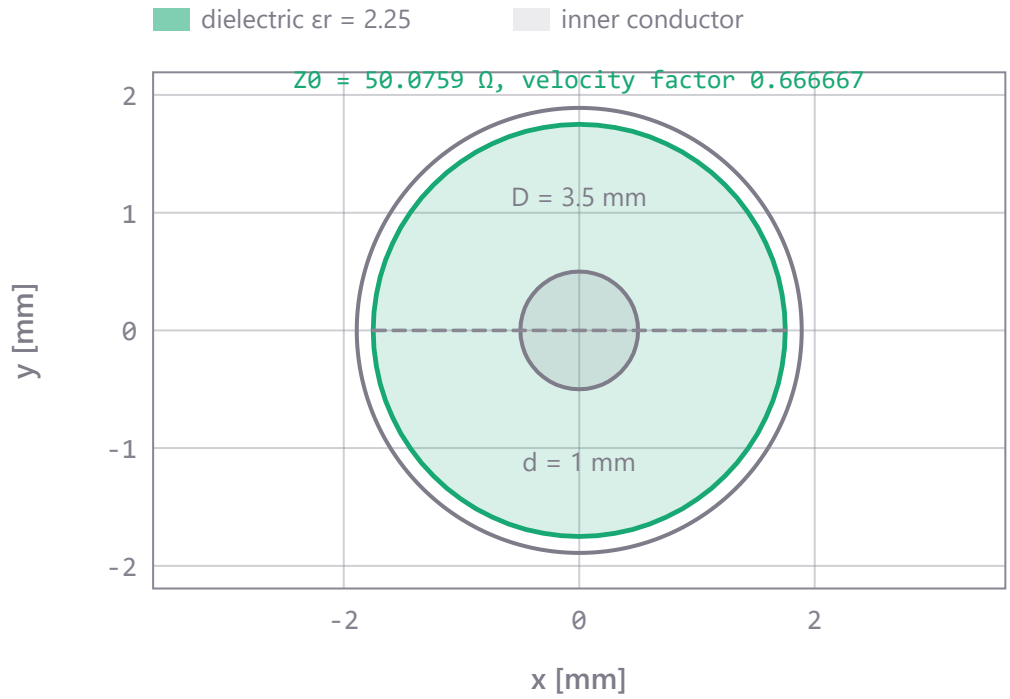
The characteristic impedance (eta0 / (2 pi sqrt(er))) ln(D/d) of a lossless coaxial line and its velocity factor $1/sqrt$(er).

$$Z_0 = \frac{\eta_0}{2\pi\sqrt{\varepsilon_r}} \ln \frac{D}{d}$$

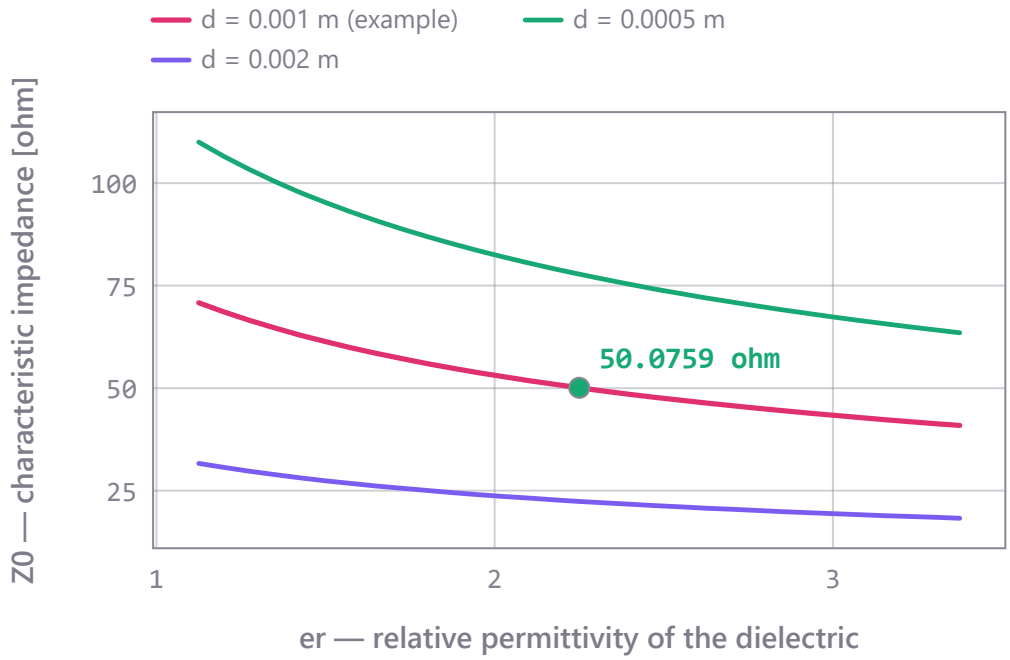
input	meaning	condition
`D`	inner diameter of the outer conductor	
`d`	diameter of the inner conductor	
`er`	relative permittivity of the dielectric (1 if not given)	

output	meaning
`Z0`	characteristic impedance
`vf`	velocity factor

- **Method:** $Z_0 = sqrt$(L'/C') with L' = mu ln(D/d)/(2 pi), C' = 2 pi eps / ln(D/d)
- **Assumes:** $d < D$; $er \geq 1$; lossless conductors and dielectric
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cross-section of a coaxial line to scale (dimensions converted to mm): inner conductor d = 0.001 m, outer conductor inner diameter D = 0.0035 m, dielectric εr = 2.25 (inputs). Returned: characteristic impedance Z0 = 50.0759 Ω and velocity factor 0.666667.



Z0 (characteristic impedance) against er (relative permittivity of the dielectric (1 if not given)) from 1.125 to 3.375. One curve for each value of d (diameter of the inner conductor): 0.001 m, 0.0005 m, 0.002 m. Other inputs as in the worked example: D = 0.0035 m. 31 points per curve computed by signal-processing-ai999x; marked: the worked example, er = 2.25, Z0 = 50.0759 ohm.

```
signal_processing_ai999x.solve('coax_impedance', D=0.0035, d=0.001, er=2.25)
# -> Z0 = 50.0759 ohm, vf = 0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the characteristic impedance of a coaxial cable with an inner conductor diameter of 1 mm, an outer conductor diameter of 3.5 mm and a dielectric constant of 2.25?	SOLVED	Z0 = 50.0759 ohm, vf = 0.666667
What is the characteristic impedance of a coaxial cable with an inner conductor diameter of 4 mm and an outer conductor diameter of 3.5 mm?	INVALID_INPUT	Characteristic impedance of a coaxial line: the inner diameter must be smaller than the outer diameter.

Digital Butterworth filter design (bilinear transform) — `butterworth_design`

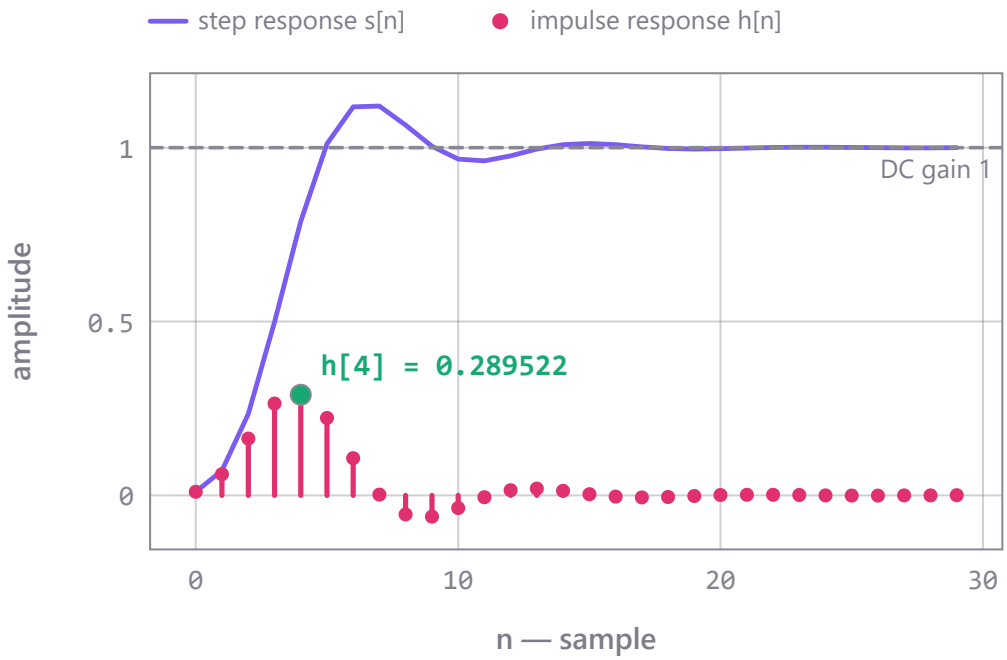
The coefficients b and a of an N-th order digital Butterworth low-pass or high-pass filter with cutoff fc at sampling rate fs (bilinear transform with pre-warping).

$$s = 2f_s\frac{1-z^{-1}}{1+z^{-1}}, \quad \omega_c = 2f_s\tan\frac{\pi f_c}{f_s}$$

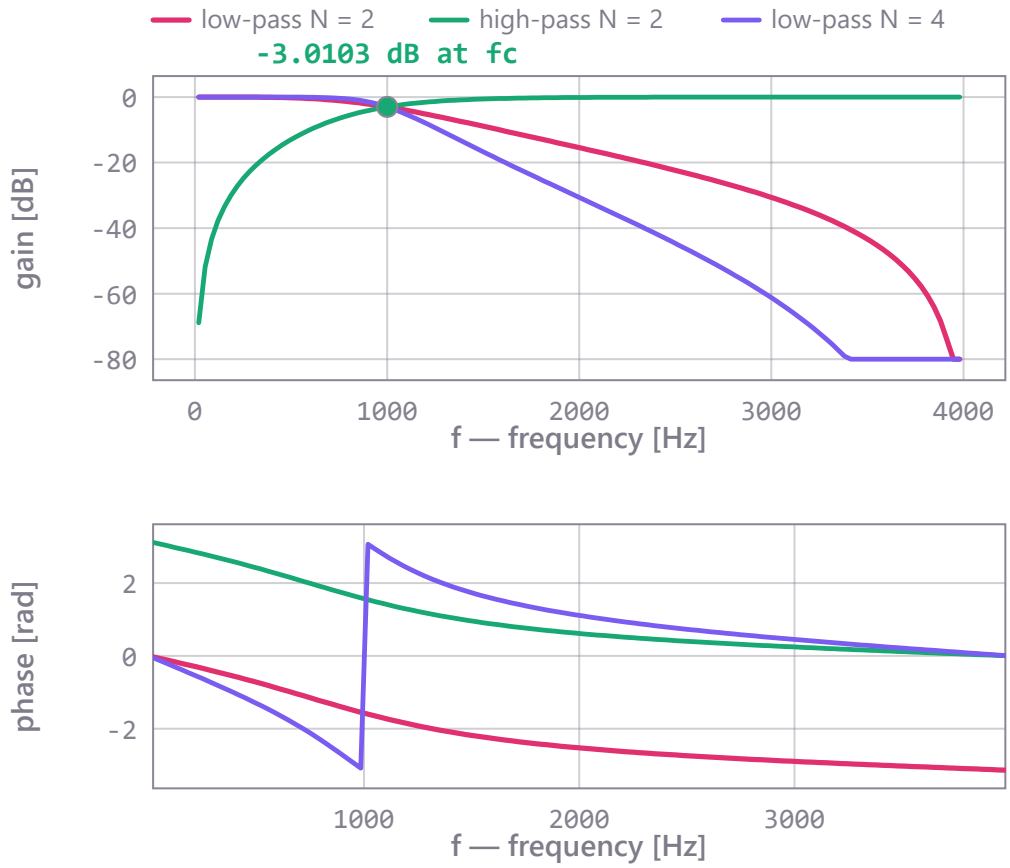
input	meaning	condition
`N`	filter order (1..8)	
`fc`	cutoff (-3 dB) frequency	
`fs`	sampling rate	
`btype`	lowpass or highpass	

output	meaning
`b`	numerator coefficients b[0..N]
`a`	denominator coefficients a[0..N] (a[0] = 1)

- **Method:** analog prototype poles, frequency scaling with the pre-warped cutoff 2 fs tan(pi fc / fs), bilinear transform, polynomial expansion
- **Assumes:** $0 < fc < fs/2$; $1 \le N \le 8$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Impulse response (stems) and step response (line) of the order-4 Butterworth low-pass filter designed by the package for fc = 1000 Hz at fs = 8000 Hz (b = [0.01021, 0.04084, 0.06126, 0.04084, 0.01021], a = [1, -1.968, 1.736, -0.7245, 0.1204]), 30 samples each computed by the package; the step response settles at the DC gain 1 after overshooting to 1.11966. Marked: the largest impulse sample h[4] = 0.289522.



Gain and phase of digital Butterworth filters designed by the package for fc = 1000 Hz at fs = 8000 Hz: the order-2 low-pass (b = [0.09763, 0.1953, 0.09763], a = [1, -0.9428, 0.3333]), the order-2 high-pass (b = [0.569, -1.138, 0.569], a = [1, -0.9428, 0.3333]) and an order-4 low-pass; each response is 120 frequency-response runs of the package from 0 to fs/2 (gain clipped at −80 dB). Marked: the order-2 low-pass at fc, -3.0103 dB.

```
signal_processing_ai999x.solve('butterworth_design', N=2, fc=1000.0, fs=8000.0, btype='lowpass')
# -> b = [0.0976310729378, 0.195262145876, 0.0976310729378], a = [1.0, -0.942809041582,
0.333333333333]   (SOLVED)
```

```
signal_processing_ai999x.solve('butterworth_design', N=1, fc=2000.0, fs=8000.0, btype='lowpass')
# -> b = [0.5, 0.5], a = [1.0, 0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Design a 2nd-order Butterworth low-pass filter with a cutoff frequency of 1 kHz at a sampling rate of 8 kHz. Give the coefficients.	SOLVED	b = [0.0976310729378, 0.195262145876, 0.0976310729378], a = [1.0, -0.942809041582, 0.333333333333]
Design a 3rd-order Butterworth high-pass filter with a cutoff frequency of 500 Hz at a sampling rate of 8 kHz.	SOLVED	b = [0.673479047714, -2.02043714314, 2.02043714314, -0.673479047714], a = [1.0, -2.21916861831, 1.71511783003, -0.453545933366]
Design a 2nd-order Butterworth low-pass filter with a cutoff frequency of 5 kHz at a sampling rate of 8 kHz.	INVALID_INPUT	Digital Butterworth filter design (bilinear transform): the cutoff must lie below fs/2.

FIR low-pass filter design by the window method — `fir_window_design`

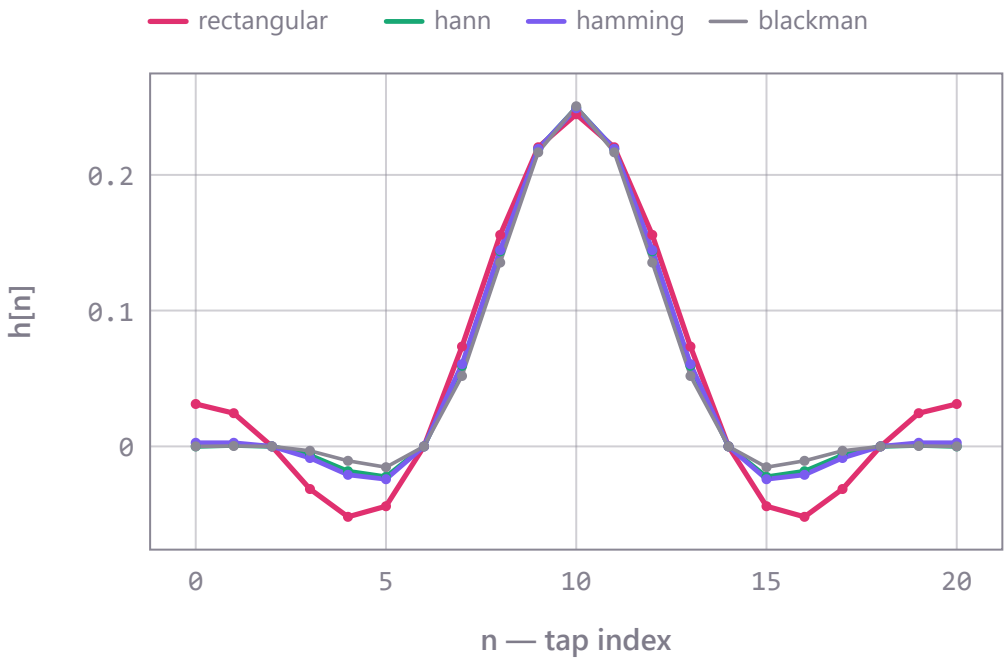
The N taps of a linear-phase FIR low-pass filter with cutoff fc: an ideal (sinc) impulse response shifted by $(N-1)/2$, multiplied by a rectangular, Hann, Hamming or Blackman window and scaled to unit DC gain.

$$h[n] \propto \frac{2f_c}{f_s}\,\mathrm{sinc}\!\left(\frac{2f_c}{f_s}\left(n-\tfrac{N-1}{2}\right)\right) w[n]$$

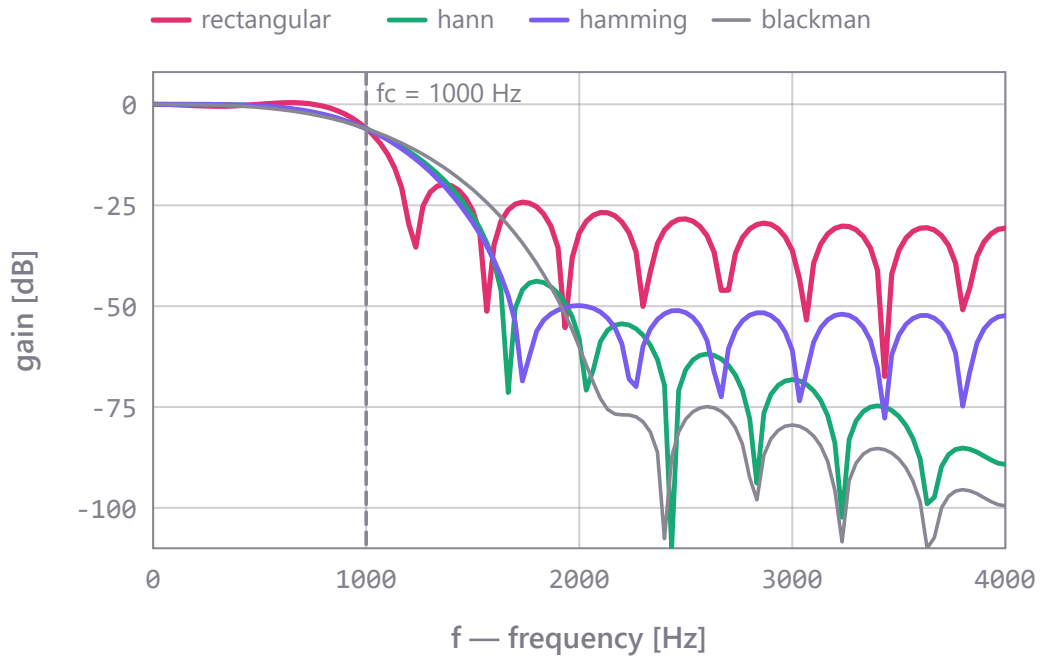
input	meaning	condition
`N`	number of taps	
`fc`	cutoff frequency	
`fs`	sampling rate	
`window`	window type	

output	meaning
`h`	filter taps h[$0..N-1$]

- **Method:** windowed ideal low-pass: h[n] = $(2fc/fs)$ sinc($2fc/fs\,(n-(N-1)/2)$) w[n], normalized to sum 1
- **Assumes:** $0 < fc < fs/2$; symmetric window; taps scaled so that the DC gain is 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Taps of 21-tap low-pass FIR filters designed by the package for fc = 1000 Hz at fs = 8000 Hz with four windows (the worked example is 5 taps, hamming): the windowed sinc, tapered toward zero at the ends by every window but the rectangular one; rectangular centre tap 0.244677; hann centre tap 0.249382; hamming centre tap 0.248999; blackman centre tap 0.250496.



Gain of the four 21-tap FIR low-pass designs of fc = 1000 Hz, fs = 8000 Hz (taps from the package, each response 121 frequency-response runs, clipped at −110 dB): the rectangular window gives the sharpest edge but the highest stopband ripple; Blackman the deepest stopband and widest edge.

```
signal_processing_ai999x.solve('fir_window_design', N=5, fc=1000.0, fs=8000.0, window='hamming')
# -> h = [0.024553834015, 0.234389464238, 0.482113403494, 0.234389464238, 0.024553834015]
(SOLVED)
```

```
signal_processing_ai999x.solve('fir_window_design', N=3, fc=2000.0, fs=8000.0,
window='rectangular')
# -> h = [0.280049576756, 0.439900846488, 0.280049576756]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Design a 5-tap FIR low-pass filter with a Hamming window, a cutoff frequency of 1 kHz and a sampling rate of 8 kHz.	SOLVED	h = [0.024553834015, 0.234389464238, 0.482113403494, 0.234389464238, 0.024553834015]
Design a 5-tap FIR low-pass filter with a cutoff frequency of 1 kHz and a sampling rate of 8 kHz.	NEEDS_INPUT	fir_window_design needs window (window type). Example: solve('fir_window_design', N=5, fc='1000 Hz', fs='8000 Hz', windo

Second-order IIR notch filter design — `notch_filter_design`

The coefficients of a second-order IIR notch filter removing frequency f0 with quality factor Q (bandwidth f_0/Q *at* -3 dB) at sampling rate fs.

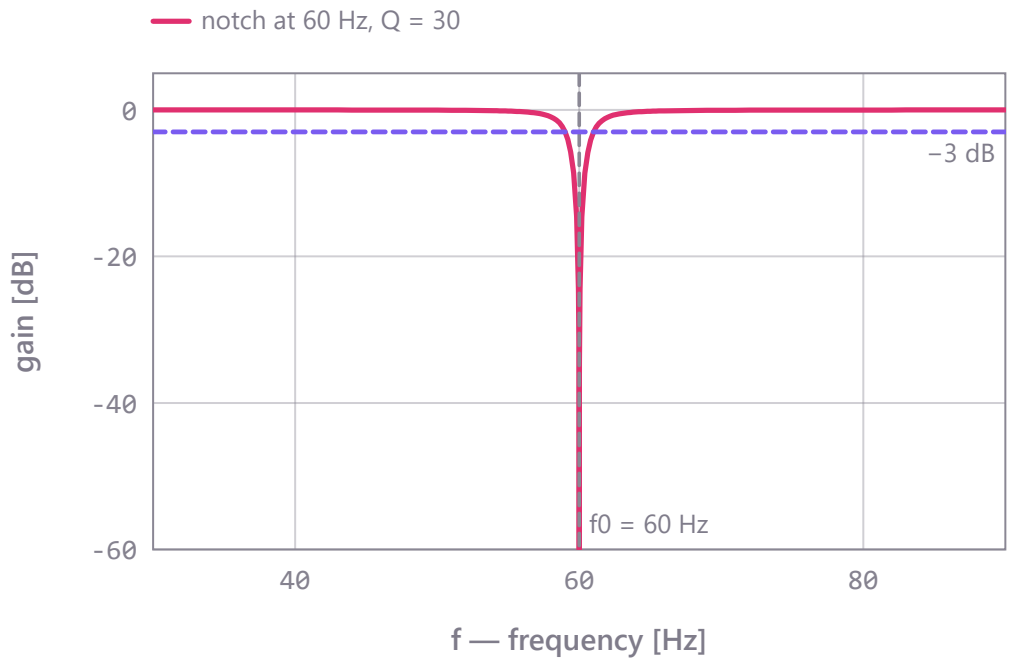
$$H(z) = g\,\frac{1 - 2\cos\omega_0 z^{-1} + z^{-2}}{1 - 2g\cos\omega_0 z^{-1} + (2g-1)z^{-2}},\; g = \frac{1}{1+\tan(\omega_0/2Q)}$$

input	meaning	condition
`f0`	frequency to remove	

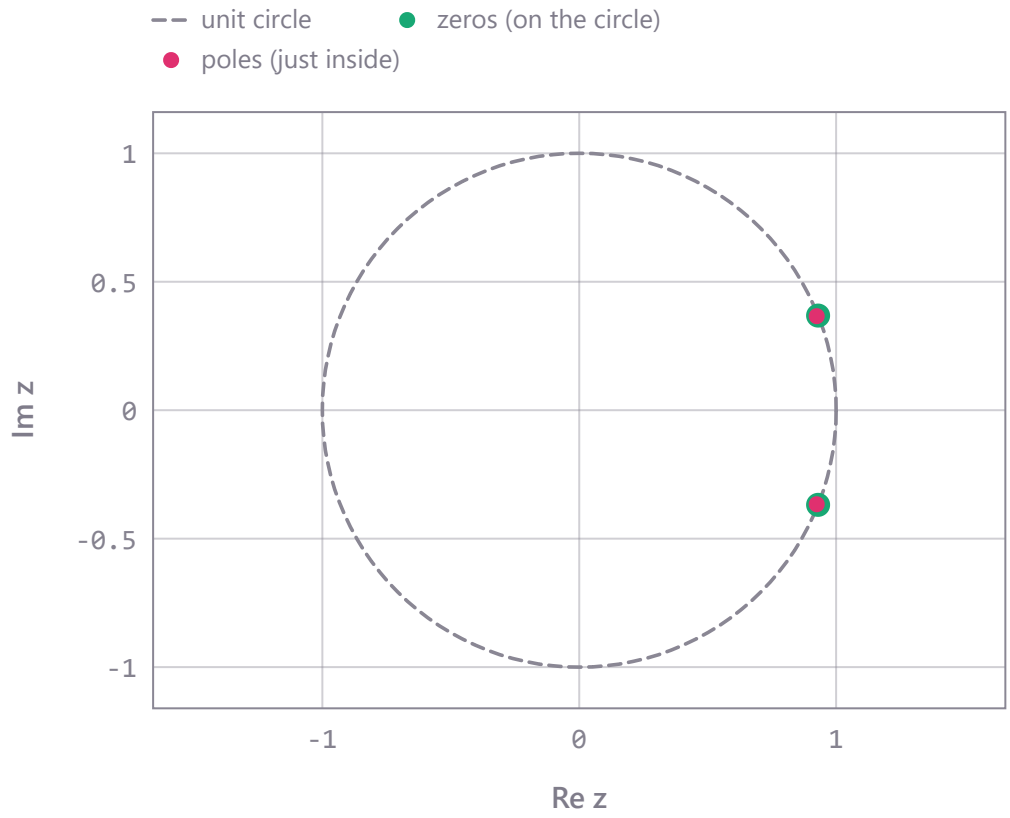
input	meaning	condition
`Q`	quality factor f0 / bandwidth	
`fs`	sampling rate	

output	meaning
`b`	numerator coefficients b0, b1, b2
`a`	denominator coefficients 1, a1, a2

- **Method:** zeros on the unit circle at f0; poles at the same angle, radius set by tan(w0 / 2Q)
- **Assumes:** $0 < f_0 < fs/2$; $Q > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The gain of the second-order notch filter designed by the package for f0 = 60 Hz, Q = 30, fs = 1000 Hz (b = [0.993756, -1.84794, 0.993756], a = [1, -1.84794, 0.987512]): 120 frequency-response runs from f0/2 to 1.5 f0, dense near f0 and down to 1 ppm from it; the plot is clipped at −60 dB (exactly at f0 the gain is zero, −∞ dB). The notch is only about f0/Q = 2 Hz wide at −3 dB; elsewhere the gain is 0 dB.



Poles and zeros of the notch filter designed by the package for f0 = 60.0 Hz, Q = 30.0, fs = 1000.0 Hz (from the package's pole–zero capability): the zeros sit on the unit circle at angle ±2πf0/fs and the poles just inside them, largest radius 0.993736 (stable).

```
signal_processing_ai999x.solve('notch_filter_design', f0=60.0, Q=30.0, fs=1000.0)
# -> b = [0.9937559649536571, -1.8479418578501994, 0.9937559649536571], a = [1.0,
-1.8479418578501994, 0.9875119299073143]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Design a notch filter at 60 Hz with a quality factor of 30 for a sampling rate of 1 kHz.	SOLVED	b = [0.9937559649536571, -1.8479418578501994, 0.9937559649536571], a = [1.0, -1.8479418578501994, 0.9875119299073143]
Design a notch filter at 600 Hz with a quality factor of 30 for a sampling rate of 1 kHz.	INVALID_INPUT	Second-order IIR notch filter design: the notch frequency must lie below fs/2.

Haar discrete wavelet transform — `haar_dwt`

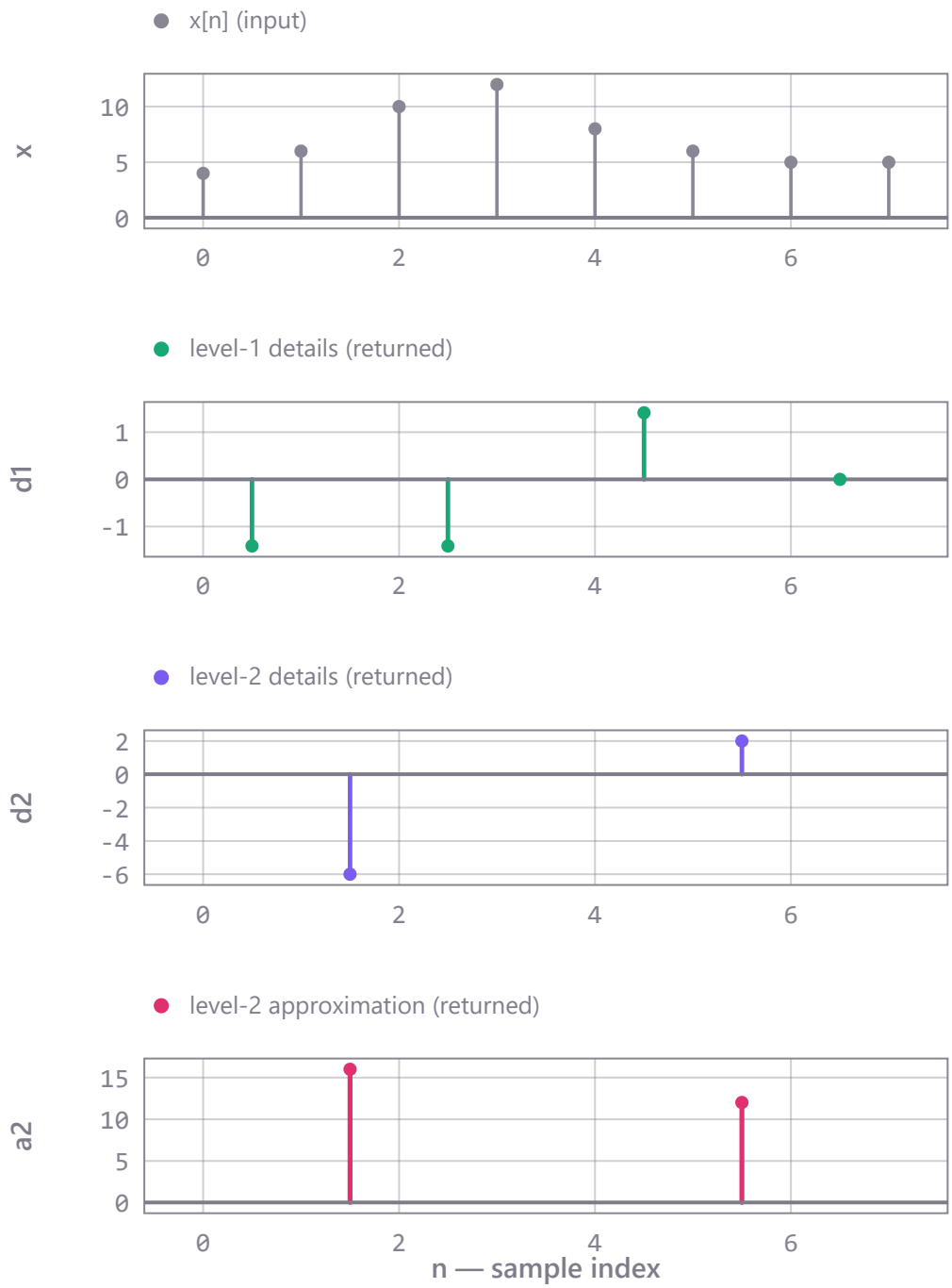
The multi-level orthonormal Haar wavelet transform of a sequence whose length is a power of two: the final approximation and the detail coefficients of every level (coarsest first).

$$c_A[k] = \frac{x[2k] + x[2k+1]}{\sqrt{2}}, \quad c_D[k] = \frac{x[2k] - x[2k+1]}{\sqrt{2}}$$

input	meaning	condition
`x`	samples (length a power of two)	
`levels`	number of levels (default 1)	

output	meaning
`approximation`	approximation coefficients after the last level
`details`	detail coefficients per level, coarsest first

- **Method:** repeated pairwise (a + b)/sqrt 2 and (a - b)/sqrt 2
- **Assumes:** length a power of two; levels <= log2 N
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The 2-level Haar wavelet transform of x = [4, 6, 10, 12, 8, 6, 5, 5]: each coefficient is drawn at the centre of the samples it summarises (a level-ℓ coefficient spans 2^ℓ samples). Returned: level-1 details [-1.414, -1.414, 1.414, 0]; level-2 details [-6, 2]; approximation [16, 12]. Large details mark jumps (from 6 to 10 and from 12 to 8); the approximation is a scaled local average.

```
signal_processing_ai999x.solve('haar_dwt', x='4, 6, 10, 12, 8, 6, 5, 5', levels=2)
# -> approximation = [16.000000000000004, 12.000000000000002], details = [[-6.000000000000001,
2.0], [-1.4142135623730951, -1.4142135623730951, 1.4142135623730951, 0.0]]   (SOLVED)
```

```
signal_processing_ai999x.solve('haar_dwt', x=[1, 3])
# -> approximation = [2.8284271247461903], details = [[-1.4142135623730951]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the 2-level Haar wavelet transform of x = [4, 6, 10, 12, 8, 6, 5, 5].	SOLVED	approximation = [16.000000000000004, 12.000000000000002], details = [[-6.000000000000001, 2.0], [-1.4142135623730951, -1.4142135623730951, 1.4142135623730951, 0.0]]
Compute the Haar wavelet transform of x = [1, 2, 3].	INVALID_INPUT	Haar discrete wavelet transform: the length must be a power of two.

Inverse Haar wavelet transform (one level) — `haar_idwt`

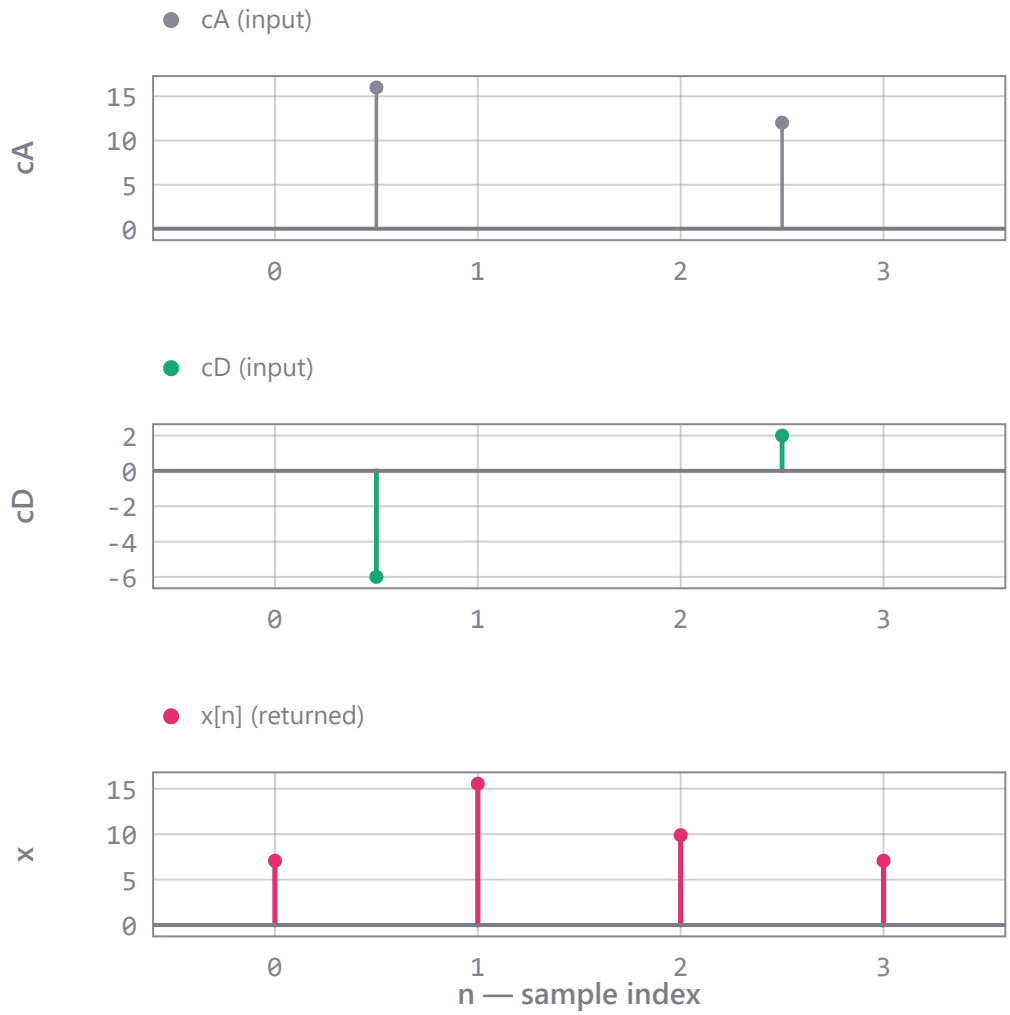
The sequence reconstructed from one level of Haar approximation and detail coefficients.

$$x[2k] = \frac{c_A[k] + c_D[k]}{\sqrt{2}}, \quad x[2k+1] = \frac{c_A[k] - c_D[k]}{\sqrt{2}}$$

input	meaning	condition
`cA`	approximation coefficients	
`cD`	detail coefficients	

output	meaning
`x`	reconstructed samples

- **Method:** x[2k] = (a + d)/sqrt 2, x[$2k+1$] = (a - d)/sqrt 2
- **Assumes:** equal lengths
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



One level of the inverse Haar transform: approximation cA = [16, 12] and details cD = [-6, 2] (inputs, drawn at the centre of the two samples each one rebuilds) give x = [7.07107, 15.5563, 9.89949, 7.07107] (returned): each pair is (cA ± cD)/√2.

```
signal_processing_ai999x.solve('haar_idwt', cA='16, 12', cD='-6, 2')
# -> x = [7.0710678118654755, 15.556349186104047, 9.899494936611665, 7.0710678118654755]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compute the inverse Haar transform with approximation coefficients [16, 12] and detail coefficients [-6, 2].	SOLVED	x = [7.0710678118654755, 15.556349186104047, 9.899494936611665, 7.0710678118654755]
Compute the inverse Haar transform with approximation coefficients [16, 12] and detail coefficients [-6].	INVALID_INPUT	Inverse Haar wavelet transform (one level): the approximation and detail lists must have the same length.